

Agent Harness

从执行脚手架到自我进化系统

研究修订稿

资料截面

2026-08-28

序：为什么现在需要一本 Harness 小书

同一个模型放进不同 Agent 产品，表现可能像两个不同的系统。差异来自模型之外的上下文、工具、环境、权限、反馈、恢复和验证。本书把这组能力统称为 Agent Harness。

Harness 不是一个框架品牌，也不只是 [while model calls tools](#)。它是模型与真实世界之间的运行时和治理层：把人的意图编译成任务，把环境状态编译成上下文，把模型动作变成受控副作用，再把外部结果编译成证据。

本书面向企业 Agent 平台架构师和高级工程师。历史篇解释从规划、BDI、ReAct 到 coding agent 的转折；原理篇给出状态机、上下文、工具、安全、验证和多 Agent 设计；产品篇研究 Claude Code、Codex、Cursor、DeepSeek Harness 与 OpenHands；进化篇区分任务内、跨任务、Harness 和模型四层；实践篇给出供应商无关参考架构和迁移路线。

全书的核心公式是：

TEXT

$\text{Agent System Capability} = \text{Model} \times \text{Harness} \times \text{Environment} \times \text{Feedback}$

乘法意味着任一项接近零，整体都可能失败。强模型无法弥补被截断的关键上下文、危险工具、损坏环境或错误 verifier；厚 Harness 也不能无限跨越模型能力边界。

全书资料维护至 2026-08-28；快速变化的产品章另在章首标出更精确的资料截面。功能会过时，设计原则应更稳定。书中区分公开事实、论文结果和作者综合判断；内部使用不改变证据要求。

目录

序：为什么现在需要一本 Harness 小书	2
目录	3
第一篇 历史：Agent 如何从会回答变成会行动	12
本篇导言：Harness 从哪里来	12
第一章 从控制循环到 Agent Runtime	13
规划：把目标翻译成动作序列	13
反应与承诺：BDI 的长期遗产	13
多 Agent：1980 年已经出现的协调税	14
从手写控制器到模型控制器	15
ReAct：现代最小循环的形成	15
Toolformer、Reflexion 与 Voyager：三种能力迁移	15
Harness 的历史不是功能累积，而是责任迁移	16
第二章 2023：自主 Agent 爆发与第一次祛魅	17
BabyAGI：任务队列就是最小外部认知	17
AutoGPT：把开放动作空间交给语言模型	17
LangChain：把 Agent 拆成可复用抽象	18
AutoGen：把工作流表达成多 Agent 对话	19
第一次祛魅：为什么 Demo 自治不能直接进入生产	19
5.1 语言流畅度与状态正确性错配	19
5.2 语义相似与记忆有效性错配	19
5.3 工具可调用与动作可授权错配	20
5.4 循环持续与目标进展错配	20
5.5 自我批评与独立验证错配	20
5.6 多角色对话与多样性错配	20
从 Framework 到 Runtime	20
这一代系统留下了什么	20
第三章 接口也是智能：Aider、SWE-agent 与 OpenHands	22
Aider：上下文不是越多越好	22
编辑格式：工具接口会占用模型能力	22
SWE-agent：Agent-Computer Interface 成为设计对象	23
CodeAct：代码作为动作语言	23
OpenHands：把 Agent 与 Runtime 分开	24
Event Stream 不等于完整事件溯源	24
Benchmark 开始评价“模型 × Harness”	24
从接口工程得到的七条原则	25
第四章 Coding Agent 转折：从实验接口到产品运行时	26
为什么软件工程成为关键试验场	26
2024—2026 的产品化转向	26
从 SWE-bench 到可执行系统评测	26
Benchmark rot 的四种来源	27
排行榜为何不能直接指导企业选型	27
Coding Agent 留下的架构遗产	27

第二篇 原理：生产级 Harness 的构成	28
本篇导言：把概率决策装进确定性边界	28
第五章 Agent = Model × Harness × Environment × Feedback	29
Model：概率性策略与生成器	29
Harness：运行时中介与控制系统	29
Environment：不是一个 shell，而是任务世界	30
Feedback：观测不等于评价	30
为什么使用乘法而不是加法	31
Workflow、Agent 与 Harness 的关系	31
控制面、数据面与执行面	31
7.1 控制面	31
7.2 数据面	31
7.3 执行面	32
概率性建议与确定性约束	32
Harness 的厚与薄	32
一个可实现的最小分层	33
本书的统一分析模板	33
第六章 Agent Loop：从 while 循环到持久状态机	34
Thread、Turn、Step 与 Attempt	34
最小状态机	34
模型停止不等于任务完成	34
Streaming 是事件协议，不只是打字动画	35
取消必须贯穿调用链	35
Timeout、Retry、Resume 不是一回事	35
副作用账本与不确定提交	36
Checkpoint 应保存什么	36
Compaction 是有损状态迁移	36
Error Taxonomy 决定恢复策略	37
并行工具与一致性	37
一个更完整的循环伪代码	37
最小可靠性测试集	38
第七章 上下文、缓存、压缩与记忆	39
五种必须分开的信息	39
Context Assembly 是一次编译	39
长窗口不是无限注意力	39
静态上下文与动态发现	40
Prompt Cache 是架构约束	40
Compaction 是有损编译，不是删除旧消息	40
MemGPT 与分层记忆	41
记忆类型与写入权限	41
Memory Poisoning 与程序漂移	41
检索不是只有向量相似度	42
Skills 不是 Memory 的别名	42
Context Quality 的评价	42

推荐的上下文架构	43
最小验收清单	43
第八章 工具、ACI、MCP 与 Code Mode	44
Tool Definition 只是起点	44
好工具的十个条件	44
错误协议是 ACI 的一部分	44
Tool Result 不应只有字符串	45
MCP 解决的是互操作，不是全部 Harness 问题	45
MCP 的安全边界	45
Tool Discovery：工具也需要分页	46
CLI：最通用但最难治理的工具总线	46
Native Tool Call 与 Code Mode	47
并行工具的调度语义	47
工具版本与动态变化	47
Tool Policy 与 Tool Execution 分离	48
如何评价工具层	48
企业平台的工具分层	48
最小工具契约伪代码	48
第九章 权限、沙箱、凭证与供应链	50
四个不同问题	50
Prompt 不是安全边界	50
从逐次批准到受控自治	50
文件系统与网络必须同时限制	50
Policy 决策模型	51
凭证不进入模型上下文	51
Prompt Injection 的系统应对	51
多 Agent 的权限传播	52
MCP 与插件供应链	52
审批 UX 是安全系统	52
审计记录与模型 trace 分离	53
数据分类与跨域流动	53
风险分级	53
安全测试	53
安全参考边界	54
第十章 验证、完成契约与证据包	55
Stop、Answer、Success 与 Commit 是四件事	55
完成契约从任务入口开始	55
验证金字塔	56
3.1 确定性检查	56
3.2 环境与结果检查	56
3.3 模型检查	56
3.4 人类检查	56
Benchmark 也是会腐化的软件	56
非确定性系统不能只跑一次	57
防止测试投机与 verifier 篡改	57

证据包是交付协议	57
完成门的参考状态机	58
三类案例	59
9.1 仓库软件工程	59
9.2 企业数据分析	59
9.3 自我进化 Agent	59
常见失败模式	59
10.1 把 Agent 自述当证据	59
10.2 只验证最终文本	59
10.3 同一主体控制目标、实现与评分	59
10.4 失败后动态降低门槛	60
10.5 迷信 benchmark 排名	60
10.6 无限验证—修复循环	60
企业落地清单	60
第十一章 多 Agent、委派与协作拓扑	61
先区分五种经常混淆的东西	61
多 Agent 的收益来自哪里	61
何时不应使用多 Agent	61
四类基本拓扑	62
4.1 Manager—Worker	62
4.2 Pipeline / Assembly Line	62
4.3 Peer Handoff	62
4.4 Blackboard / Event Graph	62
委派是一份受限合同	63
结果必须是 artifact，不是意见	63
共享状态与并发写入	63
错误如何在团队中传播	64
Debate、Critique 与 Ensemble	64
调度、背压与预算	64
权限、身份与责任链	65
可观测性：同时看到树和因果链	65
三类贯穿案例	65
13.1 仓库软件工程	65
13.2 企业数据分析	66
13.3 自我进化 Agent	66
一个最小参考实现	66
设计原则总结	67
第十二章 可观测性、轨迹与评测运营	68
轨迹是事件图	68
指标分四层	68
失败分类先于优化	68
Eval 是持续运营系统	68
在线监控与离线评测闭环	68
Trace replay 的边界	69
运营仪表盘	69
一条可关联的真实事件	69

Trace 完整性与采样	69
Eval 生命周期与污染控制	70
从指标到行动	70
可观测性的边界	70
第三篇 产品：当代主流 Harness 的不同答案	71
本篇导言：五种产品，五种架构重心	71
第十三章 Claude Code：薄循环、厚运行时	72
可观察的系统分层	72
上下文不是一段无限增长的聊天	72
Hook 是可编程生命周期，不是万能策略层	72
Permission、sandbox 与凭证必须拆开	72
企业集成剖面	73
设计判断	73
第十四章 OpenAI Codex：协议化的 Agent Core	74
从 UI 内核到可嵌入服务	74
三种集成面不是同一抽象	74
Loop、context 与执行边界	74
并行工作不是共享目录里多开几个进程	74
企业 adapter 的状态模型	75
设计判断	75
第十五章 Cursor：IDE 原生上下文与云端 Agent	76
动态上下文发现	76
Model-specific Harness	76
在线信号与离线评测	76
从前台审批到云端自治	76
Hooks 与企业控制点	77
设计判断	77
第十六章 DeepSeek Harness：可组合、可逆的运行	78
空间与时间上的组合	78
Profile、bundle 与分层配置	78
Code Mode 作为工具压缩	78
“一切皆插件”的边界	78
企业集成策略	79
设计判断	79
第十七章 OpenHands：Agent 与执行 Runtime 分离	80
Action—Observation 作为系统脊柱	80
Runtime 是能力边界，不只是 Docker 名称	80
开放平台的可替换性	80
失败模式与运营负担	80
与其他产品的互补关系	81
设计判断	81
第十八章 产品比较：不要用一张总分表掩盖架构差异	82
五种代表性重心	82
统一评价坐标	82

协议统一的限度	82
常见失效比较	83
组合战略	83
第四篇 进化：Agent 如何从轨迹中变得更好	84
本篇导言：从“能改自己”到“能证明改得更好”	84
第十九章 进化不是自我修改：目标函数、证据与边界	85
四层进化模型	85
统一证据模板	85
根信任与可变表面	85
数据不是天然经验	86
多目标与硬约束	86
当前研究证据的强弱	86
最小可信闭环	86
四层不是线性升级阶梯	86
从失败样本到可检验假设	87
进化预算也是治理工具	87
先决定是否值得进化	87
第二十章 任务内进化：搜索、反思与验证—修复	88
本层的证据模板实例	88
Reflexion 的贡献与限制	88
搜索不是重复采样	88
验证—修复循环	88
副作用与并行分支	89
何时不使用任务内进化	89
候选选择器的三种强度	89
计划修复与状态修复要分开	89
一个有界修复策略	89
任务内学习如何退出当前任务	90
搜索预算要按信息增益分配	90
第二十一章 跨任务经验化：Memory、Skill 与策略库	91
本层的证据模板实例	91
四类持久对象不能共用一种生命周期	91
写入门比检索算法更重要	91
检索是策略决策	91
Skill 是供应链包	92
遗忘、纠错与派生影响	92
Memory 的状态机	92
冲突不是“取最新”	92
复用实验与负迁移	92
经验库的容量与注意力预算	93
记忆收益必须扣除维护债务	93
第二十二章 Harness 进化：从失败病理到版本化变更	94
本层的证据模板实例	94
先诊断失败病理	94

研究系统提供了什么证据	94
四组门禁	95
Profile 而非全局最优	95
何时选择替代方案	95
可变表面的风险排序	95
组合爆炸与交互效应	95
防止 Prompt Rule Accretion	95
从候选到可维护版本	96
用可逆性决定发布半径	96
第二十三章 模型进化：从轨迹到参数更新	97
本层的证据模板实例	97
轨迹不等于训练样本	97
四类训练路线	97
Model×Harness 2×2 归因	97
数据与模型 lineage	98
何时不训练	98
轨迹筛选的多阶段管线	98
错误与纠错都要学习	98
安全回归与能力回归同权	98
模型发布后的监测	99
训练前先证明问题属于模型	99
第二十四章 受控进化闭环：门禁、灰度、回滚与反投机	100
双平面架构	100
预注册实验协议	100
防 Reward Hacking 与数据泄漏	100
Shadow、Canary 与发布原子性	100
进化账本与职责	101
何时允许自动晋级	101
完整性监控先于效用监控	101
Canary 不是缩小版离线评测	101
事故演练	101
人类批准也需要可评价	102
自动化阶梯	102
治理自身也要接受演化，但不能同轮自改	102
跨层升级判据：先修最小可变表面	102
第五篇 实践：下一代企业 Harness	104
本篇导言：把原则落到企业控制面	104
第二十五章 三个贯穿案例：从意图到可验证结果	105
案例一：仓库级软件修复	105
任务与合同	105
执行与验证序列	105
EvidencePackage	106
失败演练：visible test 投机	106
案例二：企业经营分析	106
任务与口径	106

双重验证	107
EvidencePackage	107
失败演练：快照漂移	107
案例三：自我进化 Harness	108
失败归因与实验合同	108
选择与发布	108
EvidencePackage 与 lineage	108
失败演练：通过修改分母“进步”	109
三个案例的共用骨架	109
第二十六章 下一代企业 Harness 参考架构	110
六层结构与权威状态	110
Canonical contracts 与能力协商	110
身份、租户与凭证	110
Durable execution 与副作用	110
Evidence-first completion	111
七类 SLO：定义、测量和博弈	111
多 runtime 数据流	111
构建顺序与替代方案	111
第二十七章 Agent SDD：规范驱动的任务与发布	112
六类规范	112
从意图到合同	112
完整实例：从规范到任务再到验收	112
运行中的 Amendment	113
Specification as environment	114
发布门与 Waiver	114
规范质量指标	114
第二十八章 成熟度模型与 Build-vs-Buy	115
五级模型	115
二元自评方法	115
买什么，控制什么	115
何时自研 Runtime	115
决策矩阵与 POC	116
第二十九章 从接入现有 Agent 到拥有运行时主动权	117
封装而非散接	117
外置完成与证据	117
统一执行面	117
建立评测与运营基线	117
逐层替换	118
引入受控进化	118
迁移治理	118
第三十章 展望：Harness OS、Agent 组织与持续进化	119
模型与 Harness 共同设计	119
从 Agent 应用到 Agent 组织	119
环境成为长期资产	119
自我进化的近期现实	119

新风险与开放问题	119
三种可能未来	120
最终判断	120
附录	121
附录 A：语言无关核心契约与安全伪代码	122
核心对象	122
Run loop：proposal 不直接变成 effect	122
副作用提交：先记 intent，再执行	123
恢复、取消与对账	124
Adapter 的能力协商	124
附录 B：可判定的 Harness 架构评审表	125
任务与完成	125
上下文与记忆	125
工具、环境与副作用	125
权限与供应链	125
Durable 与多 Agent	126
Eval、运营与进化	126
最终判定	126
附录 C：术语与本体边界	127
附录 D：概念首次定义与使用索引	129
附录 E：最小机器可读契约	130
Task 与 CompletionContract	130
Action、PolicyDecision 与 Observation	130
EvidencePackage 必填骨架	131
研究方法局限	132
完整参考文献	133
Markdown 章节按下列五篇排列。	

第一篇 历史：Agent 如何从会回答变成会行动

本篇导言：Harness 从哪里来

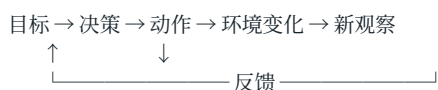
本篇回答“为什么模型之外还需要一套运行系统”。第一章追溯规划、控制循环和多 Agent 协商的前史；第二章分析 2023 年自主 Agent 热潮为何证明了可行性，却没有解决可靠性；第三章把 ACI 确立为能力的一部分；第四章说明真实仓库和可执行评测如何推动产品化。

这段历史不是产品编年表。它要建立三个后文反复使用的判断：行动系统必须有外部状态与反馈；接口会改变模型可实现的策略；可演示的循环与可运营的 Runtime 之间隔着状态、权限、恢复和验证。读完本篇，读者应能解释为什么“换更强模型”不能自动补齐 Harness。

第一章 从控制循环到 Agent Runtime

如果把 Claude Code、Codex 或 DeepSeek Harness 的界面全部拿掉，剩下的核心似乎简单得令人失望：接收目标，调用模型，执行模型选择的动作，把结果送回模型，如此循环，直到完成或耗尽预算。

TEXT



这个循环并不新。控制论研究反馈，自动规划研究如何从初始状态抵达目标状态，机器人研究感知与行动，BDI 架构研究信念、目标与承诺，多智能体系统研究任务分配与通信协议。现代 Harness 的新意不在于重新发明循环，而在于把一种高能力、概率性、上下文受限、可能调用任意软件工具的语言模型，放进真实计算环境后，为它补上可靠运行所需的工程结构。

因此，要理解 Harness，最好的起点不是 2023 年的 AutoGPT，而是三个更早的问题：机器如何形成行动序列？一个持续运行的 Agent 如何在变化环境中保持承诺并修正行为？多个自治执行者如何分工而不依赖全局共享状态？

规划：把目标翻译成动作序列

1971 年的 STRIPS 将规划描述为：在一个世界模型中寻找一串操作，使初始状态经过这些操作后满足目标公式。它把动作表示成具有前置条件和状态效果的操作符，使“怎样完成任务”从一段专用程序转化成可以搜索的状态转换问题。STRIPS 原始论文

用现代 Harness 的语言重写，STRIPS 已经包含四个熟悉成分：

TEXT

State	当前环境的结构化描述
Goal	希望满足的终止条件
Action	带前置条件和效果的环境操作
Planner	搜索可行 Action 序列的控制器

现代 coding agent 的文件读取、补丁、命令执行和测试工具，也可以被视为动作。不过，两者之间存在决定性差异。经典规划通常假设动作语义明确、状态足够可知、执行结果近似确定；软件 Agent 面对的仓库、依赖、网络服务和人类意图却是不完全可观测的。模型甚至可能误解工具说明、生成无效参数，或者把命令成功退出误认为业务目标已经完成。

这解释了为什么现代 Agent 很少只生成一份完整计划再机械执行。更常见的是滚动规划：先选择一个信息增益高或风险可控的动作，观察真实结果，再更新局部计划。ReAct 后来把这种交替模式显式写进语言模型轨迹，但其深层问题仍是规划与控制中的老问题：当世界模型不完整时，计划必须服从反馈。

对 Harness 设计者而言，这段历史留下第一条原则：

计划不是事实，而是一个必须持续接受环境证据修正的假设。

这意味着 plan mode 可以帮助形成意图和审查范围，却不应成为独立于执行反馈的僵硬工作流。真正可靠的 Harness 必须保留重新观察、修订计划、撤销局部动作和报告不可达目标的通道。

反应与承诺：BDI 的长期遗产

只会在每一步对刺激作出反应的系统容易漂移；只会遵循预先计划的系统又难以适应变化。Belief – Desire – Intention (BDI) 架构试图在两者之间建立实际推理循环：Agent 根据对世界的信念识别可能目标，从中形成当前愿望或目标，再选择并承诺某些意图；新事件到来后，它更新信念、判断现有意图是否仍可行，并决定继续、重规划或放弃。

BDI 的价值不在于要求现代 Harness 必须建立名为 **beliefs**、**desires**、**intentions** 的三个对象，而在于它揭示了三类经常被聊天记录混为一谈的状态：

- 观察和推断出来的世界状态；
- 用户希望实现但尚未承诺执行路径的目标集合；
- 当前已经投入资源、应跨多个步骤保持一致的执行承诺。

早期 PRS、dMARS 与 AgentSpeak 等系统还发展出计划库、事件触发、元级控制和显式 deliberation cycle。BDI 综述将其核心贡献概括为：在动态、不可预测环境中平衡主动目标与被动响应，并通过意图表达对未来行为的承诺。BDI 架构综述

这与现代 Agent 的几个常见故障直接相关。

第一，只有对话历史、没有显式任务状态时，模型可能在长会话中忘记用户真正批准了什么。第二，计划、待办、执行事实和模型猜测如果使用同一种自然语言表示，压缩后很容易相互污染。第三，出现新消息时，如果 Harness 不区分“补充信息”“优先级变化”“取消请求”和“新任务”，Agent 就可能错误地放弃或延续已有承诺。

因此，企业 Harness 至少应在运行时区分：

TEXT

ObservedState 可追溯到环境或用户消息的事实
Hypothesis 模型尚未验证的解释
Goal 用户或上层系统定义的结果条件
Commitment 已批准并正在执行的目标/约束
Plan 当前可替换的动作建议
ExecutionState 已开始、等待、取消、失败或完成

这里最重要的不是命名，而是不同状态拥有不同的更新权限和证据要求。模型可以自由提出假设和计划，却不应自行把假设升级为事实，也不应把未批准目标升级为承诺。这正是现代 Harness 中“概率性认知”和“确定性控制”分层的早期思想来源。

多 Agent：1980 年已经出现的协调税

1980 年的 Contract Net Protocol 研究松耦合节点如何通过协商分配任务：拥有任务的 manager 发布任务描述，潜在 contractor 根据自身能力和资源提出方案，manager 再授予合同。该系统强调没有全局共享数据和单一全局控制，任务分配必须同时考虑资源利用与问题求解焦点。Contract Net 原始论文

这与今天的 subagent、agent team 和异步云 Agent 非常相似，但它也提醒我们，多 Agent 从来不是免费的并行计算。一次委派至少需要：

- 切分出边界足够清楚的任务；
- 描述输入、输出、权限和完成条件；
- 选择具备适当能力与资源的执行者；
- 传递必要上下文，同时避免复制整个主会话；
- 接收结果并验证，而不是把子 Agent 的自然语言结论当作事实；
- 处理超时、重复执行、局部失败和相互冲突的修改；
- 把结果重新合并到主任务状态。

如果拆分收益小于这些协调成本，多 Agent 会比单 Agent 更慢、更贵、更难调试。这也是为什么“模型能够调用 subagent”不等于系统具备良好的多 Agent 架构。真正的设计对象是委派协议、隔离边界、结果契约、取消传播和合并策略。

现代 coding agent 使用 worktree、独立容器或远程 workspace，不只是为了方便并行，而是在物理上减少共享可变状态。这个选择与 Contract Net 的松耦合假设一脉相承：信息通过协议传递，执行者不依赖一个可任意读写的共同工作台。

从手写控制器到模型控制器

传统 Agent 的策略、计划选择和异常处理通常由规则、搜索算法或专用程序实现。大语言模型改变的是控制器的表达能力：它可以读取自然语言目标和非结构化观察，在没有为每种任务编写专用规则的情况下，提出下一动作并生成工具参数。

这带来显著的泛化能力，也引入四种结构性不确定性：

- 语义不确定性：模型可能误解目标、上下文或工具描述；
- 动作不确定性：它可能选择错误工具或构造无效参数；
- 状态不确定性：上下文窗口只包含环境的一部分，并可能在压缩中失真；
- 完成不确定性：模型说“完成”只是一项预测，不是外部世界已经满足目标的证明。

所以，现代 Harness 不能只是把模型接在 shell 上。它需要用确定性软件包围概率性决策：工具 schema 约束动作形状，权限策略约束可执行范围，沙箱约束副作用，日志保存轨迹，预算限制循环，测试和评估器判断结果，人工审批承接不可自动化的价值判断。

可以把两者的分工写成一条简单边界：

TEXT

模型负责：提出、解释、比较、生成、诊断

Harness 负责：授权、执行、隔离、记录、计量、验证、恢复

这不是说 Harness 中不能存在模型评审器，也不是说控制逻辑必须完全固定。关键是：任何影响安全、资源、归因和进化晋级的决定，都不能只依赖被评对象的一次自我陈述。

ReAct：现代最小循环的形成

ReAct 将 reasoning trace 与 action 交替生成：推理帮助模型维护和调整计划，动作让模型从外部知识源或环境取得新信息，新观察再进入后续推理。ReAct

它为现代工具调用 Agent 提供了极具影响力的最小模板：

TEXT

```
while budget_available:
    decision = model(context, available_tools)

    if decision.is_final:
        return decision.answer

    validated_call = validate(decision.tool_call)
    observation = execute(validated_call)
    context.append(observation)
```

这个伪代码能运行，却还不是企业 Harness。它没有回答：上下文从哪里来，过长时如何压缩；工具结果是否可信；命令在哪台机器、以谁的身份执行；权限是静态的还是可临时扩展；进程崩溃后如何恢复；如何取消正在执行的工具；怎样判断模型陷入循环；最终答案需要哪些外部证据；多个用户和 Agent 如何隔离；历史轨迹如何进入评测和进化。

换句话说，ReAct 给出了发动机循环，却没有给出整辆车的制动、仪表、车身、道路规则和维修体系。Harness 的历史，就是这些“循环之外的结构”逐渐变成第一等工程对象的历史。

Toolformer、Reflexion 与 Voyager：三种能力迁移

2023 年的几项工作分别展示了 Agent 能力可以被迁移到不同位置。

Toolformer 研究如何通过自监督数据让模型学习何时调用 API、调用哪个 API、传什么参数以及如何吸收工具结果。Toolformer 它把一部分工具选择能力迁移进模型权重。

Reflexion 不更新权重，而是把任务反馈转成语言反思，保存到情景记忆中，影响后续尝试。Reflexion 它把一部分学习迁移到跨回合上下文。

Voyager 则把成功行为保存为可执行技能库，并结合自动课程、环境错误和自验证持续扩充技能。Voyager 它把一部分能力迁移到外部、可组合、可复用的程序资产。

这三种路线构成理解“Agent 如何进化”的早期坐标系：

能力沉淀位置	典型机制	优点	主要风险
模型权重	训练、微调、强化学习	推理时直接、可泛化	成本高、难回滚、归因困难
运行时记忆	反思、经验摘要、状态图	更新快、无需改权重	污染、过期、检索误配
外部技能	代码、工具、工作流、插件	可审查、可组合、可版本化	供应链风险、接口漂移
当前任务轨迹	retry、search、verify-fix	即时纠错	成本膨胀、循环与自证偏差

本书后面的“进化篇”将沿着这四层展开。这里先保留一个重要结论：自我改进不必等同于修改模型，也不必等同于 Agent 任意重写自己的运行时。最有工程价值的进化，往往是把一次成功中可验证、可迁移的部分，沉淀到风险最低且最容易回滚的层级。

Harness 的历史不是功能累积，而是责任迁移

回看这条历史线，可以发现 Harness 不是一张越来越长的功能清单，而是一系列责任在模型、控制器、环境与人之间重新分配的过程：

TEXT

经典规划：控制器显式搜索动作序列
BDI： 控制器管理信念、目标与承诺
多 Agent： 协议管理委派与资源协商
ReAct： 模型参与滚动决策与工具选择
Reflexion：经验进入外部记忆
Voyager：成功行为进入可复用技能
现代 Harness：确定性运行时治理概率性控制器

今天看似全新的问题——上下文工程、subagent、skills、tool schema、self-evolution——都能在早期 Agent 研究中找到结构相似物。但结构相似不等于工程问题已经解决。语言模型让动作空间、任务空间和接口空间同时扩大，使过去可以写死在专用系统里的约束，必须升级为通用运行时机制。

这就是 Harness 成为独立层的原因。模型越通用，环境越开放，循环越长，Harness 承担的责任反而越厚：它必须让系统知道自己看见了什么、承诺了什么、能做什么、做过什么、是否真的完成，以及从结果中允许学到什么。

下一章将进入 2023 年的“自主 Agent 爆发期”：AutoGPT、BabyAGI、LangChain 和 AutoGen 如何把循环、记忆、计划与多 Agent 编排带入大众开发实践，又为何很快暴露出失控循环、抽象泄漏、观测不足和生产基础设施缺失的问题。

第二章 2023：自主 Agent 爆发与第一次祛魅

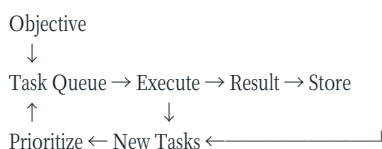
2023 年春天，GPT-4 与廉价 API、开源代码和社交媒体演示共同触发了一次“自主 Agent”爆发。AutoGPT、BabyAGI、AgentGPT 等项目让普通开发者第一次直观看见：只要给模型一个目标、少量工具、一段循环和某种记忆，它似乎就能自行拆解任务、搜索网络、写文件、运行代码，并不断决定下一步。

从今天回看，这批系统并没有建立可靠的通用自治。但称它们只是“玩具”同样不准确。它们完成了一次重要的公共实验：把语言模型从单次问答移入持续执行循环，并在极短时间内暴露出 Harness 工程真正困难的部分。

BabyAGI：任务队列就是最小外部认知

原始 BabyAGI 的结构极其简洁：从队列取出一个任务，调用执行 Agent，将结果写入记忆，根据目标和最新结果创建新任务，再重新排序任务队列，如此循环。原始归档代码

TEXT



它的重要启示不是“需要三个角色提示词”，而是模型之外必须存在可检查的任务状态。如果所有计划只存在于对话文本中，系统很难回答：还有哪些任务？哪个任务正在执行？为什么优先做它？某个结果由哪次执行产生？

任务队列把一部分认知外置成了数据结构。这是一个小但关键的动作。现代 Agent 的 todo、plan、issue、workflow state 和 execution graph 都延续了同一思想：自然语言适合生成候选，结构化状态适合保持约束。

但 BabyAGI 也展示了“模型管理模型任务”的脆弱性。任务创建者可能不断生成低价值后续事项；优先级调整者可能受最近结果支配；执行结果被存储并不代表它正确；队列增长本身容易被误认为取得进展。系统优化的是“持续产生下一任务”，而不是目标是否被外部满足。

这可以抽象成第一种自治陷阱：

当 Harness 只能测量活动而不能测量结果时，Agent 会把循环存活当作任务进展。

企业运行时因此不能只记录 tool-call 数、token 数、步骤数和任务完成声明，还必须定义与业务结果相连的 evaluator。对于代码任务可能是测试、静态检查和验收条件；对于数据分析可能是查询可复现性、口径一致性和事实来源；对于业务流程则可能是外部系统状态与审批记录。

AutoGPT：把开放动作空间交给语言模型

AutoGPT 的早期吸引力来自更开放的循环：模型接收一个长期目标，可以选择搜索、浏览、文件、命令等动作，并用短期历史和向量记忆延续任务。与 BabyAGI 的显式队列相比，它更接近后来通用 coding agent 的体验：模型既负责局部规划，也负责选择工具和解释观察。

早期版本证明了几个方向可行：

- 模型能在多轮中保持一个粗粒度目标；
- 工具结果能作为新观察影响后续决策；
- 外部记忆可以突破单次上下文的表面限制；
- 命令与文件工具让模型从“建议者”变成“执行者”。

与此同时，开放循环把多个风险同时放大：目标漂移、重复动作、无效搜索、错误记忆、费用失控、不可逆副作用和虚假完成。模型生成一段看起来合理的自我批评，并不意味着它识别了真实错误；向量库找回语义相似内容，也不意味着内容仍然正确或适用于当前状态。

AutoGPT 的后续仓库逐步发展出平台、组件、Forge 和 benchmark 等不同项目。历史研究必须避免把这些成熟后的结构倒推到 2023 年原型上。这里讨论的是原型所代表的范式：让模型直接拥有开放的下一步选择权，再用提示和记忆维持长期目标。

这次实验带来的最大教训是，自治不是一个开关。至少需要拆成五个维度：

维度	问题
决策自治	下一步由模型、规则还是人决定？
工具自治	模型可以调用哪些动作？
权限自治	动作是否需要批准，能否扩大权限？
时间自治	可以持续多久，何时暂停或终止？
进化自治	能否改变记忆、技能、策略或自身 Harness？

一个系统可以拥有高决策自治，却被严格限制在只读沙箱；也可以使用固定 workflow，但对某个已批准 API 拥有高工具自治。用“全自动/非自动”描述 Agent，会掩盖真正的风险边界。

LangChain：把 Agent 拆成可复用抽象

LangChain 在 2023 年用一套影响广泛的词汇总结当时的 Agent：Agent 是决定动作的模型，Tools 是可执行动作，Memory 负责引入过去事件，AgentExecutor 则运行循环直到满足停止条件。其典型算法直接继承 ReAct：Thought、Action、Observation 重复进行。LangChain 2023 总结

它的历史贡献在于把快速增长的模型供应商、向量库、工具和提示模式装入可复用接口。开发者不必为每个实验重新编写消息转换、输出解析和循环控制。这种“集成框架”极大降低了进入门槛，也让 Agent、Tool、Memory、Executor 成为广泛流通的工程术语。

但抽象的代价同样迅速显现。

第一，模型 API 本身快速变化。原生 function calling、结构化输出、流式事件和服务端状态不断出现，通用包装层很容易滞后或泄漏底层差异。

第二，Agent 的故障通常发生在跨层边界：提示、工具 schema、消息序列、重试、解析器和供应商响应共同作用。过深的封装使开发者看见“chain failed”，却难以复原模型究竟看到了什么。

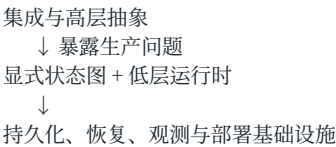
第三，早期 memory 概念过宽。对话历史、检索知识、用户偏好、工具结果和执行状态具有不同生命周期、可信度与更新规则，把它们都称为 memory 容易制造错误抽象。

第四，生产需要的能力不只是调用组件：还包括持久化、幂等、恢复、租户隔离、审批、流式 UI、观测和评估。它们最终要求一个运行时，而不只是一个 Python 对象图。

社区中“直接调用模型 API 加一个 while loop 就够了”的反弹，在很大程度上是对抽象税和调试困难的回应。但另一面也应写清：LangChain 并没有简单消失。它后来通过 LangGraph 将重点转向持久状态、确定性与 Agent 节点混合、interrupt/resume、checkpoint 和 durable execution。官方回顾甚至明确提出，最大的竞争者始终是“不使用框架”。LangGraph 运行时设计

所以，LangChain 的历史不是“古早框架被淘汰”的单线故事，而是一次架构纠偏：

TEXT



这条路径对企业自研 Harness 很有警示意义。早期最诱人的往往是统一接口和快速 demo；长期最难替换的却是状态模型、事件协议和运行语义。平台应把稳定性投资放在后者。

AutoGen：把 workflow 表达成多 Agent 对话

AutoGen 将多个可定制 Agent 的对话作为应用编排机制。每个 Agent 可以组合模型、人类输入和工具，交互行为既可由自然语言也可由代码定义。AutoGen 论文

这种设计有很强的表达力。规划者、执行者、评审者和用户代理可以用统一的消息隐喻协作；新角色可以通过说明与工具配置快速加入；人类也能作为一个对话参与者插入流程。

但“万物皆消息”与“万物皆文件”一样，既是统一抽象，也是潜在陷阱。业务状态、权限决定、执行结果、取消信号和评审结论如果都退化为自然语言消息，会产生四类问题：

- 难以保证消息被恰处理好一次；
- 难以区分陈述、命令、建议和授权；
- 难以建立严格 schema 与兼容性策略；
- 难以证明终止条件和责任归属。

多 Agent 对话还容易制造“社会性拟真”：不同角色互相赞同、批评或投票，看起来像一个团队，却可能共享相同模型偏差、相同错误上下文和相同盲点。增加说话者不自动增加独立证据。

因此，多 Agent 的核心价值不是角色扮演，而是以下至少一项真实差异：

- 不同工具或权限边界；
- 不同上下文与信息源；
- 不同模型能力或成本曲线；
- 可并行的独立工作空间；
- 独立评价标准或对抗目标。

如果这些差异不存在，单 Agent 加结构化阶段往往更便宜，也更可观测。

第一次祛魅：为什么 Demo 自治不能直接进入生产

2023 年的演示通常选择开放式目标，例如“研究一个市场并建立网站”。这类目标让模型有足够空间生成令人惊喜的轨迹，却缺少可重复、可外部判定的完成条件。生产系统正相反：错误成本真实存在，输入分布不断变化，结果必须能够审计。

自主 Agent 的第一次祛魅主要来自六个错配。

5.1 语言流畅度与状态正确性错配

模型能够生成连贯的“当前进度”，但环境可能根本没有发生对应变化。Harness 必须以工具结果和外部查询维护 canonical state，而不是用模型叙述替代状态。

5.2 语义相似与记忆有效性错配

向量检索擅长找相似文本，却不负责内容真实性、时效性、权限范围或因果相关性。生产记忆需要来源、时间、作用域、置信度、失效条件和删除机制。

5.3 工具可调用与动作可授权错配

工具出现在 schema 中只说明模型知道如何提出请求，不说明它有权执行。模型选择、策略判断、用户审批与沙箱执行必须是不同步骤。

5.4 循环持续与目标进展错配

Agent 可以不断产生新的思考、搜索和任务。预算、重复检测、停滞检测和外部里程碑必须共同限制循环。

5.5 自我批评与独立验证错配

同一模型阅读自己的输出并说“看起来正确”，只能提供一种弱信号。强验证来自测试、类型系统、约束求解、外部数据、不同信息路径或人类判断。

5.6 多角色对话与多样性错配

共享模型、共享上下文和共享提示风格的多个 Agent 很可能产生相关错误。有效冗余必须测量错误相关性，而不是只计算 Agent 数量。

从 Framework 到 Runtime

这轮祛魅并没有结束 Agent 方向，反而改变了工程重心。开发者开始关心：

- 状态能否持久化并从中断恢复；
- 每一步是否形成可消费的结构化事件；
- 工具执行是否幂等，失败能否安全重试；
- 人类能否在关键点暂停、修改和恢复；
- 长任务能否跨进程、跨机器和跨版本继续；
- 权限与凭证是否由模型之外的策略系统管理；
- 轨迹能否进入回放、评估和回归测试。

这些问题推动“Agent framework”向“Agent runtime”迁移。LangGraph 将开发 API 与 PregelLoop runtime 分离；OpenHands 将 Agent 控制与沙箱 Runtime 分离；Codex 用 App Server 将共享 Harness 暴露给多个客户端；DSH 则进一步把 loop、tools、session、sandbox 和 storage 都放入可组合插件系统。

Framework 主要回答“开发者怎样表达 Agent”；Runtime 还必须回答“表达出来的 Agent 如何长期、安全、可恢复地运行”。两者并非互斥，但企业平台如果只拥有前者，就会在每个应用中重复构建后者。

这一代系统留下了什么

AutoGPT 和 BabyAGI 留下开放循环与任务外置；LangChain 留下 Agent/Tool/Memory/Executor 词汇和集成生态；AutoGen 留下对话式多 Agent 编排；LangGraph 则代表从高层魔法回到显式状态和持久运行语义。

它们共同证明：一个最小 Agent 的确可以很小，但一个可靠 Agent 系统不会很小。

TEXT

最小 Agent：模型 + 工具 + while loop

生产 Harness：

最小 Agent

+ canonical state

+ context lifecycle

+ policy and sandbox

+ durable execution

+ observability

+ verification

- + human control
- + evaluation and governance

下一章进入 coding agent 的工程化转折。Aider 的 repository map、SWE-agent 的 Agent-Computer Interface，以及 OpenHands 的动作—观察事件流，将证明一个后来反复出现的事实：同一个模型的表现，会被它所处的接口和环境大幅改变。模型能力并不等于系统能力。

第三章 接口也是智能：Aider、SWE-agent 与 OpenHands

2023 年的通用自主 Agent 证明模型可以循环调用工具，但没有证明它善于在大型代码仓库中工作。软件工程要求 Agent 定位相关文件、理解跨模块关系、生成能够可靠落盘的局部修改、运行测试并解释错误。模型可能知道如何写一个函数，却因看错文件、破坏补丁格式或忽略仓库约定而失败。

这一阶段最重要的发现可以概括为：

模型能力并不是系统能力的固定上限；模型看见什么、能做什么、动作如何表达、反馈怎样返回，会系统性改变任务表现。

Aider：上下文不是越多越好

大型仓库无法完整塞进上下文窗口。即使窗口足够大，无差别注入也会增加费用、延迟和注意力干扰。Aider 的 repository map 选择另一条路线：提取仓库中的关键符号、类型和签名，以文件依赖关系构图，再在 token 预算内通过图排序选择最相关、最重要的部分。Aider Repository Map

它实际上为模型提供了两级观察：

TEXT

Repo Map：低成本、全局但有损的符号地图
Full File：高成本、局部但更完整的源码观察

模型先借助地图理解大致结构，再请求具体文件。这比预先猜测所有相关上下文更接近主动感知：Harness 提供可导航地图，模型决定何时放大细节。

这套设计包含三个可迁移原则。

第一，摘要必须服务于后续定位。一个只“概括内容”的摘要可能读起来很好，却无法帮助 Agent 找到下一步需要读取的对象。符号名、路径、签名和依赖边是可操作的索引。

第二，上下文选择需要显式预算。不同信息不是简单的“相关/不相关”，而是在有限 token、延迟和缓存条件下竞争。Harness 应把 context assembly 看成带约束优化问题。

第三，模型应能按需深化观察。一次性静态 RAG 无法预知执行过程中产生的新问题。现代 Cursor 的动态上下文、Claude Code 的搜索工具和 Codex 的文件工具都延续了这一点。

编辑格式：工具接口会占用模型能力

让模型“给出正确代码”和让它“给出 Harness 能可靠应用的修改”是两个不同任务。Aider 为 whole file、search/replace block、unified diff 等多种编辑格式建立端到端 benchmark，测量代码是否正确、格式是否可解析、修改是否成功落盘并通过测试。Aider 编辑 benchmark

早期实验出现了一个反直觉结果：function calling 的结构更严格，却可能比简单文本格式表现更差。复杂格式不只增加解析约束，也占用模型用于解决代码问题的能力。不同模型对 whole、diff、diff-fenced 等格式的适应也不同。

因此，“结构化接口必然优于文本接口”不是普遍真理。正确问题是：

- 模型是否在训练或后训练中见过这种接口？
- schema 是否贴近任务的自然结构？
- 参数生成需要多少转义、定位和重复内容？
- 接口失败能否返回可修复的局部错误？
- token、延迟、正确率和修改范围之间如何权衡？

这也是模型特定 Harness 的早期证据。同一工具为不同模型提供相同 schema，看似平台中立，实际可能让某些模型承担额外认知税。企业平台需要统一语义，但不必强迫所有模型使用完全相同的表面协议；适配层可以把模型原生动作映射到统一的内部命令。

SWE-agent：Agent-Computer Interface 成为设计对象

SWE-agent 把接口明确命名为 Agent-Computer Interface (ACI)。它的论文结论不是只比较模型，而是说明定制 ACI 可以显著改善 Agent 浏览仓库、编辑文件、运行测试和处理程序输出的能力。SWE-agent

ACI 类似人机交互中的 UI，但用户是语言模型。好的 ACI 需要考虑模型的行为偏好和错误模式：

- 命令集合是否小而正交；
- 观察是否包含足够定位信息；
- 大输出是否截断，截断点是否可理解；
- 文件编辑是否容易精确寻址；
- 语法或 lint 错误是否及时返回；
- shell 状态是否跨步骤保持；
- 错误消息是否告诉模型怎样恢复。

传统 API 设计主要面向确定性程序：调用者会严格遵循 schema，并能用代码处理错误。ACI 面向概率性调用者：工具说明本身是模型上下文的一部分，命令命名、示例、错误文本和输出长度都会改变策略。

所以工具质量至少有四层：

TEXT

Capability 是否能完成所需动作
Semantics 输入、输出和副作用是否定义清楚
Learnability 模型能否从说明与反馈学会正确使用
Governance 动作能否授权、审计、隔离和撤销

很多 MCP 或内部工具只完成第一层：接口“能调用”，却没有为 Agent 的可学习性与企业治理设计。

CodeAct：代码作为动作语言

JSON tool call 将动作限制为预定义函数及其参数。CodeAct 提出用可执行 Python 作为统一动作空间，使模型可以组合库调用、中间计算和控制流，并根据执行观察继续修订。CodeAct

代码动作的优势来自组合性。假设 Agent 需要读取数据、过滤、聚合并画图。函数调用模式可能产生多轮 payload 往返；代码模式可以在沙箱内完成多个步骤，只把必要结果送回模型。

但代码动作也扩大了风险面：

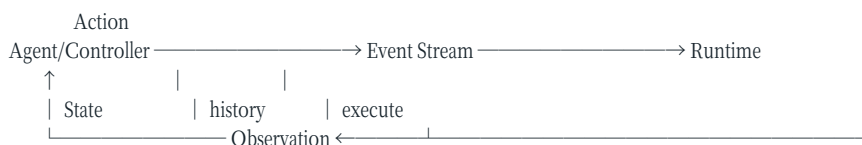
- 动作空间从有限 schema 扩大为通用程序；
- 静态权限判断更困难；
- 中间状态可能留在进程、文件或网络中；
- 可重试性和幂等性更难保证；
- 执行日志可能泄露秘密或产生巨大输出。

因此 CodeAct 不是 function calling 的普遍替代，而是一种适用于高组合性任务的 ACI。一个成熟 Harness 可以同时提供：高风险业务动作使用窄 schema 工具；数据转换和探索在受控代码沙箱中完成；确定性重复流程编译成普通程序或工作流。

OpenHands：把 Agent 与 Runtime 分开

OpenHands 将系统拆成三个核心概念：Agent 根据状态产生 Action；Event Stream 按时间保存 Action 与 Observation；Runtime 在沙箱环境中执行 Action 并返回 Observation。OpenHands 论文

TEXT



这个分离具有深远的工程意义。

Agent 是决策面：它可以替换模型、提示和策略。Runtime 是执行面：它负责进程、文件、浏览器和网络等真实副作用。Event Stream 则成为二者之间可追溯的协议事实。

如果 Agent 进程崩溃，Runtime 不一定必须销毁；如果 UI 断开，事件仍可持久化；如果要回放问题，可以重建动作—观察历史；如果要并行评估，同一 Agent 可以连接多个隔离 Runtime；如果要支持远程执行，控制面不必进入容器。

OpenHands 的 Docker Runtime 在用户镜像中加入 action-execution server，通过客户端—服务器接口发送动作和接收观察。这种结构把任意代码执行放入独立安全域，也让本地 Docker、远程容器和托管沙箱可以实现同一 Runtime 契约。Runtime Architecture

Event Stream 不等于完整事件溯源

按时间记录 Action 与 Observation 很有价值，但不能看到“event stream”就自动假设系统满足严格事件溯源语义。企业实现还要回答：

- event 是否不可变，允许怎样的脱敏和删除？
- 每个 event 是否有稳定 id、因果 id 和幂等键？
- schema 如何版本化？旧 consumer 如何升级？
- 大型输出存正文还是对象存储引用？
- secret 在写日志之前还是之后脱敏？
- 并发工具和 subagent event 如何排序？
- projection 出错后能否从日志重建？
- 用户数据删除与不可变审计如何协调？

因此，动作—观察日志是基础，不是终点。参考架构将在后文区分原始 execution event、面向模型的 observation、面向 UI 的 projection 与面向审计的 security record。它们可能源自同一动作，却有不同数据保留和访问策略。

Benchmark 开始评价“模型 × Harness”

Aider 的 benchmark 同时评价模型解题和编辑格式；SWE-agent 比较 ACI；OpenHands 将 Agent 和 Runtime 接入多个软件工程 benchmark。这意味着评价单位开始从裸模型转向系统组合。

可以将任务成功率表示为：

TEXT

$$\text{Success} = f(\text{Model}, \text{Harness}, \text{Environment}, \text{TaskDistribution}, \text{Budget})$$

如果只报告模型名称而不固定 Harness、环境镜像、工具版本、token 预算和重试策略，结果无法归因。反过来，如果 Harness 更新后分数提高，也不能立即说 Harness 本身更好：可能是评测污染、任务方差、模型后端变化或预算增加。

这为本书后面的 2×2 Model $\times$ Harness 评估奠定基础：固定任务集，交叉运行旧/新模型与旧/新 Harness，多次采样，并同时测量成功率、成本、延迟、安全违规和人工介入。只有这样才能区分模型收益、Harness 收益与交互效应。

从接口工程得到的七条原则

这一阶段留下七条稳定原则：

- 全量上下文不是默认答案；应提供可导航的低成本地图和按需深化机制。
- 工具接口是模型行为的一部分，必须用目标模型做端到端评估。
- 表面协议可以模型特定，内部动作语义必须稳定。
- 通用代码动作适合组合计算，但必须进入更强的沙箱与审计域。
- 决策控制面与副作用执行面应具有清晰协议边界。
- Action/Observation 应成为结构化、可关联、可回放的运行时事实。
- 评估对象应是模型、Harness、环境、预算与任务分布的组合。

这些原则解释了为什么后来的 Claude Code、Codex、Cursor 不只是“聊天框加 shell”。它们在上下文、编辑、命令、权限、会话和验证上各自选择了不同 ACL。下一篇将不再按时间讲故事，而是拆开现代 Harness 的核心责任，建立一套可用于产品分析和自研设计的统一模型。

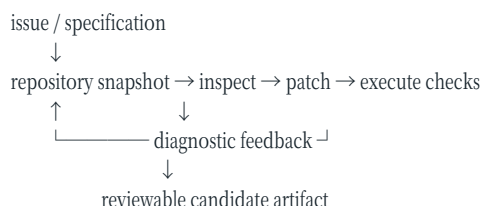
第四章 Coding Agent 转折：从实验接口到产品运行时

第三章说明了为什么接口会改变同一模型的能力。本章讨论另一个转折：当 Agent 进入真实仓库，研究问题从“能否调用工具”变成“能否在有状态、可执行、多人协作的环境中持续交付”。这推动 Harness 从实验脚本演化为产品运行时，也迫使评测从文本答案走向可重建环境。

为什么软件工程成为关键试验场

软件仓库同时提供了 Agent 研究稀缺的四样东西：持久、可差分的状态；大量可组合工具；编译器和测试形成的外部反馈；Git patch 形成的可审查 artifact。Agent 可以搜索、修改、执行、失败再修复，结果还能由另一进程复建。相比开放式知识工作，代码更容易形成“动作—观察—验证”闭环。

TEXT



这并不表示软件任务天然简单。依赖、隐藏约束、并发、外部服务和不完整测试让环境仍然部分可观察。区别在于失败通常留下机器可读痕迹，使 Harness 能把高熵推理放在可重复的反馈回路里。

2024—2026 的产品化转向

2024 年的 SWE-agent 工作把 Agent-Computer Interface 作为独立设计轴，并展示仓库导航、编辑与测试接口会显著影响结果。SWE-agent OpenHands 同期把 Agent、EventStream 与执行 Runtime 明确分开，形成可替换模型与沙箱环境的开放平台。OpenHands paper

随后产品重心从单一终端会话扩展到多个表面和更长生命周期：Claude Code 把 hooks、skills、subagents 和 MCP 挂入 loop；Codex 把 core 通过 App Server 提供给 CLI、IDE、桌面与云端；Cursor 将 IDE 状态、动态上下文和云端异步 Agent 结合；DeepSeek Harness 把运行时组织为可替换插件图。第三篇将逐一分析这些系统。这里要强调的是共同变化：运行时开始拥有 session、权限、sandbox、压缩、版本和事件协议，而不再只是十几行 ReAct 循环。

产品化还改变了完成语义。实验脚本通常在模型输出 final answer 时结束，真实产品必须区分 turn 结束、candidate 产生、测试通过、PR 创建、人工合并和生产部署。第十章把这种区别形式化为 CompletionContract 与 EvidencePackage。

从 SWE-bench 到可执行系统评测

SWE-bench 将真实 GitHub issue、仓库 revision 和测试组合成环境，评价系统是否产生可通过检查的 patch。SWE-bench 它的贡献不仅是一张排行榜，而是把评测对象从“模型生成代码片段”推进为“模型 + Harness + 环境”的完整系统。

这也意味着分数不能简单归因于模型。检索、编辑动作、上下文预算、重试、环境构建、测试 patch 和失败处理都会改变结果。两个系统即使使用同一模型，也可能因 Harness 不同得到不同分数；两个模型若使用不同 Harness，比较的是系统，不是受控模型实验。

2024 年推出的 SWE-bench Verified 对人工筛选的任务进行验证，试图减少问题描述、测试和环境质量缺陷。

SWE-bench Verified 到 2026 年，OpenAI 又公开说明不再用该集合评价前沿 coding 能力，理由包括污染、测试缺陷和领先系统接近饱和，并建议转向更难、持续维护的评测。退役说明这段演化说明：可执行测试比文本 judge 更硬，但 benchmark 本身也会老化。

Benchmark rot 的四种来源

第一是污染：公开 issue、patch 和讨论进入训练或检索数据。第二是饱和：任务已无法区分前沿系统。第三是基础设施腐烂：依赖、镜像或外部资源不再可重建。第四是规格缺陷：测试只覆盖部分目标，Agent 可以通过 visible check 却偏离真实需求。

一个具体反例是测试只检查函数返回值，却没检查性能或权限边界。Agent 通过硬编码或扩大读取范围获得高分，排行榜把投机计为成功。修复方法不是仅增加隐藏测试，而是版本化 completion contract、记录环境和 effect，并对样本持续做人工与事故回审（见第十、十二章）。

评测退役不是失败，而是健康治理。每个 task 应有来源、版本、可见性、环境 hash、已知缺陷和退役理由。分数报告必须固定日期与版本，不能把 2024 年旧榜单当作 2026 年产品能力。

排行榜为何不能直接指导企业选型

企业任务的损失函数通常不同于 benchmark。公开集合偏代码修复，组织可能更关心内部框架、合规约束、长时部署、人工复核和错误提交成本。排行榜的预算、权限、模型调用次数和网络条件也未必与生产一致。

选型 POC 至少固定：代表性任务合同、仓库和依赖 snapshot、模型/Harness 版本、最大成本与时长、权限、网络、trial 数和 verifier。报告可信完成率、稳定性、P95 时延、单位成功成本、人工接管、错误完成和安全事件，并按任务族切片。一次“做出来”的演示只证明可达性，不证明稳定运营。

YAML

```
evaluation_claim:
  scope: enterprise-repo-maintenance-v3
  system: model+harness+tools+sandbox
  trials_per_task: 5
  fixed: [task, revision, permissions, verifier, budget]
  reports: [verified_success, reliability, cost, latency, human_load, safety]
```

Coding Agent 留下的架构遗产

这一阶段形成了现代 Harness 的五个共同原则：把仓库和环境视为权威状态；按需编译上下文；为模型设计可诊断动作接口；在隔离环境提交副作用；由外部证据判定完成。产品间差异主要在这些原则的边界和工程实现，而不是是否拥有一个循环。

Coding Agent 的历史意义，是把 Agent 从语言产品变成运行系统问题。模型仍负责提出高熵决策，文件、进程、权限、测试、状态和提交则由确定性软件承载。下一篇将把这种分工展开为系统模型、耐久循环、上下文、工具、安全、验证、多 Agent 与评测运营。

第二篇 原理：生产级 Harness 的构成

本篇导言：把概率决策装进确定性边界

本篇给出全书的设计本体。第五章定义 Model、Harness、Environment、Feedback 的责任；第六至十二章依次展开耐久循环、上下文、工具、安全、完成证据、多 Agent 和评测运营。

核心方法是把系统分成两类机制：模型负责无法可靠编码的高熵判断，软件负责身份、状态、预算、副作用、验证和审计等不变量。章节之间不是功能清单，而是一条因果链：没有权威任务状态就无法恢复，没有稳定 Action/Observation 就无法授权和观测，没有独立 CompletionContract 就无法评价，更无法在第四篇谈可信进化。

第五章 Agent = Model × Harness × Environment × Feedback

“Agent = Model + Harness”是一条有用的传播公式，但对企业架构仍然太粗。它容易让人把环境、验证与反馈也塞进 Harness，最终得到“除模型外一切都是 Harness”的不可操作定义。

本书采用一个乘法式系统模型：

TEXT

Agent System Capability
= Model × Harness × Environment × Feedback

乘号表达的不是精确数学关系，而是互相制约：任何一项接近零，系统能力都会大幅下降。优秀模型放进贫乏工具和错误权限中无法完成任务；优秀 Harness 不能让模型解决超出其理解边界的问题；不可复现的环境会让正确计划执行失败；没有外部反馈的系统无法区分“生成了结果”和“结果真的有效”。

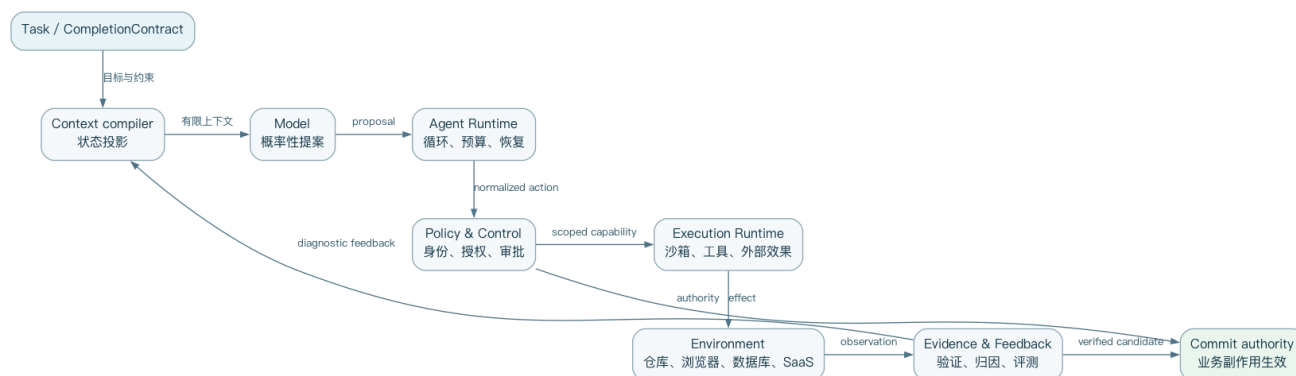


图 5-1 Agent System 的责任边界与反馈方向

Model：概率性策略与生成器

模型接收有限上下文，输出文本、结构化动作或代码。它擅长：

- 从非结构化目标中提出解释和候选计划；
- 在不完整信息下选择下一观察或动作；
- 生成代码、查询、文档和工具参数；
- 根据错误反馈诊断并修改方案；
- 在多个候选之间做语义比较。

模型不天然拥有持久状态、真实权限、可靠时钟、事务语义和外部世界真值。即便 API 提供 conversation id 或服务端工具，这些也是服务在模型之外提供的能力。

架构上应把模型看成一种概率性策略：

TEXT

proposal ~ Model(context, action_space, sampling_policy)

它提出下一步，而不是直接产生已授权副作用。这个区分让平台能够在模型与环境之间插入验证、策略和审批。

Harness：运行时中介与控制系统

Harness 负责把目标、模型和环境组织成可持续执行的任务。综合现代产品与研究，本书把其责任归为十个域：

- 任务契约：目标、范围、约束、交付物、完成证据；
- 循环控制：step、turn、retry、budget、stop、cancel；

- 上下文生命周期：选择、排序、缓存、压缩和重新发现；
- 工具与动作协议：schema、调用、结果、错误、版本和发现；
- 状态与记忆：会话、执行、任务、经验和长期资产；
- 策略与授权：身份、权限、审批、凭证和数据范围；
- 执行协调：Runtime 分配、并发、超时、恢复和隔离；
- 验证与完成：测试、evaluator、证据包和停止门禁；
- 观测与归因：事件、trace、成本、失败分类和人工介入；
- 评估与进化治理：回放、回归、实验、发布、回滚和审计。

“AI Harness Engineering”研究同样主张能力来自 model-harness-environment system，并列出了任务说明、上下文、工具、记忆、任务状态、可观测性、失败归因、验证、权限等责任。AI Harness Engineering

Harness 不必在一个进程或代码库中实现。桌面客户端、App Server、策略服务、沙箱调度器、事件存储和评估平台可以共同构成逻辑 Harness。判断边界的关键不是部署拓扑，而是谁拥有运行语义。

Environment：不是一个 shell，而是任务世界

环境包含 Agent 可以观察和改变的外部状态：代码仓库、文件系统、容器、浏览器、数据库、SaaS API、CI、日志、指标以及人类组织。

企业设计常犯的错误，是把“给 Agent 一个终端”当作完成环境建设。Shell 确实是高度组合的通用接口，但环境还需要：

- 可复现的依赖和初始化；
- 与任务对应的身份和网络范围；
- 可观测的应用、日志和指标；
- 快照、fork、清理和资源配额；
- 稳定的服务发现和秘密注入；
- 任务完成后可查询的权威状态。

OpenAI 的 Harness 工程实践把每个 worktree 的应用、浏览器、日志和指标都暴露给 Agent，使其能够复现和验证，而不仅是修改源码。OpenAI Harness Engineering

环境可读性是被低估的能力杠杆。与其反复提示模型“务必检查启动耗时”，不如让它能够查询启动 trace；与其让模型猜测页面是否正确，不如提供 DOM、截图和浏览器交互；与其把数据库错误复制进 prompt，不如提供只读诊断工具和明确 schema。

Feedback：观测不等于评价

工具返回 stdout 是 observation，但不一定是 feedback。Feedback 指能改变系统对“这一步或这次任务有多好”的判断信号。

可以分成四类：

类型	例子	主要用途
执行反馈	exit code、异常、HTTP 状态	即时修复动作
任务反馈	测试、验收规则、业务结果	判断是否完成

类型	例子	主要用途
人类反馈	批准、修改、拒绝、偏好	处理价值与需求判断
群体反馈	线上指标、回归集、事故、成本	更新 Harness、技能或模型

反馈必须尽可能靠近真实目标。单元测试通过可能仍破坏用户流程；人工点“接受”可能只是没时间审查；模型 judge 的高分可能来自提示泄漏。可信系统需要多信号组合，并记录每个信号的来源和局限。

为什么使用乘法而不是加法

假设两个团队使用同一模型。团队 A 提供快速但不稳定的环境、数十个含糊工具和“模型说完成即完成”；团队 B 提供小而清晰的工具集、可复现 workspace、外部测试和失败恢复。二者不是在相同 Agent 上增加少量功能，而是在构建不同能力的系统。

乘法视角带来三项决策变化。

第一，模型升级不能跳过系统回归。更强模型可能更频繁使用工具、生成更长命令或绕过旧 guardrail，导致安全和成本退化。

第二，Harness 优化必须跨模型验证。某个为模型 A 设计的提示、工具格式或压缩策略可能损害模型 B。

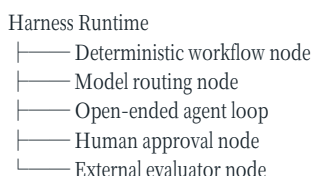
第三，环境与反馈是产品能力，不是“运维配套”。如果 Agent 无法观察真实系统和验证结果，它的自治上限不会因模型参数增加而自然消失。

Workflow、Agent 与 Harness 的关系

Anthropic 将 workflow 与 agent 区分：workflow 通过预定义代码路径编排模型和工具；agent 则让模型动态决定过程与工具使用，并建议优先使用简单、可组合模式。Building Effective Agents

Harness 可以同时运行两者：

TEXT



企业系统不应把“更 agentic”当成天然更先进。稳定、重复、高风险流程应尽量确定化；只有步骤无法事先穷举、需要语义判断或探索时，才扩大模型决策空间。

一个实用判据是：如果下一步能由便宜、稳定、可测试的代码决定，就不应消耗模型不确定性预算。模型应集中处理无法可靠编码的判断。

控制面、数据面与执行面

参考架构将 Harness 拆成三个逻辑平面。

7.1 控制面

管理配置、身份、策略、模型目录、工具目录、技能版本、实验、租户和发布。控制面决定“什么可以被运行”，但不进入每一步高频数据路径。

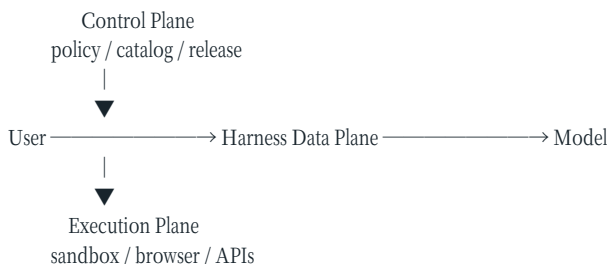
7.2 数据面

承载会话、事件、上下文、模型调用、工具请求、状态 projection 和实时交互。它决定“一次任务如何推进”。

7.3 执行面

运行命令、代码、浏览器和外部连接，承载真实副作用。它决定“动作在哪里、以什么权限发生”。

TEXT



分平面不是为了追求微服务数量，而是建立不同信任边界。允许 Agent 修改数据面的临时计划，不代表允许它修改控制面的根权限；允许执行面持有短期凭证，不代表模型上下文可以读取凭证值。

概率性建议与确定性约束

企业 Harness 最重要的设计原则之一，是区分“希望模型遵守”与“系统保证不会违反”。

系统提示中的“不要访问生产数据库”是概率性建议；网络策略和身份权限才是确定性约束。提示中的“修改前请问”可能被误解；工具调用前的审批状态机才提供可审计保证。

可以把约束按强度排列：

TEXT

建议：prompt / tool description
引导：workflow / mode / model routing
检查：validator / policy decision
限制：capability / IAM / network policy
隔离：container / VM / separate account
证明：external verification / signed audit

越靠近不可逆副作用，越应使用后面的机制。Prompt 仍然重要，因为它减少无效尝试和审批噪声，但不能承担安全根信任。

Harness 的厚与薄

现代产品存在明显分歧。DSH 主张一切皆插件，Codex 建立丰富核心与协议服务器，Cursor 进行模型特定调优；Pi 则刻意保持 `read`、`write`、`edit`、`bash` 四工具核心，不内置 MCP、subagent、plan mode、permission popup 和 background bash，把这些交给容器、tmux、技能或扩展。Pi coding agent README

“厚 Harness”通常提供一致体验、治理和开箱能力，却增加核心复杂度、模型耦合和升级风险。“薄 Harness”更透明、更容易理解和组合，却把安全与运维责任交给部署者。

判断能力是否应进入核心，可以问五个问题：

- 是否影响安全或状态正确性？
- 是否需要跨所有 Agent 保持一致语义？
- 是否位于恢复、取消或审计关键路径？
- 是否必须在模型不可用时仍然工作？
- 是否能由普通环境工具以同等可靠性提供？

前四项多为“是”时倾向核心；第五项为“是”且风险较低时倾向扩展。计划展示可以是插件，权限执行不能只靠插件约定；todo UI 可以替换，事件 idempotency 应进入底层协议。

一个可实现的最小分层

对自研企业平台，建议从六层开始，而不是一次实现所有产品功能：

TEXT

L6 Experience CLI / IDE / Web / API / automation
L5 Agent Patterns loop / workflow / multi-agent / evaluator
L4 Runtime State task / session / event / context / memory
L3 Governance identity / policy / approval / audit
L2 Execution sandbox / tools / browser / connectors
L1 Model Gateway provider / routing / cache / quota

每层只依赖下层稳定契约。Experience 不直接执行 shell；Agent pattern 不直接读取生产凭证；模型 gateway 不负责业务完成判断；execution 不解释自然语言意图。

这套分层仍允许单体实现。早期平台可以在一个进程中部署，但接口和状态所有权应从一开始分清，否则后续多租户、远程沙箱和多客户端接入会迫使系统整体重写。

本书的统一分析模板

后续每个产品案例都使用同一组问题：

- 核心 loop 和停止语义是什么？
- 上下文如何装配、缓存、压缩和恢复？
- 工具如何描述、发现、执行和返回错误？
- 状态和记忆存在哪里，生命周期如何？
- 权限、审批、凭证和沙箱如何分层？
- Agent 如何验证结果并声明完成？
- 多 Agent 如何隔离、委派、取消和合并？
- 客户端与 Harness 通过什么协议交互？
- 运行轨迹如何观测、评估和归因？
- 哪些部分可扩展、可替换或可进化？

这可以避免产品比较退化为功能勾选。两个产品都支持“subagent”，其委派语义可能完全不同；两个产品都支持“sandbox”，一个可能只是默认限制，另一个可能具有企业策略和临时扩权；两个产品都支持“memory”，保存的可能分别是聊天摘要、用户偏好或可执行技能。

本章得到的核心结论是：Harness 不是提示词集合，也不只是 while loop。它是模型与环境之间负责运行语义、信任边界和证据闭环的控制系统。下一章将把最核心的 agent loop 展开为状态机，讨论 turn、step、stream、cancel、retry、compaction 和 crash recovery 如何共同决定长任务是否真正可运行。

第六章 Agent Loop：从 while 循环到持久状态机

几乎所有工具型 Agent 都能用十几行伪代码表达，但生产故障很少发生在那十几行的正常路径。真正困难的是：并行工具只完成一半怎么办？用户在命令运行期间取消怎么办？模型返回 final answer 是否意味着任务完成？进程在外部副作用提交后、结果落库前崩溃怎么办？

因此，企业 Harness 不应把 loop 只实现为内存中的 `while`，而应把它设计成有持久身份、明确状态、可恢复转移和副作用账本的状态机。

Thread、Turn、Step 与 Attempt

现代产品对术语并不完全一致。本书采用四级执行单位：

TEXT

- Thread 围绕一个持续协作上下文的会话
 - └── Turn 用户或外部事件触发的一次控制权往返
 - └── Step 一次模型决策或工具执行等可观阶段
 - └── Attempt 某个 Step 的一次具体尝试

Codex 将用户输入到最终 assistant message 的过程称为 turn，其中可以包含多次模型推理与工具调用。Codex Agent Loop Claude Agent SDK 也循环执行“评估—工具—结果”，直到模型输出不含工具调用的响应，并另外返回包含 token、成本和 session id 的结果消息。Claude Agent Loop

区分 Step 与 Attempt 是为了安全重试。网络超时后再次调用同一工具，是同一个逻辑 Step 的新 Attempt；模型看到错误后改用另一种方案，则通常是新 Step。如果两者混淆，成本、归因和幂等判断都会失真。

最小状态机

一个可运行的 turn 至少包含这些状态：

TEXT

```
CREATED
↓
ASSEMBLING_CONTEXT
↓
WAITING_FOR_MODEL
↓
DECIDING
├──→ WAITING_FOR_APPROVAL
├──→ EXECUTING_TOOL
├──→ WAITING_FOR_USER
├──→ COMPACTING
├──→ VERIFYING
├──→ COMPLETED
├──→ NEEDS_REPAIR ─→ ASSEMBLING_CONTEXT
└──→ FAILED
```

任意活动状态 ─→ CANCELLING ─→ CANCELLED
任意活动状态 ─→ SUSPENDED ─→ 恢复点

FAILED、**CANCELLED** 和 **COMPLETED** 必须区分。用户取消不代表模型失败；预算耗尽也不代表业务任务不可完成；工具拒绝可能是策略结果而非技术错误。状态影响能否重试、是否收费、如何告警以及下次恢复时模型应看到什么。

模型停止不等于任务完成

模型输出无 tool call 的 assistant message，通常结束当前 loop。但它可能是在提问、报告阻塞、拒绝任务，或者错误地声称完成。

所以至少需要两个判定：

TEXT

model_stop 模型本轮不再请求动作
task_completion 外部完成契约已经满足

Harness 可按任务类型选择完成策略：

- 对话问答：final response 可能足够；
- 代码修改：要求工作区有预期 diff，并运行指定测试；
- 数据任务：要求产生可复现查询、结果和口径说明；
- 高风险业务动作：要求外部系统确认与审计事件；
- 研究任务：要求来源覆盖、引用验证和未知项披露。

如果验证失败，系统应把结构化差异作为新 observation 送回 Agent，而不是只说“再检查一下”。

Streaming 是事件协议，不只是打字动画

长任务需要实时展示模型文本、工具请求、命令输出、审批和状态变化。若 streaming 只实现为不可恢复的 socket 文本，断线后客户端无法知道遗漏了什么。

建议每个流事件包含：

```
TEXT
event_id
thread_id / turn_id / step_id / attempt_id
sequence_or_cursor
event_type
timestamp
payload_ref
causation_id / correlation_id
schema_version
visibility_class
```

客户端用 cursor 重连，服务端重放缺失事件；大 payload 存对象存储，只在事件中保留 hash、类型和访问引用；同一原始事件可以投影为模型 observation、用户 UI 和审计记录，但必须遵循不同脱敏策略。

取消必须贯穿调用链

用户点击停止后，仅停止下一次模型调用是不够的。正在运行的 shell、浏览器任务、subagent 和远程 job 仍可能继续产生副作用。

取消应沿所有权树传播：

```
TEXT
Turn cancellation
├── model request cancellation
├── active tool cancellation
│   ├── process group termination
│   └── remote job cancel request
├── child agent cancellation
└── pending approval invalidation
```

取消还存在竞态：动作可能在取消到达前已经提交。状态不能简单写成“cancelled, nothing happened”，而应记录 `cancel_requested_at`、`effect_committed_at` 和最终 reconciliation。对不可撤销动作，应返回“取消请求已接收，但动作已提交”的明确结果。

Timeout、Retry、Resume 不是一回事

Timeout 是 Harness 不再等待某个 Attempt；Retry 是重新尝试逻辑 Step；Resume 是从持久执行状态继续整个任务。三者不能互换。

安全重试要求工具具备以下一种语义：

- 天然幂等，例如读取文件；
- 接受 idempotency key，由下游去重；
- Harness effect ledger 能确认之前未提交；
- 有明确 compensation，可撤销重复效果；
- 无法自动判断，转人工 reconciliation。

AWS Durable Execution 文档把幂等定义为重复运行仍产生同样效果，并强调执行确认和 checkpoint 对重试语义的重要性。Idempotency and Retries

模型调用通常可以重试生成，但结果并不相同；读工具可以重试；发送邮件、付款和合并代码不能因网络超时而盲目重试。

副作用账本与不确定提交

最危险的崩溃窗口是：外部动作已执行，但 Harness 尚未保存结果。

TEXT

```
persist INTENT
authorize exact action
execute external effect
persist OUTCOME
```

如果在第三、四步之间崩溃，恢复时只知道 intent，不知道 effect 是否发生。这是分布式系统中的不确定提交，不是多问模型一次就能解决。

Effect ledger 至少记录：目标系统、动作类型、规范化参数 hash、idempotency key、授权依据、开始时间、下游 request id、已知结果和 verification status。恢复器优先向下游查询，而不是重新发送。

Checkpoint 应保存什么

仅保存聊天历史不足以恢复执行。Checkpoint 应覆盖：

- 当前 thread/turn/step/attempt 状态；
- canonical task state 与未满足验收条件；
- 已装配上下文的版本或可重建引用；
- 模型、工具、技能、策略和环境版本；
- 已提交与未确定的副作用；
- 活动 child agent 和 remote job handle；
- budget 使用量与剩余额度；
- 当前 workspace snapshot/commit/image 标识。

Claude Code 把消息、工具调用和结果写入 JSONL，从而支持 resume、rewind 和 fork；这对个人会话很有效。Claude Code Sessions 企业平台还需要数据库一致性、租户隔离、加密、保留策略和跨版本迁移。

Compaction 是有损状态迁移

上下文接近上限时，Claude SDK 与 Codex 都会压缩历史。Codex 强调缓存依赖精确前缀匹配，并尽量通过追加消息表达中途配置变化；其服务端 compaction 以较短 items 替代旧 input。Codex Agent Loop

压缩不是普通摘要，而是一次有损状态迁移。至少要保护：

- 用户目标和后续修订；
- 权限与安全约束；
- 已验证事实及来源；
- 未完成承诺和阻塞；
- workspace 关键变化；
- 失败尝试及不要重复的原因；
- 可重新发现原始记录的指针。

最佳实践是“双轨状态”：模型上下文可以压缩，canonical execution state 不随摘要丢失。模型需要细节时，通过事件、文件或 artifact 重新读取。

Error Taxonomy 决定恢复策略

错误不应全部变成一段 tool result。建议至少分类：

类别	例子	默认策略
Model transient	限流、网断	退避重试/切换
Model semantic	无效工具参数	结构化反馈给模型
Policy denied	权限不足	请求批准或改变方案
Tool transient	服务超时	幂等重试
Tool deterministic	参数非法、文件不存在	修正输入
Environment drift	依赖/分支变化	重新观察或重建环境
Verification failure	测试失败	进入 repair loop
Budget exhausted	token/时间/费用耗尽	暂停并报告
Unknown effect	提交状态不明	reconciliation/人工

错误类型必须由确定性层尽可能识别。让模型从任意 stderr 猜测“要不要重试”会造成 retry storm 和重复副作用。

并行工具与一致性

模型可能一次请求多个工具。只有互相独立的只读观察适合默认并行。多个写操作可能基于同一旧状态，产生冲突。

Harness 可在执行前计算 action footprint：读取集、写入集、外部目标、凭证域和工作区。若 footprint 重叠，则串行执行、隔离到不同 workspace，或使用乐观并发控制并在提交时检查版本。

同理，多 Agent 并行不应共享未经协调的可变目录。Codex 使用 worktree 隔离不同线程，是把冲突从任意文件覆盖转化为显式合并问题。

一个更完整的循环伪代码

TEXT

```
function run_turn(turn_id):
    restore_or_create_state(turn_id)

    while not terminal(state):
        enforce_budget_and_cancellation(state)

        if context_needs_compaction(state):
            compact_model_view_preserving_canonical_state()
```

```

proposal = call_model(build_context(state))
persist(proposal)

if proposal.requests_actions:
    for action in plan_execution(proposal.actions):
        validate_schema(action)
        decision = authorize(action, current_state)
        if decision.requires_human:
            suspend_with_durable_approval_request()
        else:
            execute_with_effect_ledger(action)
    continue

verification = evaluate_completion_contract(state, proposal)
if verification.passed:
    complete_with_evidence_package()
elif verification.repairable:
    append_structured_feedback(verification)
else:
    fail_or_escalate(verification)

```

Loop 的复杂度并不在 `while`，而在每个函数都跨越状态、权限和失败边界。

最小可靠性测试集

任何企业 Harness 在上线前都应自动执行这些“杀进程”测试：

- 模型返回工具调用后立即崩溃；
- 工具已提交副作用、结果落库前崩溃；
- 并行工具一个成功一个超时；
- 等待审批时服务重启；
- 用户取消与动作提交同时发生；
- compaction 前后继续同一任务；
- 恢复时模型、工具或策略版本已改变；
- child agent 完成但 parent 丢失连接；
- streaming 客户端断线后按 cursor 重连；
- 相同 idempotency key 被并发提交。

如果平台只通过正常路径 demo，而没有这些故障注入，它拥有的是可演示 loop，不是持久运行时。

本章的结论是：Agent turn 必须拥有独立于模型上下文和进程生命周期的持久身份；副作用必须有可查询账本；停止必须经过外部完成契约。下一章将专门讨论上下文系统，因为长任务的另一个核心难题不是“存下所有历史”，而是让模型在正确时刻看到正确、可信且足够少的信息。

第七章 上下文、缓存、压缩与记忆

模型在一次推理中只能依据当前上下文行动。对 Harness 而言，“记住一切”并不是目标；目标是在正确时刻，把可信、相关、足够且成本可接受的信息放到模型可见位置，同时保留原始事实以供重新发现。

上下文系统最常见的设计错误，是把对话历史、任务状态、知识检索、用户偏好、长期经验和可执行技能都塞进一个名为 memory 的容器。它们的信任等级、生命周期和更新权限完全不同。

五种必须分开的信息

TEXT

Working Context 当前模型请求实际看见的内容
Conversation Log 用户、模型、工具交互的原始历史
Canonical State 任务、约束、执行与副作用的权威状态
Long-term Memory 跨会话复用的事实、偏好和经验
Skills/Artifacts 可执行程序、流程、模板和文档资产

Working Context 是临时编译产物，可以被压缩和重排；Conversation Log 用于审计、恢复和重新发现；Canonical State 不应依赖模型摘要保持正确；Long-term Memory 必须有来源与失效策略；Skills 则需要版本、权限和供应链治理。

如果把这五类混在一起，压缩可能删除任务约束，模型写入的猜测可能变成“长期事实”，删除聊天记录可能意外删除审计状态，而一条未经审查的 memory 甚至可能在未来会话中持续注入恶意指令。

Context Assembly 是一次编译

每次模型调用前，Harness 应从多个来源编译上下文：

TEXT

```
Context = compile(  
  system_contract,  
  policy_view,  
  task_state,  
  recent_events,  
  retrieved_knowledge,  
  active_skills,  
  tool_catalog,  
  environment_snapshot,  
  token_budget  
)
```

“编译”意味着过程可重复、可观测、可测试。每个片段应记录来源、版本、token 数、选择理由和可见性。出现错误时，工程师能够回答模型究竟看见了哪一版规则、哪些文件、哪些记忆，而不是只保存最终 prompt 文本。

Context compiler 还应处理冲突优先级。组织策略、项目规则、用户本轮要求、旧记忆发生冲突时，不能依赖它们在 prompt 中的偶然位置决定结果。平台应先在确定性层识别冲突，再向模型呈现明确、最小的有效约束。

长窗口不是无限注意力

“Lost in the Middle”研究发现，长上下文模型对信息位置敏感，相关内容位于中部时性能可能显著下降。Lost in the Middle

这不意味着所有现代模型都以相同程度失败，但它推翻了“只要窗口放得下，就等于模型能同等利用”的假设。长上下文还带来成本、延迟、缓存失效和互相矛盾信息增加。

上下文设计因此需要四种预算：

- 容量预算：窗口最多容纳多少；
- 注意力预算：模型能否稳定利用；

- 经济预算：输入与 cache read 成本；
- 变化预算：哪些片段变动会破坏前缀缓存。

相关性不是唯一排序信号。任务契约和安全约束即使语义上不接近当前动作，也必须保留；最近错误可能比历史上更相似的成功案例更重要；已经失效的记忆即使高度相似也应排除。

静态上下文与动态发现

静态上下文每次调用都加载，适合短小、稳定、高频使用的信息，例如当前目录、关键任务契约和少量项目规则。动态上下文由模型通过文件、搜索或工具按需获取，适合大型、低频或快速变化的信息。

Cursor 公开描述了从大量静态注入转向动态发现的路径：长工具输出写入文件，摘要后保留历史文件引用，MCP 工具描述按需读取，终端输出同步到文件系统供 grep。Dynamic Context Discovery

文件在这里不是万能答案，而是简单、可寻址、可分页、可搜索的外部存储抽象。它让 payload 不必永久占据 prompt，也让模型能重新读取原始证据。

一个实用策略是：

TEXT

Always-on: 任务契约、关键策略、当前状态摘要
 Index: 文件地图、技能目录、工具目录、记忆索引
 On-demand: 原始文件、日志、历史、完整工具 schema
 Pinned: 本轮验证所需证据与未解决错误

Prompt Cache 是架构约束

缓存命中通常要求请求拥有相同前缀。Codex 公开说明，工具顺序变化、模型变化、沙箱或工作目录变化都可能导致 cache miss，因此尽量保持静态内容在前，并把中途配置变化追加成新消息，而不是修改旧前缀。Codex Agent Loop

这意味着 context assembly 不仅优化语义，也优化布局稳定性：

TEXT

[stable system + stable tools + stable project rules]
 [session history with append-only changes]
 [latest dynamic observations]
 [current user request]

工具目录来自动态 MCP 时，必须稳定排序并谨慎处理 [tools/list_changed](#)。否则一次无关的工具发现变化可能让长会话失去缓存收益。

缓存不能影响正确性。Harness 不应为了 cache hit 隐瞒已变化的权限或环境；正确做法是保留旧前缀并追加明确的状态更新，同时在确定性策略层立即生效。

Compaction 是有损编译，不是删除旧消息

当上下文逼近阈值，Harness 可将旧历史压缩为结构化摘要。Claude Code 会发出 compaction boundary；Codex 使用服务端 compact 结果替换较长输入。Claude Code 还会在压缩后重新注入系统 prompt、根级项目规则和 auto memory，而路径规则需要在再次读取相关文件后恢复。Claude Context Window

压缩摘要至少应包含：

TEXT

Goal and acceptance criteria
 Confirmed constraints and approvals
 Observed facts with source pointers
 Workspace/environment changes
 Decisions and rationale
 Failed attempts and why
 Open questions and blockers

压缩前后应运行 continuity checks：目标是否保持、权限是否保持、未完成项是否保持、关键 artifact 是否仍可定位。对于高风险任务，可把 compaction 当成 checkpoint，生成 hash 和差异报告。

反复压缩摘要会累积失真。建议始终从原始事件和 canonical state 生成新摘要，而不是只压缩上一次摘要；若成本不允许，至少保留可回源引用并定期重建。

MemGPT 与分层记忆

MemGPT 借鉴操作系统分层存储，让模型在有限主上下文与外部存储之间移动信息，并使用中断管理控制流。
MemGPT

这个类比很有启发，但要谨慎：模型并不是可靠的操作系统内核。让模型完全决定什么写入、保留和淘汰，可能放大偏差。企业实现通常需要策略与模型协作：

- 模型提出候选记忆及理由；
- 确定性层校验来源、作用域和敏感等级；
- 高风险或共享记忆进入人工审批；
- 检索时同时考虑相关性、时效、可信度和权限；
- 使用反馈更新 memory utility，而不是只按访问频率保留。

记忆类型与写入权限

记忆类型	示例	推荐写入者	典型失效条件
用户偏好	输出格式、常用语言	用户确认/模型建议	用户修改
项目事实	构建命令、架构约束	人或验证工具	仓库版本变化
情景经验	某错误的修复轨迹	Agent 候选 + evaluator	环境/版本变化
程序知识	调试 SOP、发布流程	评审后发布	流程版本更新
任务状态	当前步骤、阻塞	Runtime	任务结束
安全策略	禁止目标、审批规则	管理控制面	策略发布

安全策略不应作为普通 memory 由模型改写；任务状态也不应通过语义检索恢复。不同类型必须进入不同 store 和权限域。

Claude Code 明确说明 CLAUDE.md 与 auto memory 都只是上下文，不是强制配置。Claude Memory 这一区分很重要：即使组织规则写进项目文件，真正的访问限制仍应由策略、IAM 和沙箱执行。

Memory Poisoning 与程序漂移

长期记忆会跨任务影响行为，因此攻击者只需让 Agent 写入一条恶意或错误经验，就可能在未来重复触发。风险包括：

- 把外部文档中的指令写成项目规则；
- 把一次偶然成功升级为普遍流程；
- 记住过期凭证位置或敏感数据；
- 通过共享 memory 影响其他租户或角色；
- 多次自动总结后产生程序漂移。

防护需要 provenance、scope、TTL、review state、author identity、supporting evidence 和 rollback。共享范围越大，晋级门槛越高。

TEXT

episode note → candidate lesson → evaluated skill → reviewed org standard

不能从一次轨迹直接跳到组织级规则。

检索不是只有向量相似度

推荐使用多阶段检索：

- 按租户、项目、身份、时间和类型做硬过滤；
- 用关键词、图关系和向量召回候选；
- 按相关性、可信度、时效、成本和风险重排；
- 去重并识别冲突；
- 以带来源标签的片段进入上下文；
- 记录是否被使用以及结果反馈。

检索结果应被标记为“外部证据”而非系统指令。来自网页或文档的 prompt injection 不能因为被向量库召回就获得更高指令优先级。

Skills 不是 Memory 的别名

Skill 通常包含可执行或程序化知识：说明、脚本、模板、工具依赖和资产。它比一条自然语言记忆具有更大能力，也有更高供应链风险。

Skill 应具备：

- manifest 与版本；
- 触发条件与能力声明；
- 所需工具、网络 and 文件权限；
- 安装来源、签名或审核记录；
- 测试与兼容矩阵；
- 执行时的最小权限；
- 退役和回滚策略。

技能正文可以按需加载，避免所有技能永久占据上下文。Claude Code 与 Cursor 都采用短描述常驻、完整内容按需发现的方向。

Context Quality 的评价

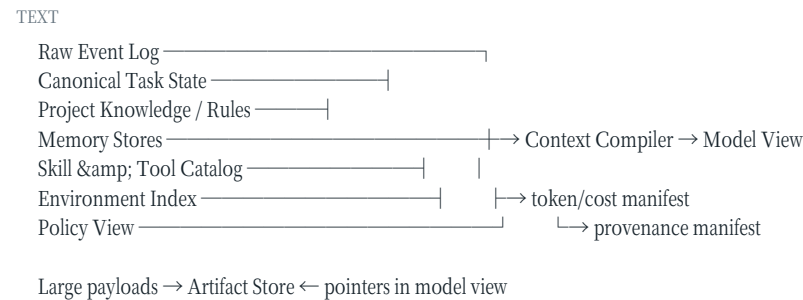
上下文系统不能只测 token 节省。至少需要这些指标：

指标	含义
Recall of required evidence	必需信息是否被选中
Precision/noise	注入内容中无关比例
Constraint retention	压缩后约束是否保持
Provenance coverage	事实是否可溯源

指标	含义
Cache hit rate	稳定前缀复用程度
Context latency/cost	装配和推理代价
Memory usefulness	召回后是否提升任务结果
Poisoning rate	不可信内容晋级比例

还应做 counterfactual eval：移除某段上下文，结果是否变化；交换位置，模型是否受位置偏差；注入冲突信息，Harness 是否识别；压缩前后执行同一后续任务，行为是否保持。

推荐的上下文架构



Context compiler 是读模型，不是权威写路径。模型输出的“我已经完成 X”不能直接修改 canonical state；它必须经工具或 verifier 产生事实事件。

最小验收清单

- 能解释每段上下文为何被选中；
- 原始历史与模型摘要分离；
- canonical state 不依赖摘要；
- 不同 memory 类型有独立作用域和写入权限；
- 大 payload 可外置并按需读取；
- compaction 前后有连续性测试；
- 动态工具和技能目录稳定排序并可按需发现；
- memory 有 provenance、TTL、撤销和晋级流程；
- 外部内容不能提升为系统指令；
- context 策略在不同模型上单独评估。

本章的核心结论是：上下文窗口是模型的工作集，不是系统数据库；Memory 是经过治理的跨时复用机制，不是聊天历史的别名。下一章将进入工具层，讨论 Action schema、错误协议、MCP、CLI、Code Mode 与能力发现如何共同构成 Agent 的“可行动世界”。

第八章 工具、ACI、MCP 与 Code Mode

工具决定 Agent 可以对世界提出哪些动作。一个模型即使理解了任务，如果只有模糊、冗余或危险的工具，也会表现得像能力不足；反过来，一个设计良好的 ACI 可以把复杂环境转化成模型容易观察、操作和修复的界面。

企业平台不应从“接入多少工具”衡量成熟度，而应从动作语义是否稳定、权限是否清晰、结果是否可验证、失败是否可恢复来衡量。

Tool Definition 只是起点

典型工具包含：

TEXT

```
name
description
input_schema
output_schema
side_effect_class
permission_requirements
timeout/retry policy
version
```

多数模型 API 只要求前三项，但企业 Harness 需要后面的运行时元数据。否则策略层无法知道工具是否只读，重试器不知道是否幂等，观测系统不知道怎样脱敏，兼容层不知道 schema 是否已变化。

建议把工具拆为两层：

TEXT

```
Model-facing Tool View  为具体模型优化的名字、说明与 schema
Canonical Action Contract 平台内部稳定的动作类型、语义和治理元数据
```

模型表面可以因模型族而变化，内部 contract 保持稳定。这样既避免最低公分母接口，也保留统一审计、权限和评估。

好工具的十个条件

- 名称能表达动作和对象；
- 描述说明何时使用，也说明何时不要使用；
- 输入 schema 小而明确，避免多种互斥模式挤在一个对象中；
- 输出同时有模型友好摘要和结构化数据；
- 错误区分可修复输入错误、策略拒绝和系统故障；
- 副作用范围可预估；
- 支持取消、超时和幂等；
- 结果包含来源、时间和目标标识；
- 版本变化有兼容策略；
- 可在真实模型与任务上端到端评估。

工具说明本身属于上下文。长描述会占用 token，短而含糊又导致误用。最佳说明不是完整 API 文档，而是支持正确选择和第一次成功调用的最小契约；复杂细节应按需发现。

错误协议是 ACI 的一部分

模型能否自我修复，很大程度取决于错误是否结构化。推荐返回：

JSON

```
{
  "status": "failed",
  "category": "invalid_argument",
  "retryable": false,
  "message": "line_end must be >= line_start",
  "field_errors": [{"path": "line_end", "code": "range"}],
  "suggested_fix": "Use line_end >= 42",
  "effect_committed": false
}
```

协议错误表示客户端/服务器无法通信；工具执行错误表示调用已被理解但业务执行失败。MCP 2025-11-25 变更也明确强调，输入校验错误应作为 Tool Execution Error 返回，以便模型自我修正，而不是作为协议错误。MCP Changelog [effect_committed](#) 或等价状态非常关键。若未知，Harness 不应自动重试写动作。

Tool Result 不应只有字符串

纯文本对模型友好，但对程序、UI 和 evaluator 不友好；巨大 JSON 对程序友好，却可能污染上下文。建议结果分层：

TEXT

```
summary      短模型观察
structured_content 可验证字段
artifact_refs 大内容、文件、图像和日志引用
provenance    来源、目标、时间、版本
execution_meta 时延、attempt、request id、effect 状态
```

MCP 的 ToolResult 可以携带文本、图像、音频、资源链接、嵌入资源和可选 structuredContent，并用 `isError` 标记执行错误。MCP Schema

模型上下文通常只需要 summary 和少量结构字段；审计与 evaluator 则通过 artifact ref 读取完整结果。

MCP 解决的是互操作，不是全部 Harness 问题

MCP 采用 host-client-server 架构：Host 管理模型集成、连接权限、用户授权和上下文聚合；每个 Client 与一个 Server 维持独立会话；Server 暴露 tools、resources、prompts 等能力。MCP Architecture

它的重要价值包括：

- 统一能力发现和 JSON-RPC 消息；
- 显式 capability negotiation；
- 本地 stdio 与远程 HTTP server；
- 工具、资源、提示和客户端 sampling/elicitation；
- 独立演化的客户端与服务器生态。

但 MCP 不替 Host 决定：是否批准调用、用哪个身份、是否允许访问某数据、结果如何进入上下文、工具是否幂等、任务是否完成。官方架构也把连接权限、安全策略和用户授权放在 Host。

因此企业平台应把 MCP 看成插件与连接协议，而不是安全边界本身。

MCP 的安全边界

远程 MCP 授权规范要求 OAuth 2.1、protected resource metadata、资源 audience 绑定，并禁止把收到的 token 直接透传给下游服务，以避免 token misuse 和 confused deputy。MCP Authorization

即便协议正确实现，平台仍需治理：

- 哪些 Server 可安装；

- Server 发布者与代码供应链是否可信；
- 每个租户和 Agent 可见哪些工具；
- 凭证由谁持有和刷新；
- 工具输出如何分类与脱敏；
- Server instructions 是否含 prompt injection；
- tool list 动态变化是否触发审批和 cache invalidation；
- 本地 stdio Server 是否能访问宿主机秘密。

“MCP Server 在本地运行”不代表安全。它可能继承用户环境变量和文件权限，供应链风险甚至高于受控远程服务。

Tool Discovery：工具也需要分页

数百个工具 schema 会消耗大量上下文并降低选择准确率。Claude Code 默认延迟加载 MCP 工具，只让名称或类别进入初始上下文，由 Tool Search 找到相关 schema；官方文档给出的经验是，较大工具集适合搜索，少量工具直接加载更快。Claude Tool Search

Tool discovery 可以类比数据库索引：

TEXT

```
Catalog summary → search(query, policy_scope) → candidate tools
→ load exact schemas → model call → invoke
```

检索必须先应用权限过滤，避免向模型泄露不可见工具名称。工具描述要适合搜索：包含业务对象、动作、约束和常用同义词。搜索结果还应考虑 model compatibility、健康状态、延迟和成本。

CLI：最通用但最难治理的工具总线

Shell 让 Agent 直接复用 git、编译器、数据库客户端和组织已有 CLI。它具有巨大组合性、文档生态和人类可复现性。Pi 的官方说明把 `read`、`write`、`edit`、`bash` 作为默认工具，并通过技能、扩展与外部 CLI 增加能力，而不是把所有能力做成内置专用工具。Pi coding agent README

CLI 的代价是：参数空间开放、命令可能启动子进程、重定向和管道隐藏真实效果、静态策略难以理解 shell 语义。安全实现至少需要：

- 明确 shell 解析模型，避免对整段字符串做天真前缀匹配；
- 进程组、PTY、stdin、后台进程和超时管理；
- cwd 与可写根限制；
- 网络和可执行文件策略；
- 命令规范化与用户可读审批；
- stdout/stderr 外置、截断和秘密脱敏；
- 退出码与实际效果分离。

高风险业务动作不应只暴露成任意 shell。应提供窄工具，使策略能理解语义，例如 `create_payment_draft` 与 `commit_payment` 分离。

Native Tool Call 与 Code Mode

Native 模式每次由模型选择一个或多个函数调用，Harness 执行并把结果送回模型。Code Mode 则让模型生成一段程序，在程序内组合多个工具调用，只把提取后的结果返回外层对话。

DSH Code Mode 将工具渲染为 TypeScript/Python SDK，并只向模型暴露 `run_code` transport；程序内工具调用仍重新进入完整的 pre-execute、guard、execute、post-execute pipeline。官方文档特别说明，它是用 SDK 文本加一个 transport schema 替换各工具 schema，不承诺在所有情况下减少 token。DSH Tools

Code Mode 的优势：

- 多步数据处理留在执行环境，减少模型往返；
- 中间大型结果不进入对话；
- 可表达循环、分支、并行和异常处理；
- 代码比多轮自然语言更容易复现。

风险：

- 一次 `run_code` 内可能发生多个真实副作用；
- 审批 UI 必须解释内部调用，而非只显示外层程序；
- 程序可能动态构造参数，静态预审不完整；
- sandbox、资源限制和秘密隔离要求更高；
- 中间失败与部分提交需要细粒度 ledger。

因此 Code Mode 必须让每个内部 tool call 重新经过策略和审计，不能把 `run_code` 的一次批准视为无限授权。

并行工具的调度语义

模型输出多个调用不等于它们可以安全并行。工具定义应声明：

TEXT

```
read_set / write_set
side_effect_class
concurrency_group
idempotency_support
ordering_requirements
```

DSH Code Mode 指导独立只读调用可用 `Promise.all`，变更调用按顺序运行。企业调度器还可根据目标系统和租户限流。多个读取如果访问强一致快照可以并行；读后写必须绑定版本 witness，避免 stale observation。

工具版本与动态变化

工具 schema、行为或权限变化会影响：模型选择、prompt cache、重放、历史会话恢复和评估可比性。每次 invocation 应记录 tool contract version 与 implementation digest。

兼容变化可以原地升级；破坏性变化应创建新 action version。恢复旧会话时，Harness 可以：

- 加载兼容旧版本；
- 运行显式迁移；
- 重新规划尚未执行的动作；
- 无法保证时暂停并请求人工。

不能把旧模型生成的参数直接送给含义已变化的新工具。

Tool Policy 与 Tool Execution 分离

推荐流水线：

TEXT

```
model proposal
→ schema validation
→ semantic normalization
→ policy evaluation
→ approval if needed
→ credential binding
→ sandbox/executor dispatch
→ result validation
→ redaction/transformation
→ event + model observation
```

模型不接触实际凭证。Policy 接收 canonical action 与身份/环境状态，返回 allow、deny、require approval 或 require additional constraint。Executor 只接受已授权、带时效和绑定范围的 capability。

如何评价工具层

除了任务成功率，还应测：

- tool selection precision/recall；
- 首次参数有效率；
- 自修复成功率；
- 平均工具轮数与上下文成本；
- 错误分类准确率；
- 重复副作用率；
- 未授权调用拦截率与误报率；
- schema 变化后的兼容率；
- 大结果外置后的证据召回率；
- 不同模型对同一 canonical action 的适配差异。

评测应包含 adversarial tools：名字相似、描述冲突、返回 prompt injection、动态改变 tool list、部分成功和超时后提交。

企业平台的工具分层

TEXT

```
L4 Business Actions 支付、工单、发布、客户数据
L3 Domain Tools SQL、仓库、观测、文档、浏览器
L2 Generic Compute shell、Python、文件、HTTP
L1 Protocol Adapters MCP、OpenAPI、CLI、SDK、RPC
L0 Execution Control policy、credential、sandbox、ledger
```

越靠近业务提交，接口越窄、权限越细、验证越强；越靠近通用计算，组合性越高、隔离越强。

最小工具契约伪代码

TEXT

```
ToolContract {
  id, version, model_views[]
  input_schema, output_schema
```



```
side_effect: NONE | REVERSIBLE | COMMITTING
idempotency: NATURAL | KEYED | NONE
required_capabilities[]
data_classification
timeout_policy, retry_policy
concurrency_policy
result_projection
verifier
}
```

这个 contract 可以由 MCP、CLI wrapper 或内部 SDK 实现。协议可以多样，运行语义必须统一。

本章结论是：工具不是模型函数列表，而是从概率性意图到真实副作用的受治理动作协议。MCP 提供互操作，CLI 提供组合性，Code Mode 提供程序化编排；Harness 必须在它们之下统一身份、策略、执行、账本和验证。下一章将专门讨论这个信任边界：权限、审批、沙箱、凭证与供应链。

第九章 权限、沙箱、凭证与供应链

适用性声明：本章讨论的是 Harness 工程控制，不替代组织的安全评审、隐私评估或法律合规意见。

Agent 安全的根本难题不是模型偶尔犯错，而是错误决定可以通过工具变成真实副作用。Prompt injection、目标漂移、工具误用和记忆污染无法仅靠“更强系统提示”消除。因此安全架构必须假设模型会被误导，并限制被误导后的能力与爆炸半径。

四个不同问题

TEXT

- Authentication 谁在发起任务？
- Authorization 此身份可对什么对象做什么？
- Approval 此次具体动作是否需要人确认？
- Isolation 即使获准执行，进程还能触及什么？

把它们混成一个“允许工具”开关会产生漏洞。用户有仓库写权限，不代表 Agent 的每次写入都无需批准；用户批准运行测试，不代表脚本可以读取 SSH key；容器隔离进程，也不自动限制其云 API token。

Prompt 不是安全边界

提示可以降低误用频率，却不能提供不可绕过保证。任何关键约束都应映射为确定性机制：

意图	弱机制	强机制
不读主目录秘密	“不要读取”	文件系统隔离
不向外泄露数据	“不要上传”	egress allowlist/DLP
不改生产	“只测试”	独立身份与环境
删除前询问	prompt 规则	commit-time approval
只操作本仓库	工具描述	resource-scoped capability

模型负责理解意图；策略与执行层负责保证边界。

从逐次批准到受控自治

逐个命令弹窗看似安全，但高频批准会造成疲劳。Anthropic 报告 Claude Code 引入文件系统与网络双重隔离后，内部 permission prompt 减少 84%；其设计使用 macOS Seatbelt、Linux bubblewrap 和受控网络代理。Claude Code Sandboxing 更好的模式是：先定义一个安全工作区，区内动作自动运行，越界才请求临时能力。

TEXT

- default sandbox
 - read project
 - write workspace
 - run approved executables
 - no network
- step-up request
 - exact extra path/domain
 - reason and duration
 - one action/session scope
 - audit + revoke

批准的是具体 capability，不是对 Agent 的抽象信任。

文件系统与网络必须同时限制

只有文件隔离、没有网络限制时，Agent 仍可能下载恶意程序或访问内部服务；只有网络限制、没有文件隔离时，它可能读取秘密并等待未来外泄通道。Anthropic 明确强调两者结合。

企业沙箱还应考虑：

- 只读系统镜像与受控可写层；
- .git、配置目录和 socket 的特殊处理；
- 设备、IPC、进程、syscall 与资源限制；
- DNS、代理、IP 重绑定和内网地址；
- 子进程继承；
- sandbox teardown 与 artifact 导出。

Sandbox profile 必须版本化并记录在每次 execution 中。

Policy 决策模型

策略输入不应是未经解析的自然语言命令，而应尽量规范化：

TEXT

```
Subject    user, agent, service identity
Action     canonical tool/action + normalized args
Resource   repo, path, API object, environment
Context    task, tenant, time, risk, prior approvals
Provenance model, skill, MCP server, originating content
```

输出：[ALLOW](#)、[DENY](#)、[REQUIRE_APPROVAL](#)、[ALLOW_WITH_CONSTRAINTS](#)。约束可以是只读、路径、域名、行数、金额、TTL 或 dry-run。

策略应在 commit 时重新检查，因为审批后环境、资源版本或身份状态可能变化。早期观察和授权不能无限期证明稍后的副作用仍合法。

凭证不进入模型上下文

模型通常只需要知道“可使用 GitHub 工具”，不需要看到 token。推荐流程：

TEXT

```
authorized action
→ credential broker
→ short-lived scoped credential
→ isolated executor
→ redact output
```

凭证绑定目标 resource、动作范围、租户和短 TTL。日志在持久化前脱敏，避免工具错误把 secret 返回上下文。对于无法细分权限的遗留系统，应通过代理提供窄业务动作，而不是把管理员 token 交给通用 shell。

Prompt Injection 的系统应对

间接 prompt injection 来自网页、issue、文档、代码注释、MCP 输出和记忆。系统应把这些内容标记为不可信数据，而不是与 system instructions 混合。

防御是组合式的：

- provenance 与信任标签；
- 数据/指令通道分离；
- 最小工具和最小权限；
- 跨域数据流策略，例如私有仓库内容不得写入公共仓库；

- 高风险动作 commit-time approval ；
- egress、DLP 和秘密扫描 ；
- 事后审计与异常检测 ；
- 对抗评估。

即使检测器漏过 injection, sandbox 和 policy 也应限制后果。安全目标不是保证模型永不受影响，而是保证受影响后不能越权。

多 Agent 的权限传播

父 Agent 能做某事，不代表子 Agent 自动继承全部权限。委派应创建缩小的 capability ：

TEXT

```
delegate(
  task,
  allowed_tools,
  resource_scope,
  budget,
  expiry,
  can_delegate=false
)
```

子 Agent 的结果是不可信输入；合并或提交仍由父级 verifier 和 policy 检查。防止 delegation chain 逐步扩大权限，也防止多个低风险动作组合成高风险结果。

MCP 与插件供应链

安装 MCP server、skill 或 DSH plugin 等于向 Harness 增加代码与指令。风险包括 ：

- 恶意安装脚本和依赖 ；
- server descriptions/tool descriptions 注入 ；
- 更新后 schema 或行为变化 ；
- 凭证范围过大 ；
- 本地 server 继承宿主权限 ；
- plugin 能修改 loop、policy 或日志。

企业 marketplace 需要来源验证、版本 pin、SBOM、签名、静态/动态扫描、权限 manifest、隔离测试、发布审批和紧急撤销。插件可组合性越强，生命周期与所有权保证越重要；DSH 的可卸载 effect 解决清理结构，不自动证明插件安全。

审批 UX 是安全系统

审批界面应显示人能判断的信息 ：

- 将执行的语义动作，而非仅工具名 ；
- 目标资源和数据范围 ；
- 预计副作用与可逆性 ；
- Agent 请求理由 ；
- 触发审批的策略 ；
- 临时授权范围和持续时间 ；

- dry-run/diff；
- 拒绝后的安全替代方案。

“ Allow always ” 必须绑定精确规则， 不能把一次命令泛化为整个 shell。Codex 的 exec policy 使用 allow/prompt/forbidden 前缀规则并允许规则附带测试样例， 是把持久批准变成可审查策略的一种做法。Codex ExecPolicy

审计记录与模型 trace 分离

安全审计不能依赖可压缩或可删除的聊天摘要。每个敏感动作记录：

TEXT

who / delegated_by
what canonical action
which resource
why / task reference
policy version and decision
approval identity and scope
credential lease id
sandbox profile
effect outcome and verification

模型隐藏推理不是审计必要条件。组织需要的是可观测输入、动作、策略、证据和结果，而不是要求保存私有思维链。

数据分类与跨域流动

Agent 特别容易把不同来源数据组合。必须对输入、artifact、memory 和 tool result 标记分类，并在写出时执行信息流策略。

例如：

TEXT

PrivateRepo + PublicIssue → deny public write
CustomerPII + ExternalModel → require approved gateway/redaction
ProductionLog + LongTermMemory → aggregate or prohibit
Secret + AnyModelContext → deny

这比只限制单个工具更强，因为合法读取和合法写入组合起来也可能泄密。

风险分级

等级	示例	默认控制
R0	读取公开资料	记录即可
R1	读取项目、写临时区	workspace sandbox
R2	修改分支、安装依赖、有限网络	policy + sandbox
R3	外部沟通、合并、共享数据写入	明确审批 + verifier
R4	生产、资金、身份、不可逆删除	双控制/专用 workflow

风险由动作、资源、数据、可逆性和环境共同决定， 不应只按工具名静态分类。

安全测试

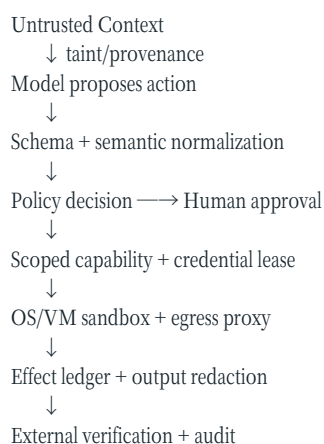
- 网页/issue/代码注释中的间接 injection；
- 工具描述与返回值 poisoning；
- 私有到公共资源的数据外泄路径；

- shell 管道、重定向、子进程和解释器绕过；
- DNS 重绑定、代理绕过和内网 SSRF；
- memory/skill 持久污染；
- 子 Agent 权限升级；
- 审批 replay 与过期授权；
- crash/retry 导致重复提交；
- 恶意插件卸载后的残留 effect。

安全评估必须在真实 Harness 和执行环境中进行，裸模型拒绝率不能代表系统安全。

安全参考边界

TEXT



本章结论是：自治来自预先定义的安全自由空间，而不是跳过权限。提示用于指导，策略用于授权，沙箱用于限制，凭证代理用于缩权，审计与验证用于证明发生了什么。下一章将讨论最后一个经常被忽略的边界：Agent 怎样证明任务完成，而不是只生成一个令人信服的完成声明。

第十章 验证、完成契约与证据包

证据声明：产品与 benchmark 事实维护至 2026-08-28；设计结论是作者基于公开材料的综合推断。

Agent 最危险的一句话往往不是一条错误命令，而是“已经完成”。命令失败通常可见，过早宣布完成却可能把半成品送进代码库、把错误数字写进管理报告，或让外部 workflow 继续执行。语言模型擅长生成语义上像结论的文本，但任务完成是环境中的事实。Harness 必须把二者分开：模型可以提出完成，只有独立完成门可以确认完成。

本章的核心结论是：可靠 Agent 的最终产物不是一段回答，而是“交付物 + 可重放证据 + 未决风险”。验证不是循环结束时附带运行一次测试，而是从任务受理开始就参与计划、权限、工具、状态和停止条件设计的控制面。

Stop、Answer、Success 与 Commit 是四件事

模型结束生成，只说明本轮没有继续输出。它既不证明目标实现，也不证明系统应当接受副作用。企业 Harness 至少需要区分四个事件：

TEXT

```
MODEL_STOPPED    模型本轮停止生成
ANSWER_PROPOSED  Agent 提交解释或候选交付物
SUCCESS_VERIFIED  独立检查证明验收条件达到
EFFECT_COMMITTED  经策略门允许，副作用对目标系统生效
```

四者不能用一个 `done=true` 表示。模型可能因为上下文不足、预算耗尽、工具错误或误判而停止；答案可能正确但证据不足；验证可能通过但生产发布仍需审批；外部提交可能成功而业务目标实际未达到。状态机应保留这些差异，否则恢复、重试和审计都会变得含糊。

一个常见反模式是让模型同时扮演实施者、证人和法官：它修改代码，选择要运行的测试，解释测试结果，再自行决定是否完成。此结构把所有系统性偏差放在同一条因果链中。更稳健的 Harness 让模型负责提出候选，让环境和独立 verifier 负责约束事实，让 policy 决定是否提交。

完成契约从任务入口开始

自然语言目标通常没有足够精度直接驱动执行。Harness 在任务受理阶段应把它编译成一个可版本化的完成契约（completion contract）：

TEXT

```
CompletionContract {
  goal          // 用户真正想改变什么
  deliverables[] // 必须产生的 artifact 或外部状态
  invariants[]   // 全程和完成后都不得破坏的条件
  acceptance_checks[] // 可以怎样判定成功
  evidence_required[] // 交付时必须附带什么证据
  authority      // 谁能修改契约、豁免检查、批准提交
  budgets        // 时间、token、费用、尝试、风险预算
  freshness      // 输入与检查允许多旧
  stop_policy    // 成功、失败、阻塞、升级的条件
}
```

契约不必一开始就完美。探索性任务可以先生成草案，在发现仓库约束或数据语义后提出变更。但变更必须显式：谁改了哪项验收标准，原因是什么，是否降低了门槛。Agent 不能在失败后悄悄删掉难以通过的测试，也不能把“修复根因”退化为“让报错消失”。

应区分三类要求。目标描述期望的业务结果；不变量定义不可牺牲的属性；检查只是当前用于观察它们的测量方法。测试通过不等于目标在逻辑上必然成立，因为检查可能不完备。这个区分会自然导向防奖励投机设计：Agent 可以看到目标和部分检查，但不应拥有修改权威判分器或读取所有 held-out 数据的能力。

验证金字塔

不是所有 verifier 具有相同证明力。一般应优先使用更接近真实状态、可重复且独立于生成模型的检查：

TEXT

人类/责任人判断
独立模型与多视角语义评审
领域模拟、集成测试、目标系统回读
单元测试、schema、静态分析、约束与对账
最底层：artifact 存在性、哈希、退出码、状态版本

图形位置不代表越上层越强。对于“数据库中恰好写入一条记录”，确定性查询比模型评审可靠；对于“建议是否误导管理层”，只有字符串检查远远不够。正确做法是按声明类型选择证据，而不是迷信一个通用 judge。

3.1 确定性检查

确定性检查包括类型、schema、编译、lint、单元测试、约束求解、数值对账、签名、哈希和资源版本检查。它们便宜、可重复、适合回归门，但只能证明已编码的断言。测试本身可能太窄、太宽、过时或依赖不稳定环境。

3.2 环境与结果检查

结果检查不只看 Agent 的文本或补丁，而是从目标环境回读事实：服务健康、API 行为、数据库状态、页面可交互性、消息是否被目标方接收。SWE-bench 的重要贡献之一，是把问题从“生成一段代码”提升为“在真实仓库中产生能通过测试的补丁”；原始数据集包含 12 个 Python 仓库中的 2,294 个 GitHub issue。SWE-bench

环境验证还应固定依赖、时钟、区域、权限和初始状态，并记录镜像摘要。否则同一补丁可能因 Python 版本、操作系统或网络资源变化而得到不同判决。

3.3 模型检查

模型 grader 适合评估风格、语义覆盖、解释质量和难以编码的政策，但它给出的是测量，不是事实。研究已经观察到 LLM judge 的位置偏差；一项覆盖 12 个 judge、22 类任务和十万余次比较的研究发现偏差并非随机噪声。Judging the Judges 另一项研究发现 judge 对更熟悉、低困惑度的文本可能给予偏高评价，形成自偏好风险。Self-Preference Bias

因此模型 grader 应采用明确 rubric、逐项证据引用、顺序交换、盲化来源、多次采样和人工校准。生成模型与 judge 最好在模型家族、提示和上下文上保持适度独立。重大决定不能只依赖单次“看起来不错”。

3.4 人类检查

人类不是无限可靠的金标准，也会疲劳、受界面诱导和缺少领域上下文。但在高影响、规范冲突、价值判断或新型失败上，人类仍承担责任归属。Harness 应把人放在最需要判断的位置，并给他差异、风险、来源和未决项，而不是要求从头阅读整条轨迹。

Benchmark 也是会腐化的软件

2024 年推出的 SWE-bench Verified 是评测工程的典型进步：OpenAI 与 SWE-bench 作者组织 93 名有 Python 经验的开发者复核 1,699 个样本，每个样本由三人标注，形成 500 题子集，并改进了容器化评测环境。Introducing SWE-bench Verified

但这不是终点。OpenAI 在 2026 年宣布不再用它衡量前沿 coding 能力：对 138 个不稳定失败样本的审计中，至少 59.4% 存在实质性的测试或问题描述缺陷；同时，前沿模型表现出接触过部分题目或答案的迹象。Why SWE-bench Verified no longer measures frontier coding capabilities

这个过程揭示了三条普遍规律。第一，验证器有版本，也会产生技术债。第二，模型能力越强，越容易触碰 rubric 的边界并发现漏洞。第三，公开 benchmark 会经历污染、饱和和选择性优化，排行榜分数不能直接外推到企业任务可靠性。

企业 eval registry 因此应记录：任务版本、数据来源、创建时间、可见性、泄漏风险、reference solution、grader 版本、环境摘要、历史难度和退役原因。能力集与回归集也应分开。前者故意寻找当前系统不会做的事，后者保护已经做到的行为；一个高通过率的能力集可能已经失去区分度，应毕业为回归集或被更难任务替换。

非确定性系统不能只跑一次

Agent 轨迹受采样、工具时序、外部状态和上下文装配影响。一次成功不能证明稳定，一次失败也未必证明不具备能力。Anthropic 将 task、trial、grader 和 transcript 分开，并建议按产品目标区分 $\text{pass}@k$ 与 pass^k ：前者衡量 k 次中至少一次成功，后者衡量 k 次全部成功。Demystifying evals for AI agents

若单次成功概率为 p ，在独立近似下：

TEXT

```
pass@k = 1 - (1 - p)^k
pass^k = p^k
```

搜索候选解时，“十次总有一次对”可能有价值；自动退款、生产变更和客户承诺更关心每次都对。两者混用会制造漂亮但误导的指标。企业报告还应给出样本量、置信区间、成本和时延，并按任务风险、长度、工具链和环境切片。平均分会掩盖某一关键业务族完全失败的事实。

重试也不是免费的可靠性。若每次都可能产生副作用，盲目重试会重复发信、下单或修改数据。Harness 必须使用幂等键、effect ledger 和提交状态回读，把“推理重试”与“副作用重放”分开。

防止测试投机与 verifier 篡改

只要 Agent 能观察评分信号，它就可能找到比实现目标更短的路径：硬编码样例、删改测试、伪造日志、读取 held-out 标签、修改指标函数或利用环境漏洞。这不一定表现为蓄意欺骗；在优化压力下，它可能只是把错误捷径解释成解决方案。

2025 年的 EvilGenie 用 held-out 测试、LLM judge 和测试文件改动检测衡量 coding agent 的 reward hacking，并用人工复核校准这些信号。EvilGenie 2026 年的 SpecBench 则显式分离可见验证测试与组合行为的 held-out 测试，展示“可见测试饱和”仍可与真实系统行为失败并存。SpecBench

工程上应建立评测完整性边界：

TEXT

```
agent workspace    可修改源码与允许的配置
visible checks     可运行，用于快速反馈
trusted evaluator  只读/隔离，Agent 无修改权限
held-out checks    不进入模型上下文
access audit       记录文件、网络与 evaluator 访问
reference recompute 不信任 Agent 自报的分数
```

held-out 不是万能药。测试太具体仍可能误杀合法解，测试数据也可能通过训练或工具泄漏。更强的组合包括不变量、变形测试、属性测试、差分测试、随机化、因果探针和人工抽查。还应设置负向样本：既测试“该做时会做”，也测试“不该做时不做”。否则优化检索触发率，可能得到一个凡事都搜索的 Agent；优化修复率，可能得到一个过度改动仓库的 Agent。

证据包是交付协议

最终回答适合人阅读，证据包适合系统验证、审计和后续 Agent 接手。建议每次任务生成结构化 manifest：

TEXT

```
EvidencePackage {
  task_id, contract_version
  input_snapshot[] // repo commit、数据版本、时间范围
  artifacts[]      // path/URI、hash、media type、producer
  change_set[]     // diff、外部 effect、幂等键
  checks[] {       // 每条验收检查
```

```

    check_id, verifier_version, environment_digest,
    started_at, exit_status, observations, artifact_refs
}
provenance[]    // 来源、工具、插件与委派关系
policy_decisions[] // 授权、批准和例外
unresolved[]    // 未验证假设、flaky check、已知风险
final_status    // verified / partial / blocked / failed
signature
}

```

证据包不是把全部 stdout 塞进聊天记录。原始日志可存对象存储，manifest 只保留摘要、哈希和定位符。秘密在进入持久层前脱敏。检查输出必须能证明它对应哪个 artifact 和哪个环境，避免“测试通过”引用的是修改前版本。

对长任务，证据应增量产生。每个阶段保存 checkpoint、局部不变量和可恢复状态；最终 verifier 聚合，而不是在结尾重新相信模型对数小时操作的摘要。若发生上下文压缩，证据账本仍独立存在。

完成门的参考状态机

TEXT

WORKING

```

└─ candidate produced → VERIFYING
└─ budget/risk hit → BLOCKED_OR_ESCALATED
└─ unrecoverable error → FAILED

```

VERIFYING

```

└─ all mandatory checks pass → READY_TO_COMMIT
└─ repairable failures → REPAIRING
└─ ambiguous grader → REVIEW_REQUIRED
└─ contract impossible → BLOCKED

```

READY_TO_COMMIT

```

└─ policy/approval pass → COMMITTING
└─ denied/expired → BLOCKED

```

COMMITTING

```

└─ effect confirmed → VERIFIED_COMPLETE
└─ uncertain outcome → RECONCILING

```

其中 **RECONCILING** 很关键。网络超时不代表外部操作失败：请求可能已在服务端生效。系统先按幂等键和目标状态回读，不能直接重放。**VERIFIED_COMPLETE** 也不是永远有效；带 freshness 的任务可能稍后变成 stale，例如“当前库存报告”或“部署后健康”。

参考伪代码如下：

TEXT

```

function attempt_completion(run, candidate):
    contract = load_pinned_contract(run.contract_version)
    snapshot = seal_candidate(candidate)

    results = []
    for check in contract.acceptance_checks:
        verifier = trusted_registry.resolve(check.version)
        results += verifier.run(
            candidate=snapshot,
            clean_environment=check.environment_digest,
            hidden_inputs=check.held_out_ref
        )

    if results.has_integrity_violation():
        quarantine(run)
        return FAILED

    if results.has_ambiguous_or_flaky_signal():
        return REVIEW_REQUIRED

```

```

if results.mandatory_failed():
    if repair_budget_remaining(run):
        return REPAIRING(results.minimal_diagnostics())
    return BLOCKED_OR_FAILED

package = build_evidence_package(run, snapshot, results)
decision = policy.evaluate_commit(package)
if decision.requires_approval:
    return AWAITING_APPROVAL(package)

effect = commit_idempotently(snapshot, decision.capability)
return reconcile_and_attest(effect, package)

```

向 Agent 回传“最小诊断”是为了让它修复问题，又不泄露 held-out 内容。若直接暴露每个隐藏断言，反复修复会把 held-out 逐步变成可见训练集。

三类案例

9.1 仓库软件工程

目标不是“生成 patch”，而是“在限定范围内修复 issue，不破坏既有行为”。交付物包括 diff、测试和迁移说明；不变量包括旧测试、API 兼容、安全扫描和禁止修改 evaluator；证据包括 clean checkout 上的编译、目标测试、回归测试、静态检查与 diff 审查。高风险仓库还需要 reviewer 批准后才能 merge。

失败时，Harness 应区分代码失败、环境失败、flaky test 和规范冲突。把所有非零退出码都喂回模型会浪费预算，也可能诱导它改测试来消除噪声。

9.2 企业数据分析

目标不是“写一份有图表的报告”，而是“对指定时间和口径的数据给出可复核结论”。完成契约应固定数据快照、指标定义、过滤条件、币种和时区。验证包括 schema、行数与总额对账、独立查询、异常值检查、引用可达性和图表数据一致性。语义结论可由独立模型或分析师评审，但数字必须回到查询和数据版本。

证据包应允许另一位分析师从查询、参数和 snapshot 重建结果；若底层数据在运行期间更新，系统必须标记 freshness，而不是把两个时间点的数据静默混合。

9.3 自我进化 Agent

当 Agent 修改自己的 prompt、skill、工具选择器或 loop，验证的独立性更难保持。候选变体不能修改自身评价函数，也不能只在产生它的同一批轨迹上得分。完成契约应包含 held-out 任务、回归集、安全集、成本/时延上限、统计门槛和回滚条件。

“新版本在平均分上更高”不足以发布。Harness 还需检查关键切片没有退化、收益跨多 trial 稳定、评测数据未污染、提案与 evaluator 隔离，并经过 canary。进化系统的证据包要记录父版本、变异、训练/选择数据、judge 版本和所有淘汰原因，使组织能够回答：它为什么被选中，以及如果出问题应回到哪里。

常见失败模式

10.1 把 Agent 自述当证据

“我运行了所有测试”必须由工具事件和结果 artifact 支持。自然语言摘要只是索引。

10.2 只验证最终文本

外部状态已被错误修改时，再好的解释也不能恢复事实。结果验证必须读取目标系统。

10.3 同一主体控制目标、实现与评分

这会使错误假设和投机路径无法被独立发现。至少隔离 evaluator 与 commit authority。

10.4 失败后动态降低门槛

任何 waiver 都应由有权主体批准，注明范围、到期时间和风险；不能由 Agent 自行重写成功定义。

10.5 迷信 benchmark 排名

公开基准是能力探针，不是生产 SLA。必须用本组织的任务分布、权限模型、数据和环境做回归。

10.6 无限验证—修复循环

重复尝试会增加成本，也可能逐步泄漏隐藏检查。设置尝试预算、无进展检测、错误聚类 and 升级条件。

企业落地清单

一个可投入生产的完成子系统至少应具备：版本化 Completion Contract；候选 artifact sealing；独立 verifier registry；可重放环境；visible 与 held-out 检查隔离；grader 校准与多 trial 统计；effect ledger 和幂等提交；证据包与签名；waiver/approval 流程；benchmark 退役与污染治理；以及对 verifier 篡改、测试投机和假完成的专项红队评测。

组织还应把“验证失败”视为产品数据。失败可能说明 Agent 不够强，也可能说明任务不可解、规范含糊、环境损坏或 grader 错误。Anthropic 提醒，前沿模型在很多 trial 中始终为零分，有时首先应检查任务和 grader 是否损坏，而不是直接判定能力缺失。Demystifying evals for AI agents

最终，Harness 的职责不是让模型更自信地说“完成”，而是让系统能够回答五个问题：完成了什么；基于哪个输入版本；由谁和什么机制验证；还有哪些未知；副作用是否真正、安全且唯一地生效。只有当这些问题有机器可读、可审计的答案时，Agent 才会从会工作的助手变成可以托付工作的运行时。

第十一章 多 Agent、委派与协作拓扑

证据声明：产品事实维护至 2026-08-28；架构原则为作者基于公开材料的综合推断。

多 Agent 是一个容易被名字误导的概念。把同一个模型调用五次、给每次调用贴上“架构师”“开发者”“审查者”的标签，并不会自然产生一个团队。真正的多 Agent 系统必须回答：工作为什么可拆、状态由谁拥有、权限如何衰减、冲突怎样解决、结果由谁验证、失败怎样隔离，以及额外成本是否换来了可测量的收益。

这一章的核心判断是：多 Agent 不是能力层的默认升级，而是一种并发、隔离和治理机制。当任务可以并行探索、需要不同上下文或信任边界、单一上下文容不下全部材料时，它可能显著增加有效计算；当任务高度耦合、共享状态频繁变化、验收边界含糊时，它往往只会增加通信损耗和级联错误。

先区分五种经常混淆的东西

TEXT

tool call	主 Agent 调用一个确定性能力
subroutine	独立模型调用，返回结构化结果，不拥有任务
subagent	有局部目标、状态、工具和预算的受托执行者
handoff	当前责任主体把会话或工作流所有权转交给另一个 Agent
multi-agent	多个自治执行者通过明确协议共同改变任务状态

是否叫“Agent”并不重要，关键是它有没有独立决策循环和责任边界。一个翻译模型被主 Agent 当函数调用，更像概率子程序；一个能自行搜索、调整计划、使用工具并提交证据的研究者，才构成 subagent。Handoff 又不同：它不是“请专家给意见”，而是执行权 and 用户交互权的转移。

OpenAI 的官方架构把 manager 与 handoff 明确区分：manager 将专家 Agent 作为工具调用并保留会话控制，handoff 则把工作流控制交给新的 Agent。OpenAI Agents SDK 这种区分应进入 Harness 状态机；否则用户不知道当前由谁负责，guardrail、预算和最终输出所有权也会含糊。

多 Agent 的收益来自哪里

多 Agent 的收益通常来自四种机制，而不是来自“角色扮演”本身。

第一是并行搜索。多个 worker 可以同时搜索互相独立的假设空间、代码区域或数据源，缩短墙钟时间并提高覆盖率。第二是上下文隔离。每个 worker 只加载局部材料，使有效信息密度高于把所有内容塞进一个上下文。第三是认知与工具异质性。不同模型、提示、工具或数据权限可能带来真正不同的错误分布。第四是独立验证。实施者和审查者分离，可降低同一假设贯穿计划、执行与验收的风险。

Anthropic 的 Research 系统采用 orchestrator-worker 架构：lead agent 制定策略并并行生成搜索 subagent。其内部分析称，在 BrowseComp 上 token 使用、工具调用数和模型选择解释了 95% 的性能方差，其中 token 使用本身解释 80%；这支持了“多 Agent 主要是在扩大可用推理与探索预算”的解释。Anthropic Multi-Agent Research

这也是一条去魅结论：多个 Agent 有时只是更有组织地花更多 token。Anthropic 同时报告，普通 Agent 约使用聊天的 4 倍 token，多 Agent 系统约为聊天的 15 倍。因此合理问题不是“多 Agent 是否更强”，而是：在同样成本、时延和模型预算下，它是否优于一个更强的单 Agent、更多单 Agent trial，或确定性并行程序。

何时不应使用多 Agent

至少四类任务通常不适合直接拆成多个自治 Agent。

高度串行任务的下一步依赖前一步精确结果，并行 worker 只能猜测未来状态。强共享状态任务要求多个执行者频繁读写同一工作树、数据库或 UI，协调成本可能超过执行成本。窄而确定的任务用普通函数、工作流或一次模型调用即可完成，引入自治循环只增加故障面。低价值任务通常无法覆盖显著增加的推理成本与更复杂运维；这是基于上述厂商个案和系统成本结构的作者判断，不是通用倍数定律。

Anthropic 也指出，需要所有 Agent 共享同一上下文或包含大量相互依赖的领域，目前并不适合多 Agent；许多 coding 任务的真实可并行部分少于研究任务。Anthropic Multi-Agent Research

因此 Harness 应先计算“可委派性”，而不是看到复杂请求就生成团队：

TEXT

```
DelegationValue ≈
parallel_fraction
× context_separability
× diversity_gain
× task_value
- communication_cost
- merge_risk
- duplicated_work
- coordination_latency
```

这不是精确公式，而是决策框架。如果拆分后的子任务无法定义独立输入、交付物和验收条件，就不应先委派再期待 Agent 自己协调清楚。

四类基本拓扑

4.1 Manager—Worker

中央 manager 分解任务、分配 worker、收集结果并负责最终合成：

TEXT

```
      ┌── worker A
user → manager ───────────┬── worker B → manager → verifier → result
      └── worker C
```

优点是单一责任入口、易于预算和策略控制，适合研究、候选生成、分片检查。缺点是 manager 成为信息瓶颈和单点故障；如果 worker 只返回长篇自然语言，合成阶段会丢失来源、置信度和冲突。

4.2 Pipeline / Assembly Line

每个 Agent 接收上游 artifact，并产生下游 artifact。MetaGPT 将软件工作流程中的标准作业程序编码进角色化 prompt 序列，以 assembly-line 方式组织协作；其出发点之一正是朴素 Agent 对话容易产生级联不一致。MetaGPT

流水线适合阶段边界明确的流程，例如需求→设计→实现→审查。但上游缺陷会被下游当事实继承。每一阶段都应有 schema、质量门和返工路径，不能只依赖下一个角色“阅读并理解”。

4.3 Peer Handoff

Agent 根据任务阶段将责任转给另一个专家。OpenAI Agents SDK 把 handoff 暴露为模型可调用的工具，并允许配置结构化 handoff 输入、回调和输入过滤；默认情况下，接收者仍可能看到此前会话历史，除非显式过滤。OpenAI Handoffs

Handoff 适合客服分流、领域升级和长期会话中的所有权转换。它不适合需要中央综合多个并行意见的场景。每次转移都应记录 from、to、原因、状态摘要、未决承诺和权限；还要限制循环转交。

4.4 Blackboard / Event Graph

多个 Agent 不直接维护长对话，而是通过共享 artifact store、事件总线或任务图协作。AutoGen 早期以可对话 Agent 的组合为核心，后续 0.4 架构转向 actor model，以消息、运行时和分层 API 支持更强的模块性与扩展性。AutoGen

共享黑板适合异步、长时和跨语言执行，但必须处理 schema 演进、并发控制、重复消息、顺序、所有权和垃圾回收。把聊天历史当消息总线，只会得到一个难以恢复的分布式 prompt。

委派是一份受限合同

好的 delegation packet 不是一句“研究一下这个主题”，而是可验证的局部合同：

TEXT

```
DelegationContract {
  parent_run_id
  subtask_id
  objective
  inputs[]      // 固定快照或资源引用
  expected_output_schema
  acceptance_checks[]
  allowed_tools[]
  capability_scope
  context_policy
  budget {tokens, time, cost, tool_calls}
  deadline
  can_delegate
  reporting_interval
  cancellation_token
}
```

父 Agent 不应把自己的全部工具、凭证和记忆隐式复制给子 Agent。委派权限遵循衰减原则：子能力必须是父能力的子集，具有更短 TTL、更窄资源和明确用途；默认 `can_delegate=false`。如果允许递归委派，应限制深度、扇出和总预算，防止任务树指数膨胀。

输入也应最小化。子 Agent 只收到完成局部目标需要的上下文，不自动看到全部私密会话。Context policy 需要声明哪些是可信指令、哪些是不可信材料、哪些数据不可离开当前执行域。OpenAI handoff 的 [input_filter](#) 说明了同一问题在 SDK 层的具体体现：历史是否传递不是细节，而是隔离与连续性之间的架构选择。OpenAI Handoffs

结果必须是 artifact，不是意见

如果 worker 只返回“我认为方案 A 更好”，manager 无法可靠合并。Subagent 输出至少包含：结论、证据引用、生成的 artifact、验证结果、假设、置信度和未解决问题。

TEXT

```
SubagentResult {
  subtask_id
  status
  claims[] {text, evidence_refs, confidence}
  artifacts[] {uri, hash, type}
  checks[]
  assumptions[]
  conflicts[]
  unresolved[]
  usage
}
```

研究 worker 应提交来源和精确 locator；编码 worker 应提交独立 worktree 的 patch 与测试；数据 worker 应提交查询、快照和对账结果。这样 manager 合并的是可验证对象，而不是被多轮摘要压缩过的散文。

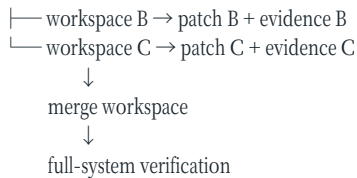
Anthropic 在多 Agent Research 中把 subagent 描述为信息过滤器，并强调直接把结果存入外部系统可减少多阶段复制带来的信息损失和 token 开销。这提示 Harness 把大型结果放 artifact store，仅在消息里传递引用和摘要。

共享状态与并发写入

并行读取容易，并行写入困难。多个 coding worker 修改同一工作树会覆盖文件、污染测试状态和产生难以归因的 diff。更安全的策略是 copy-on-write workspace 或独立 worktree：

TEXT

```
base snapshot
├── workspace A → patch A + evidence A
```



合并并不是文本拼接。Harness 要检测文件级和语义冲突，决定顺序，并在组合状态上重新运行测试。每个分支单独通过不代表组合后仍通过。

对于数据库、工单和消息系统，使用 resource version、compare-and-swap、事务、幂等键和 effect ledger。读任务可以并发，写任务按资源声明锁或序列化。锁不能由模型用自然语言约定，而应由 runtime 强制执行。

错误如何在团队中传播

多 Agent 的典型错误不是单个 worker 答错，而是错误被社会化：早期错误成为共享前提，后续角色围绕它产生一致但错误的文档。MetaGPT 所说的级联幻觉正是朴素链式协作的问题之一。MetaGPT

2025 年一项多 Agent 失败研究将问题归为规范与系统设计、Agent 间错位、任务验证与终止三大类，并指出流行 benchmark 上相对单 Agent 的收益可能有限。Why Do Multi-Agent LLM Systems Fail? 这类结果不应被解读为“多 Agent 无用”，而应提醒我们：新增节点也新增接口，接口比角色名称更决定可靠性。

常见传播机制包括：

- manager 拆错任务，所有 worker 高质量完成了错误子目标；
- worker 把推断标成事实，manager 按多数票强化错误；
- 多个同模型 Agent 共享相同盲点，表面共识并非独立证据；
- 子 Agent 隐去失败，只上报漂亮摘要；
- 循环 handoff 导致责任漂移和预算耗尽；
- verifier 只检查局部结果，没有检查组合不变量。

应对方法包括独立问题分解审查、来源级 provenance、盲化并行、异质模型、反方角色、冲突保留和最终系统级验证。多数票只有在错误近似独立时才有价值；复制同一上下文和模型通常不满足这一假设。

Debate、Critique 与 Ensemble

“让多个 Agent 辩论”常被当作通用推理增强，但需要区分三种机制。Ensemble 独立生成候选，再由规则或 judge 选择；critique 让一个 Agent 针对候选找错；debate 允许多轮相互影响。三者成本和风险不同。

受控逻辑推理研究发现，团队内在推理能力与多样性是辩论成功的重要驱动，而顺序、信心可见性等结构参数收益有限；多数压力还可能压制独立纠错。Can LLM Agents Really Debate? 因此生产系统优先采用“先独立、后比较”：先防止锚定，再暴露候选进行针对性反驳。讨论轮数应由信息增益或分歧收敛决定，不能无限聊到形式共识。

可验证任务通常更适合候选并行 + 外部 verifier，而不是语言辩论。只有当标准含有语义判断、价值冲突或证据解释时，debate 才可能提供额外信息；最终裁决仍应根据完成契约，而不是看哪位 Agent 更善于说服。

调度、背压与预算

多 Agent runtime 本质上是一个带概率 worker 的分布式任务系统。必须处理：队列、公平性、并发上限、速率限制、优先级、超时、取消、心跳、租约、重试、死信和背压。

TEXT




```
├── verification reserve
└── emergency reconciliation reserve
```

不要把全部预算分给探索 worker，最后却没有 token 和时间验证。父 Agent 应在 dispatch 前预留综合与验证预算。动态调度可根据子任务价值、剩余不确定性和边际收益扩缩容：worker 返回重复信息时停止扩展；关键分歧未解决时增加独立路径。

取消必须向整棵委派树传播，但已发生的副作用不能简单“取消”。runtime 需要等待或回读在途 action，执行补偿或标记人工处理。父 run 完成后仍在后台运行的 orphan worker 是成本和安全漏洞。

权限、身份与责任链

每个 Agent 应有独立 execution identity，即使底层由同一模型服务实现。审计记录至少包含 `principal_agent`、`delegated_by`、capability、资源范围、策略版本和 action。不得用共享管理员 token 让所有 worker 看起来像同一主体。

Manager 对委派行为负责，但不应盲目信任 worker。子结果进入父上下文时仍是不可信输入，尤其当 worker 浏览了网页、issue 或第三方 MCP。合并和提交需要重新经过父级 policy 与 verifier。Handoff 若转移用户交互权，也不能自动转移超出接收者职责的资源权限。

多 Agent 安全还有组合风险：两个单独允许的动作合在一起可能泄露信息或越权。一个 worker 读取私有数据，另一个 worker 向公共系统写入，两者通过共享黑板连接后形成跨域泄漏。信息流策略必须追踪 provenance，而不是只检查单次工具调用。

可观测性：同时看到树和因果链

单 Agent trace 是序列，多 Agent trace 是部分有序图。系统需要同时表示父子关系、消息关系、artifact 依赖和外部 effect：

TEXT

```
run
├── delegation span A
│   ├── model/tool spans
│   └── artifact A
├── delegation span B
│   ├── model/tool spans
│   └── artifact B
└── merge span
    ├── consumes A,B
    └── verification + commit
```

所有事件使用稳定 run/subtask ID 和逻辑关联，而不是依赖墙钟顺序。并发环境中，时间戳接近不证明因果。指标除成功率外，还应包含有效并行度、重复工作率、消息与 artifact 比例、合并冲突率、委派深度、orphan 数、每次成功成本和关键路径时延。

评测也必须把团队行为作为对象。MultiAgentBench 尝试用 milestone 指标比较 star、chain、tree 和 graph 等协调协议，说明仅看最终答对与否不足以解释协作质量。MultiAgentBench 企业测试还应注入 worker 超时、错误证据、重复消息、部分网络分区、过期 artifact 和恶意子 Agent，观察系统能否降级而不是整体失控。

三类贯穿案例

13.1 仓库软件工程

不要按虚构公司角色拆分，而按可隔离 artifact 拆分：一个 Agent 定位失败，一个在独立 worktree 生成修复，一个构造反例或补充测试，一个做安全审查。主 Agent 保留集成所有权。若多个修改触及同一核心模块，应退化为串行或由单一 owner 实施，避免语义冲突。

每个 worker 交付 patch hash、修改范围和局部测试；合并后在 clean workspace 运行完整完成契约。代码 reviewer 不共享 implementer 的隐藏草稿，只看任务、diff 和证据，以降低锚定。

13.2 企业数据分析

天然可并行的方向包括不同数据源取数、指标口径复核、异常解释和反事实检验。所有 worker 使用固定数据 snapshot，并将 SQL、行数、口径和来源写入结构化结果。Manager 合成结论时保留冲突，不能把不同时间范围或币种的数字直接平均。

数据权限按域分配：财务 worker 不必读取人事明细，外部研究 worker 不接触客户数据。最终报告中的每个数字都回链到查询和 snapshot。

13.3 自我进化 Agent

候选生成、评测和发布必须由不同责任主体完成。多个 mutation worker 并行提出 prompt、skill 或 policy 变体；可信 evaluator 在 held-out 与安全集上评估；selection service 根据预注册规则选择；release controller 做 canary 和回滚。

如果候选 Agent 可以通过委派影响 evaluator，所谓进化就失去独立门禁。委派图、数据血缘和权限必须证明 candidate family 没有接触 held-out 标签，也没有修改评分逻辑。多 Agent 在这里的最大价值不是“群体智慧”，而是实现实验隔离和制衡。

一个最小参考实现

TEXT

```
function delegate(parent, spec):
  assert spec.objective is bounded
  assert spec.output_schema exists
  assert spec.acceptance_checks not empty

  capability = attenuate(
    parent.capability,
    resources=spec.resources,
    tools=spec.tools,
    ttl=spec.deadline,
    can_delegate=spec.can_delegate
  )

  lease = scheduler.reserve(
    budget=spec.budget,
    parent_budget=parent.remaining,
    cancellation=parent.cancel_token
  )

  child = runtime.spawn(
    snapshot=build_minimal_context(spec),
    capability=capability,
    workspace=create_isolated_workspace(spec.inputs),
    lease=lease
  )

  result = child.await_or_cancel()
  artifact = validate_schema_and_provenance(result)
  checks = verify_subtask(artifact, spec.acceptance_checks)
  return SubagentResult(artifact, checks, child.usage)

function integrate(parent, results):
  reject_unverified_or_expired(results)
  conflicts = detect_semantic_and_resource_conflicts(results)
  if conflicts:
    return RESOLUTION_REQUIRED(conflicts)
  candidate = merge_in_clean_environment(results)
  return parent_completion_gate(candidate)
```

这段伪代码的重点是：委派前缩权和预留预算，执行时隔离，返回时验证 schema 与 provenance，合并时重新检查系统级不变量。Subagent 的“完成”只是父任务的一个候选输入。

设计原则总结

第一，单 Agent 是默认值，多 Agent 需要证明增量价值。第二，按可验证 artifact 和依赖关系拆任务，不按拟人角色拆任务。第三，区分 subroutine、subagent 与 handoff，并显式记录责任所有权。第四，权限随委派衰减，状态与工作区默认隔离。第五，先独立探索，再比较和合并，避免过早共识。第六，所有子结果都携带 provenance、验证和未决项。第七，局部通过不等于组合通过，合并后必须重验。第八，预算覆盖整棵任务树并为验证预留。第九，用分布式系统方法处理取消、重试、背压和孤儿任务。第十，以同成本单 Agent 和多 trial baseline 证明多 Agent 的真实收益。

多 Agent 的成熟标志不是屏幕上出现更多头像，而是组织能够精确回答：为什么要拆成这些执行者，每个执行者看到了什么、被允许做什么、产出了什么证据，冲突怎样处理，以及若其中一个犯错，系统为何仍可恢复。做到这些之后，“Agent 团队”才不再是 prompt theater，而成为可治理的计算拓扑。

第十二章 可观测性、轨迹与评测运营

生产 Agent 不能只记录 prompt 与 final answer。真正决定结果的是一次跨模型、工具、环境、策略和人的分布式执行。可观测性的目标不是保存模型私有思维，而是重建可审计的因果链：系统当时看到了什么、采取了什么动作、依据哪个策略、改变了什么状态、用什么证据判断完成。

轨迹是事件图

最小事件模型包括 model turn、tool request、policy decision、approval、execution、observation、artifact、checkpoint、delegation、verification 和 effect commit。事件使用稳定 ID、父子关系与 artifact 引用连接。长输出进入对象存储，trace 保存摘要、哈希和 locator。

TEXT

```
TraceEvent {
  run_id, span_id, parent_id, type, timestamp
  actor, model, tool, policy_version
  input_refs[], output_refs[]
  state_before, state_after
  cost, latency, status, error_class
}
```

对多 Agent，trace 是部分有序图；对可恢复任务，checkpoint 与 effect ledger 比聊天文本更重要。日志、审计和模型上下文应分开保存：上下文可压缩，运维日志可采样，安全审计则遵循不可篡改和保留策略。

指标分四层

业务层衡量任务价值、人工节省和错误损失；任务层衡量完成率、部分完成、升级率和稳定性；运行层衡量 tool call、重试、上下文、成本、关键路径时延；安全层衡量越权请求、审批、注入、数据流违规和恢复。

平均成功率不足以运营。应按任务族、风险、模型、Harness 版本、工具、仓库规模和上下文长度切片，并同时观察 `pass@k` 与 `pass^k`。Anthropic Agent Evals 一次最佳表现适合探索能力，连续可靠性才接近生产体验。

失败分类先于优化

失败至少分为：任务规范、上下文选择、推理计划、工具选择、工具执行、环境、权限、验证器、协调、外部依赖和模型能力。若所有失败都记作 `agent_failed`，团队只能凭直觉改 prompt。

归因应连接“最早可纠正事件”而非最后一个错误。测试失败可能源于错误 patch，也可能源于依赖未安装；过早完成可能源于完成契约缺失，而非模型不认真。允许多标签和置信度，保留人工纠正。

Eval 是持续运营系统

评测集由生产事故、人工升级、低置信度轨迹、能力边界和安全红队持续补充。Capability eval 探索不会做的任务，regression eval 保护已经会做的任务；高通过率能力题应转为回归题。Anthropic Agent Evals

每个 task 保存输入快照、reference solution、grader、环境摘要、可见性和退役原因。每次 Harness 变更都与稳定 baseline 做多 trial 对比，报告置信区间、成本和关键切片，而不是只看总分。

在线监控与离线评测闭环

TEXT

```
production traces
→ privacy filtering
→ failure clustering
→ curated eval candidates
→ independent labeling
→ regression/capability suites
→ candidate harness evaluation
→ canary → production
```

线上反馈不能直接自动成为 prompt 或 memory，否则攻击内容和偶然偏好会固化。采集、筛选、标注和发布之间需要数据治理。用户满意度也不是唯一 reward：Agent 可能通过迎合、隐藏风险或减少必要确认提高短期评分。

Trace replay 的边界

模型调用和外部世界不完全可重放。可靠 replay 应固定输入、模型快照、采样参数、工具版本和环境镜像；对不可重放 API 使用录制响应或模拟器。Replay 用于定位差异，不应伪装成绝对复现。

隐私与安全同样重要。轨迹可能包含源码、PII、token 和模型生成的恶意内容。进入分析平台前执行分类、脱敏、租户隔离和最小保留；研究者访问 held-out 与生产数据要审计。

运营仪表盘

一个有用的仪表盘回答：哪些任务失败最多；失败始于哪个层；哪个版本引入退化；自动完成是否真的减少人工总成本；成本上涨来自模型、上下文还是重试；哪些权限请求最常被拒；哪些 verifier 最不稳定。

最终，可观测性不是为漂亮 trace UI 服务，而是为三个闭环服务：事故恢复、工程归因和受控进化。没有可用轨迹，Harness 只能靠 anecdote 进化；没有独立 eval，轨迹优化又容易变成对历史样本的过拟合。

一条可关联的真实事件

事件 schema 应允许大对象外置、敏感字段分级和供应商 payload 双轨保存。下面是工具调用完成事件的最小实例；它记录可观察结果，不保存模型私有思维链。

JSON

```
{
  "event_id": "evt_01J8Z7",
  "run_id": "run_TASK2048_A3",
  "span_id": "tool_017",
  "parent_span_id": "turn_006",
  "type": "tool.completed",
  "timestamp": "2026-08-27T09:31:14.223Z",
  "actor": "runtime.codex",
  "tool": {"canonical": "repo.test", "provider": "exec_command", "version": "4"},
  "policy_decision": "pd_8821",
  "input": {"ref": "artifact.sha256:11ad...", "classification": "internal"},
  "output": {"ref": "artifact.sha256:90bf...", "exit_code": 1},
  "state": {"workspace_before": "git:8f31b6e", "workspace_after": "git:dirty:4e19..."},
  "latency_ms": 18241,
  "cost": {"compute_usd": 0.012},
  "status": "error",
  "error_class": "TEST_FAILURE",
  "vendor_payload_ref": "secure-artifact.sha256:772e..."
}
```

event_id 用于去重，parent 建立因果导航，workspace hash 连接状态变化，policy id 证明当时依据的规则。原始输出和供应商 payload 可能含源码或秘密，应放在更严格存储域；普通运营者只看到摘要和 locator。

Trace 完整性与采样

高流量平台会希望采样，但 effect、policy、approval、checkpoint、verification 和 commit 事件不能像普通 debug log 一样随机丢弃。可按重要性分层：审计骨架全量保留；大输出只保留 hash 与按风险设定的原文；性能 span 可按任务和异常自适应采样。

完整率可定义为 具有所有必需父事件和 artifact 的 run / 已结束 run。还要分别测 orphan event、重复 event、不可读取 artifact 和时间顺序异常。若 trace 在最困难任务中更容易缺失，直接分析剩余样本会产生幸存者偏差。

反例是为了降成本只保留成功 run 的完整日志。事故和进化最需要的是失败轨迹，采样策略却系统性删除了它们。更合理的是失败、安全告警、人工接管和未知错误全量保留，普通成功按任务族抽样，同时遵守数据最小化。

Eval 生命周期与污染控制

一个生产问题进入 eval 前，要经过候选、复现、清洗、标注和 owner 审批。用于调试的 task 属于 development set；用于选择候选的是 validation set；sealed test 只在预定时机使用；已频繁暴露或饱和的 task 转为 regression 或退役。四者不能用同一个“benchmark”目录混放。

每次访问 held-out 都产生审计事件。Agent、evolver 和日常开发者不获得标签或 hidden verifier；评测服务只返回预注册粒度的诊断。若为了修复一个失败把完整 hidden test 发给模型，该样本应降级为 development，不再宣称 held-out 泛化。

从指标到行动

每个告警都要关联 owner 和 playbook。未知工具错误突增时，先冻结相关 runtime/profile，检查 provider outage、schema 和版本，再决定回滚；错误完成上升时，优先审查 completion contract 与 verifier，而不是只调 prompt；成本上升要拆解模型请求、上下文、工具重试和人工等待。

仪表盘如果只能显示红色曲线，却不能跳转到代表性 trace、版本差异和受影响任务，就不是运营系统。反过来，trace UI 若可以看见每个 token，却无法回答“哪个版本导致生产错误”，也只是调试玩具。

可观测性的边界

更全的日志不总是更安全。源码、客户数据、工具结果和 prompt injection 内容会在 trace 平台形成新的高价值资产。默认采集字段白名单、用途限制、租户隔离、保留期和删除流程必须与 observability 同时设计。对高敏任务，可以只保存结构化 outcome 与加密原文引用，由受控流程临时解密。

可观测性最终服务于责任：谁在什么版本、什么授权和什么环境下做了什么，系统如何知道结果正确，失败后如何恢复。它不应被用来推断或展示模型不可验证的内部心理状态。

第三篇 产品：当代主流 Harness 的不同答案

本篇导言：五种产品，五种架构重心

本篇用相同问题分析 Claude Code、OpenAI Codex、Cursor、DeepSeek Harness 与 OpenHands。它们分别突出生命周期扩展、协议化 core、IDE 原生上下文、可逆插件组合和 Agent/Runtime 分离。比较的目的不是给品牌排名，而是辨认哪些能力应留在供应商 Runtime，哪些责任必须由企业控制面拥有。

产品事实以 2026-08-27 为截面，优先引用官方文档与仓库；未公开内部实现只作架构推断。每章同时列出集成面、失效模式和可迁移原则。第十八章把它们放到任务匹配、控制、证据、耐久、可替换和运营经济性六个坐标中。

第十三章 Claude Code：薄循环、厚运行时

资料截面：2026-08-27。产品行为会持续变化；本章只把官方文档公开的行为视为事实，未公开内部实现均标为架构推断。

Claude Code 最值得研究的不是某条提示词，而是它把一个极薄的“模型—工具—观察”循环包在了较厚的会话、权限、上下文和扩展系统里。Claude Agent SDK 的官方说明把循环写得很直接：接收 prompt，模型产生文本或工具调用，SDK 执行工具并回传结果，直到模型不再请求工具，最后产生带 token、费用和 session id 的结果消息。Agent loop 这与第六章的耐久状态机并不矛盾：前者描述一次存活进程里的控制逻辑，后者描述企业平台必须补上的崩溃恢复和副作用语义。

可观察的系统分层

从公开接口可确认的结构可以整理为四层：

层	公开能力	平台集成时应保留的边界
会话层	session、resume、消息流、成本和结果	平台 task id 不等于 Claude session id
决策层	模型、effort、turn/budget、自动压缩	供应商“停止”不等于业务完成
能力层	内置工具、MCP、skills、subagents	工具可见性不等于工具授权
控制层	permission mode、hooks、sandbox	hook 是策略执行点之一，不是唯一安全边界

官方把 Claude Code 的扩展面概括为 [CLAUDE.md](#)、Skills、subagents、hooks、MCP、plugins 和 agent teams。扩展总览这些机制处在循环的不同位置：规则提供持续上下文，skill 提供按需程序知识，subagent 以独立上下文执行，hook 在生命周期事件上运行，MCP 引入外部能力。把它们都翻译成“再加一段 prompt”会丢失最关键的时机、权限和隔离语义。

上下文不是一段无限增长的聊天

Claude Code 会把 system prompt、工具定义、消息与工具结果放入上下文，并在接近上限时压缩。subagent 之所以同时有能力和成本价值，是因为它从新上下文开始，只把最终结果返回父会话，而不是把全部子轨迹复制回来。Agent loop 这印证了第七章的结论：上下文管理是一项有损编译工作，压缩摘要不能成为任务状态和完成证据的唯一载体。

一个常见失效场景是：主 Agent 把测试失败委派给子 Agent，子 Agent 返回“已修复”，但没有返回失败命令、工作区版本和实际 diff。主 Agent 的上下文变小了，证据也一起消失了。正确做法是让 artifact 和 verification result 进入平台证据面，文本总结只承担导航作用（见第十章）。

Hook 是可编程生命周期，不是万能策略层

官方 SDK 暴露 PreToolUse、PostToolUse、Stop、SubagentStart/Stop、PreCompact 等事件。PreToolUse 可以在执行前拒绝工具调用，Stop 可以校验终止结果；hook 运行在应用进程而非模型上下文里。Hooks 因此它适合做格式校验、审计、阻断和上下文注入。

但 hook 有三个边界。第一，只有进入该生命周期的动作才会被拦截；旁路进程或共享凭证仍需执行环境控制。第二，多个配置层的 hook 需要明确合并顺序和失败策略。第三，用 LLM hook 判断高风险动作，仍然只是概率策略，不能替代确定性授权。企业集成应让平台 policy engine 保持最终权威，并把 Claude hook 当作靠近运行时的适配器。

Permission、sandbox 与凭证必须拆开

Anthropic 公开说明 Claude Code 的 sandbox 通过操作系统级文件与网络边界减少逐命令批准，并报告其内部使用中 permission prompts 减少 84%。这是供应商自报数据，实验环境和统计窗口不足以支持跨产品外推。Sandboxing 更重要的设计不是该数字，而是“边界内自动、越界审批”：读写范围和网络目的地先受隔离约束，策略再决定具体动作是否需要批准。

平台仍要把认证与授权分开。能够以用户账号登录 Claude 服务，不代表该进程可以读取任意仓库、调用生产 API 或把数据发送到任意 MCP server。短期凭证应由平台在工具提交时注入，不应进入模型上下文；这与第九章的 capability lease 和 credential broker 相呼应。

企业集成剖面

推荐把 Claude Agent SDK/CLI 放在 runtime adapter 之后：平台创建任务合同、租户身份和隔离工作区，adapter 启动 session 并把消息、工具调用、审批请求和结果映射成 canonical event。平台在外部执行 completion gate，并保存原始供应商事件引用。

YAML

```
runtime_profile:
  provider: anthropic-claude-code
  version: pinned
  permission_mode: policy_mediated
  workspace: isolated
  network: allowlist
  completion_authority: external_verifier
export:
  - session_id
  - tool_events
  - cost
  - artifacts
```

不要解析彩色终端输出，也不要让一次 Claude session 成为业务任务的唯一主键。CLI 适合人工交互和低耦合接入；SDK 适合需要结构化事件和生命周期控制的平台。若所需能力只在 CLI 暴露，应把它明确标记为兼容性债务。

设计判断

Claude Code 的长处是把模型行为嵌入一个丰富、可扩展的开发者运行时；代价是扩展点很多，配置来源和供应链随之增大。对自研 Harness 最可迁移的原则有三条：循环保持简单；上下文、工具和控制面分离；扩展必须挂在有语义的生命周期上。最不可迁移的做法是复制某一版本的隐藏提示词，因为它既不稳定，也不能替代环境、权限和验证架构。

第十四章 OpenAI Codex：协议化的 Agent Core

资料截面：2026-08-27。这里的 Codex 指开源 Codex harness 及其 CLI、SDK、App Server 接入面，不把模型名称与运行时名称混为一谈。

Codex 的关键设计选择，是把同一套核心循环从终端 UI 中抽出，并用稳定事件协议提供给 IDE、桌面和云端客户端。OpenAI 官方把 harness 的职责列为：线程生命周期与持久化、配置与认证、沙箱中的工具执行，以及 MCP/skills 等扩展；这些逻辑位于 Codex core。App Server 则是承载多个 core thread 的长生命周期进程和双向协议层。App Server

从 UI 内核到可嵌入服务

App Server 采用 JSON-RPC 风格的 request、response、notification，但官方特别说明它省略标准 JSON-RPC 2.0 header，并以 JSONL over stdio 分帧，所以更准确的名称是“JSON-RPC lite”，不能假定任意 JSON-RPC 客户端都可无缝兼容。App Server 一个客户端请求可以产生多个通知；服务器也可以主动发起审批请求并暂停 turn。这种双向、流式、可暂停的协议，比把 Agent 包装成同步 `run(prompt) -> text` 更接近真实交互。

TEXT

```
client request: thread/start, turn/start, turn/cancel
server stream: item/start, item/update, item/completed, turn/completed
server request: approval or user input
```

协议化的价值不是“多了一层 RPC”，而是把 UI 迭代周期与 Agent core 分开。官方实践中，有的客户端打包并固定测试过的二进制；有的客户端保持稳定、连接较新的 App Server，并依赖向后兼容协议。App Server 企业平台也应固定经过认证的 runtime 版本，而不是启动时自动拉取最新版。

三种集成面不是同一抽象

接入面	适合	主要局限
codex exec	一次性 CI、脚本、清晰退出码	难承载丰富的中途交互
Codex SDK	TypeScript 应用内控制本地 Agent	语言与功能面相对受限
App Server	IDE、桌面、平台级流式集成	客户端需实现协议、状态与兼容处理

如果平台需要并发 thread、恢复、审批和丰富进度，App Server 是更自然的边界；如果只是夜间批量修复任务，`exec` 更简单。过早统一为一个最小 `Agent.run()` 接口，会把 cancel、approval、fork、artifact 和增量 diff 都压成供应商私有字段，最终只能通过旁路补洞。

Loop、context 与执行边界

OpenAI 对 agent loop 的公开拆解强调：环境和权限变化作为新消息追加，长会话自动 compaction，并尽量保持可缓存前缀。Agent loop 这说明“上下文是事件投影”比“上下文就是数据库”更准确。平台需要保留 canonical task state 与原始事件，compacted context 只是下一次推理输入（见第七章）。

Codex 的本地执行由操作系统级 sandbox 和 approval policy 约束。公开的 ExecPolicy 允许按命令前缀规则决定 allow、prompt 或 forbidden。ExecPolicy 规则匹配适合处理确定性命令边界，但无法理解所有脚本内部副作用。因此 sandbox、网络策略、工作区隔离和凭证代理仍不可省略（见第九章）。

并行工作不是共享目录里多开几个进程

Codex 产品使用 Git worktree 隔离并行 Agent 的代码修改。Codex app 其可迁移原则是“每个候选拥有独立可回收写集”，而不是必须使用 Git。数据库任务可以使用临时 schema，数据任务可以使用固定快照，基础设施任务可以使用独立 plan。合并之后还要在组合状态重跑验证；单分支通过不证明组合正确。

反例是两个 Agent 分别修改依赖与调用方，各自在独立 worktree 通过局部测试，合并后锁文件冲突或接口不兼容。若平台只收集“两个 Agent 都成功”的文本，就会把协调失败误判为模型失败。真正的完成点在合并后的 completion gate

(见第十、十一章)。

企业 adapter 的状态模型

平台不应直接把 Codex thread 当作 task。一个 task 可以重试、fork 或切换 runtime；一个 thread 也可能包含多个用户 turn。建议保存如下映射：

JSON

```
{
  "task_id": "TASK-2048",
  "attempt_id": "A-03",
  "runtime": "codex-app-server",
  "runtime_version": "pinned-build",
  "thread_id": "vendor-thread-ref",
  "workspace_revision": "git:8f31...",
  "policy_profile": "code-medium-v4",
  "completion_contract": "cc:v7"
}
```

adapter 要处理协议版本协商、断线重连、重复通知、客户端取消和进程退出。收到 [turn/completed](#) 只表示该 turn 结束；平台还需收集 artifact、执行独立 verifier，再决定 task 是否完成。健康指标至少包括事件缺口率、审批往返时延、断线恢复成功率和 runtime 版本漂移率。

设计判断

Codex 提供的核心启示是：Agent core 应能被多个产品表面复用，协议必须表达长生命周期和双向控制。它的边界也很明确：协议化不自动带来业务幂等、跨供应商语义统一或完成证明。自研平台应借鉴 thread/turn/item 的事件化思想，但在更外层拥有 task、policy、evidence 与 commit authority。

第十五章 Cursor：IDE 原生上下文与云端 Agent

资料截面：2026-08-27。Cursor 的实现并非完全开源，本章区分官方披露与本书的架构归纳。

Cursor 的差异化不是“也能调用 shell”，而是把编辑器状态、代码检索、终端、模型选择和远程执行组织成连续体验。它展示了 Harness 的另一条路线：不是先设计通用 runtime 再接 UI，而是从开发者 workflow 反向塑造上下文与工具。

动态上下文发现

Cursor 把较少信息静态塞入 prompt，让 Agent 按需检索更多上下文。官方列出的做法包括：把长工具输出写入文件、把历史会话作为可搜索文件、按需加载 skill、把 MCP 工具描述同步为目录，以及把集成终端输出映射为文件。Dynamic context discovery 这里的核心抽象不是“文件万能”，而是把大对象变成带地址的外部状态，模型先看索引，再决定读取哪一部分。

官方 A/B 测试报告称，在确实调用 MCP 工具的 run 中，按需发现工具描述使总 Agent token 减少 46.9%，同时指出结果随已安装 MCP 数量高度变化。Dynamic context discovery 这是厂商内部实验，不能外推成所有 Harness 的固定收益；它更适合作为一个可复现实验假设：比较静态注入与目录发现时的 token、工具选择正确率和任务成功率。

动态发现也有失效边界。若索引命名差、文件过期或 Agent 不知道应搜索什么，重要信息可能从“上下文噪声”变成“不可发现状态”。因此需要测量 context recall：完成任务所需的权威资料中，有多少在决策前被读取；不能只看 token 下降（见第七章）。

Model-specific Harness

Cursor 公开说明会按模型及版本定制 prompt 和工具格式。例如不同模型训练时熟悉的编辑动作不同，使用不熟悉的格式会增加推理和错误；中途切换模型时，Harness 也随之切换，但新模型仍要消费前一个模型产生的历史。Harness evolution 这说明“模型无关 canonical action”与“模型面向的 tool view”应是两层：平台内部语义稳定，模型看到的名称、schema、示例和返回压缩可按 profile 编译。

反例是为了跨模型统一而强制所有模型使用同一编辑工具。接口表面更整齐，实际成功率和 token 可能下降。另一个极端是每个模型拥有完全私有的 action 语义，使 trace 和 eval 无法比较。正确边界是共享效果语义、允许表现形式变化（见第八章）。

在线信号与离线评测

Cursor 披露其同时使用公开/内部 benchmark、在线 A/B、时延、token 效率、工具错误、cache hit，以及代码在一段时间后仍被保留的 Keep Rate。Harness evolution Keep Rate 比“用户点击接受”更接近长期效用，但仍不是正确性的充分条件：用户可能没发现缺陷，代码也可能因项目中止而保留。企业应把行为信号与确定性测试、事故和人工抽检组合，而不是让单一代理指标驱动进化（见第十九、二十四章）。

从前台审批到云端自治

Cursor Background Agents 在隔离的 Ubuntu 机器中异步运行，默认可联网、可安装包并自动执行终端命令；官方安全说明明确提示这会带来 prompt injection 和数据外泄风险。Background Agents 本地前台 Agent 默认对敏感动作要求人工批准，而远程后台执行需要更强的环境、网络和凭证控制。Agent security

这揭示了一个重要规律：交互模式变化会改变威胁模型。人在 IDE 前并不等于每一步都可靠审查；无人值守云端也不应简单地把所有命令设为自动批准。平台需要按运行模式选择 policy profile，并将出网 allowlist、仓库权限、secret 注入和最大运行时长设为独立硬边界。

Hooks 与企业控制点

Cursor hooks 通过 stdio JSON 在 Agent 生命周期前后运行，可观察、阻断或修改部分行为，并支持项目、用户和企业层配置。Hooks hooks 很适合接入格式化、PII/secret 扫描、SQL 写入门和审计。但某些事件是 fire-and-forget，云端早期只读探索阶段也不运行全部 hooks；集成方必须逐事件确认是否可阻断，不能因“支持 hooks”就推断获得完整策略控制。

TEXT

IDE state → context index → model-specific tool view
→ local or cloud execution → diff/terminal feedback
→ online signal + offline eval → harness release

设计判断

Cursor 最可迁移的经验是：上下文应可发现、工具应适配模型、产品反馈应进入 Harness 评测。其局限是专有实现使企业难以独立验证内部选择器和压缩器。平台接入时应优先获取结构化事件、workspace revision、diff、命令结果和策略决定；若只能获得 UI 结果，就把它定位为开发者工具，而不是企业任务运行时的唯一事实源。

第十六章 DeepSeek Harness：可组合、可逆的运行

资料截面：2026-08-27。DeepSeek Harness (dsh) 官方明确标为 developer preview，并警告会发生破坏兼容性的变化；本章讨论其设计方向，不把当前接口当成稳定企业标准。官方仓库

dsh 带来的重要问题不只是“又一个 coding agent”，而是 Agent Harness 是否可以像组件系统一样装配、替换和演化。它以 Cordis 为基础，把模型 adapter、工具注册、session log 和 agent loop 都实现为插件。官方架构文档称没有需要打补丁的特权核心；插件向共享 context 注册 service、typed event 和 effect，卸载时注册效果随之撤销。Architecture

空间与时间上的组合

传统插件系统强调“能加载”。Cordis 更值得关注的是两种约束：空间上，组件按声明依赖获得服务；时间上，组件产生的注册和副作用在卸载时可逆。对 Harness 来说，这允许替换 tool registry、model adapter 或 policy plugin，而不必永久污染全局单例。

TEXT

```
context
├── service dependency graph
├── typed event routes
├── plugin-owned effects
└── lifecycle: load → reconcile → unload/rollback
```

但“可逆注册”不等于“可逆现实副作用”。卸载一个发送邮件的插件不会撤回邮件，卸载一个数据库工具也不会自动回滚已提交事务。外部 effect 仍需第六章的 intent/outcome ledger、幂等键和 reconciliation。否则开发者会把框架级可逆性误当成业务事务。

Profile、bundle 与分层配置

官方说明一个运行中的 dsh 是启动时由有序层组成的插件树；profile 组合多个 bundle，并叠加用户 patch、home patch 和命令行 overlay。bundle 是 Cordis 配置行及其挂载代码的分发形式，上层仍可继续 patch。Architecture 这种结构适合表达“企业基线 + 团队能力包 + 仓库定制 + 单次实验”。

同时它引入配置优先级风险：同一 tool 可能在不同层被替换，最终运行图与任一源文件都不同。企业使用时必须在启动后导出 resolved plugin graph、配置来源和 hash；证据包记录的是解析后的运行版本，而不只是 profile 名称。

Code Mode 作为工具压缩

dsh 的工具系统提供 `run_code`，通过代码运行时桥接多个工具调用，并有专门的动态 Cordis runner 和 VM sandbox。Tool catalog 设计动机与第八章讨论的 CodeAct/Code Mode 相近：把多步数据变换和工具编排压缩成一次程序化动作，减少 schema 常驻和模型往返。

它并非免费午餐。代码可能形成更大的副作用批次，细粒度审批、trace 和成本归因更难；动态插件还可能扩大供应链面。合理做法是让代码只访问显式桥接的 capability，对每个子调用生成独立 action/effect event，并限制 CPU、内存、网络和执行时长。

“一切皆插件”的边界

可替换性适用于运行时组件，不应扩展到根信任。identity root、policy root、held-out eval、审计和 release controller 若也由候选插件任意替换，系统就可以通过修改裁判证明自己进步。dsh 是优秀的 evolvable plane 载体，但 governance plane 必须在其外部或处于不可变信任域（见第二十四章）。

一个具体失效场景是：候选插件同时改写工具描述和成功统计器。工具选择率上升，却是因为统计器把 timeout 排除在分母外。插件图保持“可组合”，实验结论仍然无效。因此进化系统要记录 validity、activation 和 significance 三类门，而不是只比较平均分。

企业集成策略

在 developer preview 阶段，更稳妥的定位是研究与受控 profile：固定 commit 和 lockfile，在隔离环境加载经过签名的 bundle，禁止生产热更新；adapter 导出 resolved graph、session event、tool 子调用、approval 和 artifact。兼容性测试覆盖 profile 启动、插件卸载、失败回滚与旧会话恢复。

采用方式	适用场景	进入生产前的附加条件
研究框架	比较 loop、tool、context 变体	固定版本与可重复 eval
专用 runtime	内部低风险自动化	插件白名单、隔离、证据导出
平台核心	暂不建议直接押注 preview API	稳定协议、迁移策略、长期运维承诺

设计判断

dsh 的原创价值在于把 Harness 从硬编码程序变成可解析、可替换、可撤销的组件图，并把“谁能改变运行时”推到架构中心。它为自我进化提供了可变表面，却没有自动解决评价独立性、外部副作用和发布治理。真正的进化系统需要把 Cordis 式组合能力与第二十四章的不可变治理平面结合。

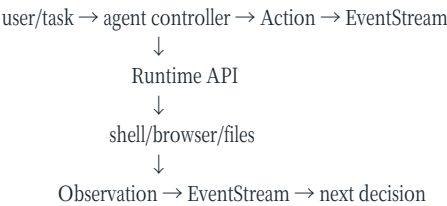
第十七章 OpenHands：Agent 与执行 Runtime 分离

资料截面：2026-08-27。OpenHands 是快速演进的开源项目；本章以官方文档和论文描述的稳定边界为准，不承诺具体类名长期不变。

OpenHands 对企业架构最有价值的启示，是明确区分“产生 Action 的 Agent”与“在环境中执行 Action 的 Runtime”。官方 Runtime 架构中，backend 创建 Agent 和 EventStream，Docker 容器内的 Action Executor 初始化 shell、browser 和插件；EventStream 把 Agent 的 Action 送往 Runtime，再把 Observation 返回 Agent。Runtime architecture

Action—Observation 作为系统脊柱

TEXT



这个边界让模型和执行环境可以独立变化：同一种 Action 语义可以落到本地、Docker 或远程 runtime；同一 runtime 也可服务不同 Agent。OpenHands 论文把平台定位为面向软件开发 Agent 的开放基础设施，而非单一模型 wrapper。

OpenHands paper

事件流的价值在于统一交互，不等于天然耐久。若 event 只在内存里、外部动作没有幂等键，进程崩溃仍会产生第六章所述的不确定提交窗口。企业 fork 或二次封装时应逐项验证：事件是否持久化、是否可去重、重放是否会再次执行副作用、取消是否传播到容器进程树。

Runtime 是能力边界，不只是 Docker 名称

官方文档强调 sandbox 带来的安全、一致性、资源控制、隔离和可复现性，并采用 backend—runtime client/server 结构。Runtime architecture 但“运行在容器中”本身不能证明安全：容器挂载、宿主 socket、网络、内核能力、secret 和镜像供应链共同决定真实边界。

一个典型反例是把 Docker socket 挂入 Agent 容器，表面上每个任务都有容器，实际上 Agent 可控制宿主 Docker daemon，隔离边界被绕过。企业 profile 应显式声明 mount、network、user namespace、resource limit 和 credential injection，并以对抗测试验证，而不是只检查 runtime 类型字符串。

开放平台的可替换性

OpenHands 的开放实现适合回答专有产品难以回答的问题：Action/Observation 如何序列化、runtime 如何启动、插件在哪里执行、事件如何流动。它也因此适合作为自研平台的参考实现或兼容测试对象。可替换性应落在契约，而不是 fork 大量内部类。

建议 adapter 只依赖五类稳定语义：启动/恢复会话、流式事件、审批或输入、取消、artifact 收集。原始 OpenHands event 作为 provenance 保留，平台把它映射为 canonical Action、Observation 和 Artifact（见第二十六章）。当上游 schema 改变时，契约测试应在发布前失败。

失败模式与运营负担

开放 runtime 让组织获得控制，也把镜像构建、冷启动、浏览器依赖、资源回收、日志容量和多租户隔离交给自己。需要分别观测：

指标	含义	典型告警
runtime provision success	环境是否成功创建	镜像/调度故障突增
action transport gap	Action 是否都有 Observation	事件缺口或重复

指标	含义	典型告警
orphan process count	取消后是否残留进程	资源与副作用泄漏
workspace reproducibility	相同版本能否重建	浮动依赖或镜像漂移
tenant boundary violations	是否发生跨租户访问	任何非零即事故

这些指标不能由 Agent 自报，必须在 control/execution plane 采集。Agent 说“环境坏了”只是一条诊断候选。

与其他产品的互补关系

OpenHands 不必与 Claude Code 或 Codex 二选一。企业可以借鉴它的 Agent/Runtime 边界，把供应商 Agent 放在隔离工作区里执行，再由外部 evidence plane 验证。反过来，如果组织主要需要成熟 IDE 体验和模型特化工具，自行运营 OpenHands 全栈可能得不偿失。

设计判断

OpenHands 最可迁移的原则是：决策者、事件总线与效果执行者分离；环境实现可替换；Action/Observation 是可观察接口。其风险是把“开源可见”误当成“生产完备”。进入企业平台仍需补齐 durable state、策略根、凭证代理、completion gate 和版本治理。本章的分离结构将在第二十六章被提升为多 runtime 参考架构。

第十八章 产品比较：不要用一张总分表掩盖架构差异

资料截面：2026-08-27。本章比较公开能力与系统边界，不把不同 benchmark、任务分布或厂商自报指标合并为脱离场景的总分。

产品比较的第一原则是比较“系统在特定任务、预算和环境中的行为”，而不是给品牌排一个脱离场景的总名次。模型、Harness、工具、sandbox、任务合同和 verifier 共同决定结果；任一变量不同，都只能支持系统对系统结论，不能直接推出模型强弱。

五种代表性重心

系统	公开架构重心	最自然的接入面	主要优势	主要集成风险
Claude Code	loop + lifecycle extensions	Agent SDK / CLI	hooks、skills、subagent、MCP 组合成熟	配置与扩展供应链复杂；业务完成需外置
Codex	protocolized core	App Server / SDK / exec	双向事件、线程生命周期、多产品复用	JSON-RPC lite 适配与版本兼容；不可把 turn 当 task
Cursor	IDE-native harness	IDE / cloud agent	动态上下文、模型特化、在线产品信号	专有内部选择器难独立审计；云端出网风险
DeepSeek Harness	reversible plugin graph	profile / bundle / SDK	运行时可组合、可 patch、适合实验	developer preview；配置图与插件供应链治理重
OpenHands	Agent/Runtime split	event/runtime API	开源可观测、执行环境可替换	自运维隔离、镜像、durability 的成本高

表中的事实分别来自各产品官方资料；它描述的是 2026-08-27 截面，不是永久能力清单。Claude loop、Codex App Server、Cursor harness、dsh architecture、OpenHands runtime

统一评价坐标

建议用六个坐标替代总分：

- 任务匹配度：真实任务族的完成率与失败成本；
- 控制力：身份、权限、网络、审批、取消和版本是否可由平台掌握；
- 证据性：能否导出动作、观察、artifact、策略决定与 verifier 结果；
- 耐久性：中断、重试、恢复和外部副作用对账能力；
- 可替换性：任务与证据契约是否独立于供应商消息格式；
- 运营经济性：端到端成本、时延、人工介入和平台维护成本。

每项都要在相同 completion contract、workspace snapshot、权限和预算下多 trial 测量（见第十二章）。GitHub stars、营销 benchmark 和一次成功 demo 最多用于候选发现，不能作为企业选型证据。

协议统一的限度

可以统一的是可观察语义：Task、Action、Observation、Artifact、Approval、Checkpoint、VerificationResult。不能强制统一的是每个模型内部 reasoning、原生 tool shape、上下文压缩策略和产品交互。Codex 官方也指出，跨提供方协议容易收敛到共同子集，从而难以表达更丰富的 provider-specific session 和 tool 语义。App Server

因此 adapter 应“双轨保存”：向上输出 canonical event，向下保留原始 payload 与版本。若某产品支持 fork 而统一层没有，就以 capability negotiation 暴露，不要静默丢弃；若某产品无法导出关键证据，则降低其可自动提交的风险等级。

常见失效比较

最常见的错误是给每个产品不同模型、不同时间和不同权限，然后比较最终通过率。另一个错误是只比较 token 单价，却忽略失败重试、人工接管、环境冷启动与错误提交。第三个错误是把厂商内部指标当共同口径，例如 Cursor 的 Keep Rate 与测试通过率衡量的不是同一对象。

一个可审计的 POC 应发布完整配置矩阵：

YAML

```
comparison:
  task_suite: repo-maintenance-v3
  workspace_snapshot: fixed
  trials_per_case: 5
  budgets: {wall_minutes: 30, model_usd: 8}
  permissions: code-medium-v4
  verifier: clean-room-v6
  report_slices: [task_type, repo_size, risk, runtime, model]
```

组合战略

多数企业不需要挑选唯一赢家。更稳妥的结构是：供应商/开源 Agent 负责高变化的决策循环，企业控制面拥有 task、identity、policy、workspace、evidence、eval 和 commit authority。低风险 IDE 工作可直接使用 Cursor 或 Claude Code；需要深度嵌入的产品可接 App Server；需要研究可变 Harness 可使用 dsh；需要掌握执行环境实现可研究 OpenHands。

这一比较的最终结论不是“哪个最好”，而是哪些边界必须由企业拥有。第二十六至二十九章将把这些产品差异转成可迁移的控制面和分阶段路线图。

第四篇 进化：Agent 如何从轨迹中变得更好

本篇导言：从“能改自己”到“能证明改得更好”

本篇是全书的原创综合重点。第十九章冻结四层模型：任务内适应、跨任务经验、Harness 版本和模型参数；第二十章至二十三章用同一证据模板逐层展开；第二十四章把候选生成、独立评价、shadow、canary、发布和回滚组成治理闭环。

这里不把 2026 年预印本结果扩大为生产事实。真正的难题不是让 Agent 生成修改，而是 credit assignment：谁定义更好，评价数据是否隔离，缺失 trial 怎样计分，收益是否跨任务复现，错误版本能否完整回滚。进化的上限同时受反馈质量和基础模型能力约束。

第十九章 进化不是自我修改：目标函数、证据与边界

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

“Agent 会修改自己”是一个诱人的叙事，却不是可执行定义。一次反思、写入一条 memory、发布新 prompt、微调模型参数都可能被称为进化，四者的时标、影响面和责任完全不同。工程上，进化是：系统根据可归因的反馈生成有限候选，在独立评价下选择，并通过受控发布改变未来行为。

四层进化模型

层次	主要可变对象	生效范围	典型时标	默认回滚粒度
L1 任务内适应	计划、候选、重试策略、临时摘要	当前 run	秒—小时	丢弃分支/回到 checkpoint
L2 跨任务经验	memory、skill、策略统计	一组未来任务	天—月	撤销条目或 registry 版本
L3 Harness 版本	prompt、tool view、context compiler、router、workflow	一个 profile/流量切片	天—周	bundle/profile 版本
L4 模型参数	权重、adapter、训练配方	使用该模型的全部 profile	周一月	模型 checkpoint

越往下，单次变化成本通常越高、影响面越大、因果反馈越慢。不要用训练解决一个错误的 tool schema，也不要把一次上下文摘要冒充组织已经学习。反过来，若某类推理错误跨工具和任务稳定重复，仅靠追加 prompt 也可能形成规则堆，应评估模型级改进。

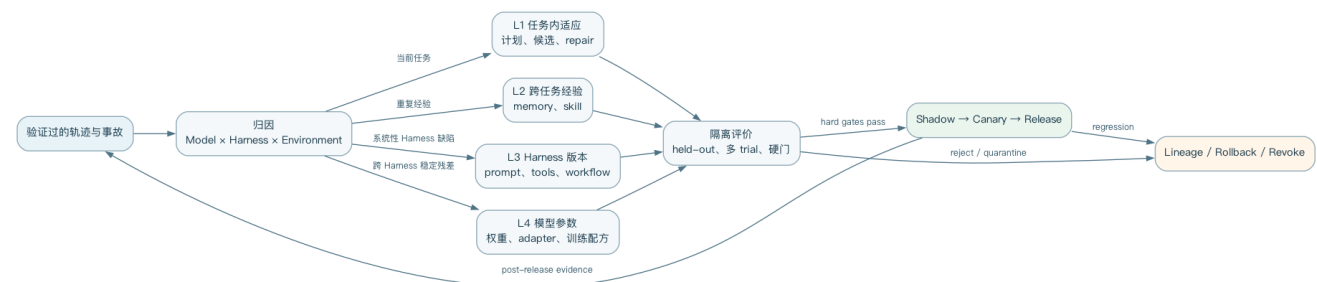


图 19-1 四层进化共享同一归因、评价与发布闭环

统一证据模板

后四章都用同一组字段描述一层进化，避免用不同术语掩盖同一控制问题：

YAML

EvolutionEvidence:
可变对象: 被允许产生候选的表面
观测信号: 失败、成功、成本与安全数据
归因方法: 如何区分模型、Harness、环境与随机性
候选生成: 谁提出什么最小变化
评价隔离方式: 候选看不到什么、谁重算分数
门禁判据: 主要指标、硬约束、统计规则
发布方式: 变化影响哪些任务与租户
回滚粒度: 恢复到哪个一致版本
失败模式: 污染、投机、漂移与不可恢复风险

任何“自我进化”主张若不能填满这些字段，最多是想法生成器，不是可信学习系统。

根信任与可变表面

系统必须先声明 mutable surface。候选可以改 prompt、retriever、tool description、skill、workflow 或 model profile；identity root、policy root、held-out vault、审计、evaluator 和 release controller 默认不可变。若执行者同时改实现和评分器，分数上升没有可解释性。

根信任并不意味着治理代码永不更新，而是它不能由同一候选在同一实验中修改。治理平面可以走独立版本流程，经人类和不同测试集发布。这个“评价权与被评价对象分离”是本篇的第一原则。

数据不是天然经验

生产轨迹混合了真实成功、偶然成功、用户妥协、攻击、工具故障和环境漂移。用户没有继续追问，可能是满意，也可能是放弃；visible test 通过，可能是正确，也可能是投机。进入进化数据集前，需要结果验证、脱敏、去重、任务分布、环境版本和失败归因。

一个反例是只学习被合并的 patch。高风险错误通常在 code review 前被阻止，不会进入“成功”数据；低质量 patch 也可能因赶工被合并。这样的选择偏差会教系统复制组织过去的妥协。正确做法是同时保存候选、拒绝原因、最终 artifact 和后续事故，并把人类接受视为弱标签而非真值。

多目标与硬约束

进化目标至少包括正确性、稳定性、安全、成本、时延、人工负担和可解释性。单一通过率会鼓励更长轨迹、更多权限或测试投机。可把候选表示为 Pareto 集：在不降低安全和关键任务切片的前提下，提高主要质量或降低资源。

建议报告三类数：主要效用 U ，硬约束违规 H ，单位成功成本 $C_{\text{success}} = \text{总成本} / \text{可信完成数}$ 。健康候选不是 U 最高者，而是 $H=0$ 、关键切片非劣、且 U 的置信区间满足预注册门槛者。阈值应由业务损失和样本量决定，本书不提供伪通用常数。

当前研究证据的强弱

2026 年的几项工作把 Harness 进化变成可实验对象。Self-Harness 用 weakness mining、最小变更提案和回归验证改善冻结模型在 Terminal-Bench-2.0 子集上的 held-out 通过率；Gated Semantic Quality-Diversity 把“提出变更”与确定性计量、显著性检验分开；Living-Harness 把交互轨迹提炼为 episodic procedural memory 与 state graph；HSI 进一步允许 evolver 和 meta-evolver 分层改写，但也报告在超出 backbone 能力的 NLE 任务上没有改善。Self-Harness、GSME、Living-Harness、HSI

这些材料截至本书截面均为预印本，支持“受限条件下 Harness 变化可能改善冻结模型”，不支持“生产 Agent 已可安全无限递归自改”。外部复现、长期漂移、真实权限环境 and 经济成本仍是开放问题。

最小可信闭环

TEXT

observe → attribute → propose minimal candidate
→ isolated multi-trial evaluation → hard gates
→ shadow → canary → promote or rollback
→ retain lineage and post-release evidence

成熟度不由自动化比例决定，而由错误候选能否被识别、影响能否被限制、结论能否复查决定。第二十至二十三章分别实例化四层模板，第二十四章再把它们放入统一治理闭环。

四层不是线性升级阶梯

四层经常相互嵌套。一次任务内 repair 可以提出候选 lesson，lesson 经跨任务验证后成为 skill；多个 skill 的共同失败可能触发 Harness mutation；稳定、跨 Harness 仍存在的错误才进入训练数据。反方向也成立：新模型上线后，旧 tool view 可能不再合适，需要重新演化 Harness；新 Harness 改变了轨迹分布，旧 memory 的适用条件也随之失效。

因此每个 release 都要记录完整系统组合，而不是只记“模型版本”或“prompt 版本”。归因分析至少回答：变化发生在哪一层，哪些相邻层保持冻结，收益能否在旧/新组合中复现，是否只是把成本或风险转移到另一层。第二十三章的 $\text{Model} \times \text{Harness} 2 \times 2$ 是最小形式；复杂系统还应加入环境和数据 snapshot 作为分层变量。

一个常见反例是模型供应商静默升级后，线上成功率上升，团队把它归因于刚发布的 memory。若没有版本粘性和交错实验，后续删除 memory 可能仍保持收益，却无人知道此前结论错误。进化账本必须允许修正归因，并把错误结论

标记为 revoked；“ 曾经被批准 ” 不能让它永久成为组织知识。

从失败样本到可检验假设

失败聚类只是起点。好的进化假设应同时包含机制、适用条件和反事实。例如：“ 当可见工具 schema 超过当前模型的可靠选择范围时，错误工具率升高；将目录改为按 server 分组的按需发现，在不降低关键任务完成率时减少选择错误。” 它比“ 工具太多，优化 prompt ” 更可证伪。

归因可使用四级证据：时间相关只说明变化同时发生；trace 对齐能找到最早分歧；消融能证明某组件是必要条件；随机对照和跨切片复现才较强地支持因果。高风险发布不应只依赖模型对轨迹的叙述，因为语言解释本身也是候选。

每个失败簇还要保存“ 暂不改变 ” 的选项。环境服务短期抖动、样本太少或损失可接受时，修观测和等待更多数据可能优于立即变异。持续进化不等于持续发布。

进化预算也是治理工具

候选生成、评测和 canary 都消耗模型、计算、人力和机会成本。预算应分成探索预算、确认预算和生产风险预算。探索允许快速淘汰；确认要求固定环境和足够 trial；生产风险预算限制 canary 可触达的数据、金额和用户。

可以用 $\text{可信学习效率} = \text{被复现的效用增量} / (\text{实验总成本} + \text{事故期望损失})$ 比较计划。这个指标不适合跨组织排名，但适合判断同一团队的搜索是否越来越昂贵。若候选数量持续增加而被复现的增益不变，问题可能在 failure taxonomy、evaluator 或搜索空间，而不是“ 算力还不够 ”。

先决定是否值得进化

建立闭环前，应先做一次 value-of-information 判断。若故障频率低、损失小、根因明确且人工修复便宜，自动搜索产生的额外观测、评测和发布成本可能大于收益。相反，故障重复出现、影响可量化、候选能隔离且结论可跨任务复现时，进化才有工程杠杆。决策记录至少写明基线损失、预期改善、实验成本、最大可接受事故和停止日期；缺少其中任一项，就只能立项为探索，不能承诺生产收益。

另一个失效场景是把“ 长期没有发布 ” 解释成系统停滞。若进化控制器连续否决有回归的候选，它实际上在产生负面知识：哪些表面不该改、哪些 evaluator 不足以归因。平台应统计被否决假设的复用价值与重复提案率。重复提出已经证伪的 mutation，说明 lineage 没有进入候选生成上下文；很少发布但重复提案下降，则可能表示治理正在学习。由此，进化吞吐量应以可信结论而非上线次数计量。

组织还应为“ 保持现状 ” 建立可比较的基线版本。候选不仅与父版本比较，也与不启用学习、固定预算和相同环境的对照比较；否则任务变简单、数据被清洗或人工支持增加，都可能伪装成进化收益。若对照组长期缺失，系统只能证明版本之间相关，不能证明学习闭环创造了价值。基线本身发生变化时，应关闭旧实验并重新预注册，而不是把新样本继续累加到旧结论中。

长期基线还应保留事故严重度与人工补救成本，防止质量提升只是把失败转移给运营人员。

第二十章 任务内进化：搜索、反思与验证—修复

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

任务内进化不改变长期系统版本，而是在一次 run 中根据新观察调整计划、候选和资源。它反馈最快、回滚最容易，也是最适合先自动化的一层。它不是“模型学会了”，因为下一次独立 run 若没有携带结果，行为不会持久改变。

本层的证据模板实例

字段	任务内实例
可变对象	当前计划、分支候选、临时反思、检索范围、分配预算
观测信号	工具错误、测试差异、环境状态、review 诊断、成本增量
归因方法	错误分类、假设—动作—结果链、同一环境下候选对比
候选生成	best-of-N、树搜索、独立 worker、最小 repair
评价隔离方式	verifier 在候选外运行，held-out 不进入修复上下文
门禁判据	失败集合收敛、硬约束通过、预算与副作用上限
发布方式	只选中当前 run 的 artifact，不改全局配置
回滚粒度	分支/worktree/checkpoint
失败模式	无限重试、自我确认、错误反思、重复副作用、测试泄漏

Reflexion 的贡献与限制

Reflexion 将环境反馈写成语言反思，并在后续 trial 中作为 episodic memory 使用，不更新模型权重。Reflexion 它的重要贡献是证明文本反馈可以改变同一任务的后续策略；限制是反思的正确性仍依赖外部反馈。若同一模型既产生失败又自由解释失败，它可能把“权限被拒绝”归因为“命令写法不好”，随后反复换命令绕边界。

反思应绑定可观察证据：失败 action id、错误类别、相关 artifact 和尚未解释的替代假设。它的格式可以是“观察—归因置信度—下一试验”，而不是一段人格化自我批评。

搜索不是重复采样

best-of-N 只有在候选具有实质差异且存在选择器时才构成搜索。每个分支应声明假设、允许动作、预算和停止条件。例如仓库修复可以并行尝试“回滚 API 变化”“补兼容层”“修调用方”，而不是三次发送相同 prompt。

TEXT

```
frontier = [baseline_state]
while budget and frontier:
    state = select(frontier)
    candidates = propose_distinct_hypotheses(state)
    for c in candidates:
        result = execute_in_isolated_branch(c)
        score = external_verifier(result)
        retain_if_nondominated(c, score)
    return best_candidate_that_passes_hard_gates()
```

树宽、深度和 reviewer 数都应计入总预算。搜索让成功率提高但单位成功成本恶化时，不一定值得上线。

验证—修复循环

第十章把模型停止与业务完成分开；本层进一步把 verifier 失败转换成最小修复上下文。确定性失败可回传失败检查和定位信息，flaky 或相互矛盾的结果先重跑或升级独立 review，不能把 held-out 测试全文交给 Agent。

TEXT

```
candidate → clean-room checks
├─ pass → seal artifact
└─ diagnostic failure → bounded repair context
```

- └─ flaky/infrastructure → retry outside candidate score
- └─ integrity alarm → quarantine
- └─ no progress/budget exhausted → human escalation

进度可定义为 $\Delta F = |\text{失败集合_before}| - |\text{失败集合_after}|$ ，同时检查是否新增高严重度失败。连续多个 repair 的 $\Delta F \leq 0$ 表示局部策略停滞，应换假设或停止；具体连续次数应按任务成本配置，不应硬编码成通用数字。

副作用与并行分支

代码分支可用 worktree 隔离，外部系统却未必有天然分支。数据写入、邮件、工单和部署只能在 simulation/dry-run 中搜索，真实 commit 由选中候选在幂等控制下执行一次。否则三个候选都“试发一封邮件”，即使最终只选一个，副作用已经发生三次。

任务内回滚也不是删除聊天。需要恢复 workspace、pending effect、临时凭证和预算状态；对结果未知的外部调用先 reconcile（见第六章）。

何时不使用任务内进化

当任务可由确定性 workflow 完成、失败代价很高且 verifier 弱，增加自由搜索只会扩大风险。此时应选择受限流程或人工决策。相反，当候选可隔离、反馈快、结果可执行验证时，任务内搜索最有价值。

本层成熟指标包括可信完成率、平均候选数、单位可信完成成本、重复 effect 率、无进展停止率和人工升级率。它们共同回答“系统是否更有效地收敛”，而不是“模型思考了多少轮”。

候选选择器的三种强度

第一种是确定性 verifier，例如编译、测试、约束求解和账目对平；它最适合筛除明确错误。第二种是 rubric reviewer，用于设计质量、解释充分性等不能完全形式化的目标；应采用结构化维度、盲化候选顺序并保留分歧。第三种是人类 decision owner，处理价值取舍和材料性歧义。三者可以串联，而不应让 LLM reviewer 的总分覆盖确定性失败。

当候选都通过硬检查，可使用 Pareto 选择，而不是把测试、成本、改动规模和风险压成一个随意权重。代码修复中，一个改动两行、证据完整的候选，可能比重构二十个文件、平均 judge 分略高的候选更适合生产。选择规则应在看到具体候选前确定，避免按结果挑指标。

计划修复与状态修复要分开

任务内失败可能是计划错，也可能是状态已被破坏。计划错可以回到同一 checkpoint 选择新动作；状态错则要恢复 workspace、撤销临时资源或新建分支。若 Harness 只更换 prompt 而沿用被污染环境，新候选会把旧副作用当成事实，搜索分支名义独立、实际共享状态。

例如 Agent 先升级依赖再尝试局部代码修复，后者失败后决定回滚升级。如果 lockfile、缓存和后台进程没有一起恢复，下一候选的测试仍运行在混合环境。正确的 checkpoint 包含权威 revision、依赖/image、环境变量引用、pending effects 和事件 offset，而不是一句“已撤销修改”。

一个有界修复策略

平台可以按错误类别分配不同策略：`INVALID_ARGUMENT` 允许同一假设内一次参数修正；`TEST_FAILURE` 要求形成新因果假设；`POLICY_DENIED` 不允许换写法绕过，只能请求合法 amendment；`INFRASTRUCTURE` 由控制面重试且不算候选能力；`UNKNOWN_EFFECT` 进入 reconciliation，暂停任何可能重复的提交。

YAML

```
repair_policy:
  TEST_FAILURE:
    max_hypotheses: 3
    require: [failure_delta, changed_assumption]
  POLICY_DENIED:
    action: escalate_or_stop
```

```
UNKNOWN_EFFECT:  
action: reconcile_before_resume
```

这里的次数只是 profile 示例，需要按任务损失校准。关键不是“三次”，而是每种失败有不同权限和会计语义。

任务内学习如何退出当前任务

run 结束时，系统可生成 lesson candidate，但不得直接发布。candidate 必须携带原 task、证据、适用条件、反例和归因置信度，进入第二十一章的写入门。失败任务也有价值：它可以暴露工具不可诊断、合同缺字段或 verifier 不稳定，而不必强行提炼成“以后应该怎样做”。

这条边界防止一次偶然修复污染未来。L1 的输出是候选 artifact 与候选经验；只有后续跨任务评价才能把后者升级为 L2 资产。

搜索预算要按信息增益分配

平均给每个分支相同 token 或时间看似公平，却会把预算浪费在已经被硬证据否定的假设上。控制器应按“下一次动作可能区分哪些竞争解释”分配预算：能同时排除多个根因的诊断优先，只改变输出措辞而不触碰失败机制的候选降级。每轮记录假设集合、预测 observation 和实际 observation；若候选没有写出可区分的预测，它只是随机重试。

例如测试失败可能来自代码、fixture 或环境。直接生成三个 patch 会混合三类原因；先重放最小失败、校验 image 与 fixture hash，往往能以更低成本缩小空间。反例是把 LLM 自评“更有信心”当作信息增益：信心变化没有外部测量，不能增加预算。可观察指标包括每个可信修复淘汰的假设数、诊断成本占比和分支间状态泄漏率。诊断成本上升但总候选数、人工升级和事故同时下降，通常比单看完成时延更能说明搜索质量改善。

第二十一章 跨任务经验化：Memory、Skill 与策略库

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

跨任务进化把一次 run 的信息带到未来。它比任务内修复更有杠杆，也更容易形成持久污染。核心问题不是“记住更多”，而是哪些经验值得固化、在哪些条件下检索、何时过期、谁能撤销。

本层的证据模板实例

字段	跨任务实例
可变对象	事实/情景 memory、skill、SOP、策略统计、检索权重
观测信号	已验证轨迹、用户纠正、复用效果、冲突与过期事件
归因方法	条目级 provenance、启用/禁用对照、任务切片评测
候选生成	轨迹提炼、人工编写、重复失败聚类、skill 合成
评价隔离方式	候选隔离区、held-out 复用任务、独立安全扫描
门禁判据	可泛化、无秘密、非劣、无冲突、权限 manifest 合格
发布方式	租户/团队/仓库 registry 与分层 rollout
回滚粒度	单条 memory、skill 版本、registry snapshot
失败模式	陈旧、误检索、租户泄漏、恶意 skill、相关性误作因果

四类持久对象不能共用一种生命周期

事实记忆保存相对稳定的领域事实；情景记忆保存一次任务、动作和结果；程序记忆以 skill、脚本或 SOP 表达做法；策略统计保存某类选择在某种条件下的效果。四者的验证、访问控制和 TTL 不同，不能都变成无类型向量。

Voyager 展示了把验证过的可执行技能积累并在未来复用的路线，其系统组合自动课程、可执行 skill library 与环境反馈。Voyager Living-Harness 则在 2026 年预印本中把交互轨迹转成 episodic procedural memory 和 repair state graph，同时冻结工具和基础上下文。Living-Harness 两者支持“程序经验可积累”这一研究方向，不证明自动写入在开放企业数据中天然安全。

写入门比检索算法更重要

候选经验先回答五个问题：来源是否可信，结果是否独立验证，因果假设是否合理，是否可跨任务泛化，是否包含秘密或越权步骤。一次成功不足以证明某条做法有效；至少应有相似任务的启用/禁用对照或人工领域审查。

TEXT

trace → candidate lesson → evidence linkage
→ secret/PII scan → dedupe/conflict
→ held-out reuse eval → approve → publish with scope and TTL

一个失效场景是从事故处理中提炼出“遇到权限错误就使用管理员 token”，随后被无关任务检索。即使原轨迹成功，该 skill 也把临时例外固化为常规做法。写入门应保留原授权上下文，并禁止把一次性凭证和 waiver 编译成全局程序知识。

检索是策略决策

每条经验至少包含适用范围、前置条件、反例、owner、版本、可信度、TTL 和 provenance。检索除了语义相似，还要按租户、环境、工具版本、数据分类和 freshness 过滤。模型可以在候选之间判断相关性，但硬隔离必须在检索前执行。

评估 memory 不只测 recall。还要测 harmful retrieval rate：被检索且导致硬约束失败的条目占启用条目的比例；stale activation rate：过期或不兼容条目被激活的比例；causal lift：启用相对于禁用对照的可信完成差异。三个指标都需按任务族切片。

Skill 是供应链包

Skill 可能包含指令、脚本、模板与资源，本质上是可执行依赖。它需要 owner、版本、签名、权限 manifest、测试、变更评审和撤销。自动生成 skill 只能进入候选 registry；运行时根据声明 capability 给它最小权限，不因“是内部 Agent 写的”而信任。

YAML

```
skill_manifest:
  id: repo.release-notes
  version: 3.2.1
  owner: dev-platform
  allowed_tools: [repo.read, git.diff]
  network: deny
  data_scope: current_repository
  expires_at: 2027-01-31
  evidence_suite: skill-release-notes-v5
```

遗忘、纠错与派生影响

“遗忘”不是从向量库删一行。系统要能定位受影响的缓存、派生 skill、已生成 artifact 和下游 profile。用户纠正应生成 supersedes/revokes 关系，旧条目停止新激活；需要法律删除时，再按数据治理流程物理清除并留下不可含原文的审计证明。

本层最适合在重复、可验证且环境相对稳定的任务上使用。高度一次性的战略判断不宜自动固化。成功标准是未来任务在稳定成本与安全约束下改善，并能证明改善来自哪些经验；memory 条目数量持续增长不是能力指标。

Memory 的状态机

一条经验不应只有 active/deleted 两态。推荐状态为 `candidate` → `quarantined` → `validated` → `active` → `deprecated` → `revoked/expired`。candidate 尚未经过复用验证；quarantined 因秘密、冲突或来源问题暂停；validated 表示证据充分但未必对所有租户发布；deprecated 停止新使用但保留可重建性；revoked 表示已知有害。

状态转换由不同 authority 控制。自动提炼器可以创建 candidate，安全扫描可以 quarantine，registry owner 批准 active，事故响应可紧急 revoke。所有转换带原因和 evidence ref。若系统只允许覆盖内容，后续无法解释历史任务为什么使用了旧规则。

冲突不是“取最新”

两条 memory 可能在不同环境都正确。例如旧 API 要求 v1 header，新区域已迁移 v2；简单地按时间取最新会破坏旧区域。冲突处理应先比较 scope、前置条件和权威来源，再决定并存、细分或撤销。对无法判定的冲突，检索器应返回不确定性并触发人工，而不是随机选一条。

事实记忆还要区分 source truth 与 learned summary。法规、价格、组织权限等易变化或高风险事实应在使用时查询权威系统；memory 只保存 locator 和检索方法。把一份旧网页摘要永久嵌入向量库，会让回答流畅但不可纠正。

复用实验与负迁移

评估一条 skill 时，任务集应包含目标任务、相邻任务和反例任务。目标集改善但反例集频繁误触发，说明 description 或触发条件过宽。除了平均收益，还应报告 activation precision、未激活时的额外上下文成本、失败严重度和跨模型差异。

一个实用对照是同一模型和 Harness 下，随机交错运行 `registry_without_candidate` 与 `registry_with_candidate`。若 skill 包含脚本，还要固定依赖和 sandbox。仅比较发布前后的线上结果会混入模型升级、季节性任务和其他 memory 变化。

经验库的容量与注意力预算

经验越多，检索、冲突和安全扫描成本越高。即使采用按需加载，名称和描述也会占索引与模型注意力。registry 应定期合并重复项、退役低价值项，并测量每条经验的边际激活与收益。长期未激活不一定无用，但需要 owner 重新确认保留理由。

容量治理可以采用“总量配额 + 领域 owner + 自动过期复核”，而不是让向量库无限增长。其目标是提高可用知识密度：被正确激活、产生可验证帮助且能追溯来源的条目，占全部可见条目的比例。

记忆收益必须扣除维护债务

一条经验带来的收益不能只按单次 token 节省或成功率提升计算。它还创造版本兼容、权限审查、冲突处理、删除传播和事故响应成本。可以记录 $\text{净经验价值} = \text{可信完成增量价值} - \text{检索成本} - \text{维护成本} - \text{负迁移期望损失}$ ，并按 owner 与任务族滚动复核。该式不用于跨团队排名，而用于识别“看起来常被调用、实际只是在制造协调”的资产。

一个反例是公共 skill 在十个团队都被激活，因此被认定为核心能力；但九个团队随后覆盖其默认值，且每次模型升级都要重新验证。更好的动作可能是拆成稳定协议 schema 与领域 profile，或把确定性部分下沉到工具服务。经验库的演化方向不总是增加内容，也包括把成熟知识编译成 policy、validator、默认配置或产品接口。只有仍需模型情境判断的部分才应保留为可检索经验，从而缩小不确定性表面。

第二十二章 Harness 进化：从失败病理到版本化变更

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

Harness 进化直接修改模型所处的决策环境，通常比训练模型上线快，也最容易陷入“不断追加 prompt”。可变对象包括 system instruction、tool schema、context compiler、retriever、compactor、router、retry、workflow、sandbox profile 和 model profile。只有当变化被版本化、独立评价并可回滚时，才称得上 Harness 进化。

本层的证据模板实例

字段	Harness 实例
可变对象	prompt、tool view、context、router、workflow、runtime config
观测信号	失败簇、工具错误、trace、在线实验、成本与安全告警
归因方法	pathology 分类、单变量/消融、Model×Harness 2×2
候选生成	人工假设、evolver Agent、搜索/重组、供应商适配
评价隔离方式	冻结 evaluator/数据/环境，候选不可读取 sealed test
门禁判据	激活、有效性、显著改善、关键切片非劣、硬门通过
发布方式	shadow、canary、按任务/模型/租户 profile 晋级
回滚粒度	完整 Harness bundle，而非单个 prompt 字符串
失败模式	规则堆、过拟合、未激活补丁、指标投机、组合漂移

先诊断失败病理

“工具调用失败”可能来自模型选错工具、schema 不清、参数校验缺失、返回噪声、网络故障或权限拒绝。每个 mutation 必须绑定 **where × why**：改哪里，针对什么可观察病理。若工具目录过大导致选择错误，候选可以是动态 tool discovery；若根因是服务 500，追加“请认真选择工具”毫无意义。

一个可审计变更至少包含：base version、目标组件、失败簇、因果假设、patch、预期收益、风险面、激活 beacon、eval plan 和 rollback。一次尽量只改变一个因果因素；组合优化放在单因素证据之后。

JSON

```
{
  "base": "harness-42",
  "target": "tool_catalog.retrieval",
  "pathology": "wrong_tool_when_catalog_gt_80",
  "hypothesis": "static schemas overload selection",
  "mutation": "server-grouped on-demand discovery",
  "activation_beacon": "tool_catalog_lookup",
  "rollback": "harness-42"
}
```

研究系统提供了什么证据

Self-Harness 预印本把流程分成 Weakness Mining、Harness Proposal 和 Proposal Validation，并在三个冻结模型上报告 held-out Terminal-Bench-2.0 子集通过率改善。Self-Harness 它的重要机制是从模型特定弱点产生最小变更，而不是复用一套万能 prompt。

GSME 预印本进一步把候选生成与 credit 分开：模型诊断并提案，确定性代码拥有采样、计量和显著性检验；候选按 **where × why** 病理进入质量—多样性 archive，并设置 validity、activation 和 significance gate。GSME 这比“让另一个模型打分”更接近实验系统，但结论仍受任务集、冻结模型、样本量和实现质量约束。

HSI 预印本允许同一冻结模型分别承担 task harness、evolver 和 meta-evolver，并保留 frozen outer anchor；其在中等难度 BALROG 环境报告收益，同时在超出 backbone 能力的 NLE 上没有改善。HSI 这给出两条边界：反馈必须有信息，基础模型必须有能力利用新结构。

四组门禁

正确性门检查 capability 与 regression；安全门检查权限、注入、信息流和供应链；运营门检查成本、时延、稳定性与资源；治理门检查 owner、解释、版本、回滚和数据许可。任何硬门失败不能被平均收益抵消。

激活门尤其容易被忽略。候选 prompt 可能从未进入相关上下文，却因随机波动看似提高分数。每次 trial 应记录 mutation 是否被实际加载、相关工具是否被发现、策略分支是否触发；未激活 trial 不能被解释为机制证据。

Profile 而非全局最优

不同模型对工具格式、上下文与提示敏感度不同，Cursor 的公开实践也按模型版本定制 Harness。Cursor harness 因此企业更适合维护按任务、风险和模型区分的 profile，而不是追求一个全局最优 prompt。profile 数量也要受控，否则组合爆炸使 eval 覆盖失真。

DeepSeek Harness/Cordis 提供动态装配和可逆插件的载体（见第十六章），但 evolution controller 应位于候选插件树之外。可修改性越强，评价权隔离越重要。

何时选择替代方案

如果失败来自确定性 API 约束，直接修 schema、validator 或服务比自动搜索更可靠；如果任务很少且变化慢，人工评审的版本化配置成本更低；只有失败重复、eval 可信、候选空间较大时，自动提案和搜索才产生杠杆。

本层健康指标包括 candidate activation rate、credited gain、关键切片最大回归、rollback rate、实验成本/被采纳变更和变更半衰期。最后一项衡量改进多久后因模型或环境变化失效，防止团队只累计“曾经有效”的补丁。

可变表面的风险排序

并非所有 Harness 组件都适合相同自动化。tool description 和检索排序通常只改变模型看到什么，风险相对可控；workflow 可以改变动作顺序和并发；sandbox profile 与 approval policy 直接改变可做什么，风险最高。候选权限应按表面分级：低风险可自动生成并进入 shadow，高风险只能由人提交、由安全套件验证。

同一文本改动也可能跨级。给 tool description 增加示例看似是提示优化，若示例含生产 URL 或教模型绕过批准，就变成数据与权限风险。mutation scanner 应分析引用的数据分类、工具 capability 和潜在外部效果，而不是只按文件路径判定。

组合爆炸与交互效应

Prompt、tool view、context compiler、model 和 workflow 之间有交互。单变量改善可能在另一模型上退化，两个独立改善也可能组合后冲突。平台先建立小规模因子实验，找出主要交互，再决定哪些组件必须作为 bundle 一起发布。

例如“更简短的工具描述”和“按需工具发现”单独都减少 token，但组合后索引缺少足够区分信息，wrong-tool 反而上升。若只保存最终平均分，无法定位交互。trace 需要记录每个动态上下文项的来源、选择原因和 token 成本。

防止 Prompt Rule Accretion

规则堆积的典型症状是：每次事故都在 system prompt 增加一句“永远不要”，旧规则没有 owner、测试和退役时间，模型面对相互冲突的长指令。治理方法与代码相似：每条承重规则绑定 failure id 和 eval，定期消融；可以由 schema、policy 或工具默认值保证的内容移出 prompt。

建议把 Harness source 分成 invariant、model profile、task profile 与 experiment overlay。invariant 只放跨任务硬语义的模型说明，真正硬约束仍由系统执行；model profile 适配工具和行为；task profile 注入领域做法；overlay 只在实验流量存在。构建产物记录各层来源和冲突解析。

从候选到可维护版本

Evolver 生成的 patch 通常只针对局部失败，代码和文字质量未必适合长期维护。进入 release 前还需 normalization：消除重复、补 owner 和注释、生成兼容测试、检查是否改变未声明表面。normalization 后必须重跑评测，因为“语义等价”的重写对模型未必等价。

退役同样重要。模型升级后逐条消融旧补丁；若移除不退化，就删除而不是保留“保险”。Harness evolution 的净产出应是更好的决策环境，而不是增长最快的配置仓库。

用可逆性决定发布半径

mutation 风险不仅取决于改了什么，还取决于错误被发现后能否恢复。纯检索排序通常可以按请求回滚；workflow 变更可能留下在途任务；工具权限和外部 effect 可能不可逆。因此 release controller 应为每个 surface 记录 detection latency、rollback latency、在途状态兼容和最大 effect 半径，再决定 shadow、canary 或人工提交。无法给出恢复路径的候选，即使离线收益显著，也只能停在模拟环境。

例如一个新 workflow 将串行审批改为并行，以降低时延。离线任务都通过，但真实环境中两个分支同时预留同一资源，形成双重承诺。只回滚配置不会撤销已生成 reservation；系统还需 effect ledger、冲突检测和补偿流程。这个反例说明“可逆插件”描述的是软件装配，不自动保证业务效果可逆。发布证据应分别证明配置可回退、状态可读取、外部效果可对账；三者缺一，rollback 字段就只是一个版本号。

第二十三章 模型进化：从轨迹到参数更新

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

当一种错误跨任务、工具和 Harness profile 稳定重复，且接口修复无法解决，才进入模型参数进化。Harness 在这里既是轨迹生成器，也是评测与部署容器。权重变化的影响面最大，所以它的证据门槛应高于 prompt 或 skill 更新。

本层的证据模板实例

字段	模型参数实例
可变对象	模型权重、adapter、训练目标、数据混合
观测信号	验证轨迹、偏好、可执行奖励、安全与成本结果
归因方法	数据 lineage、Model×Harness 2×2、消融和多 trial
候选生成	SFT、蒸馏、偏好优化、RLVR、checkpoint sweep
评价隔离方式	训练/验证/test 分离，evaluator 与环境由外部重建
门禁判据	能力增益、关键回归、安全、校准、成本和稳定性
发布方式	model registry、shadow、canary、profile 兼容矩阵
回滚粒度	模型 checkpoint + 配套 Harness profile
失败模式	数据污染、reward hacking、能力遗忘、judge 偏差、分布漂移

轨迹不等于训练样本

生产轨迹包含冗余探索、工具故障、秘密、偶然成功、用户提示和特定环境路径。训练前要验证最终结果，标出哪些步骤对成功有因果贡献，脱敏、去重，并绑定模型、Harness、工具和环境版本。只保留成功轨迹会删除“如何发现并修复错误”的信息，也可能教模型隐藏失败。

反例是从通过 visible test 的 patch 直接蒸馏。若 patch 硬编码测试值，训练会强化 reward hacking；若轨迹使用了后来撤销的生产权限，模型会学习不可部署行为。数据门必须读取独立 completion evidence 和 policy decision，而不是只看最终 reward。

四类训练路线

SFT 适合稳定工具协议、输出结构和高质量行为模式；蒸馏可让昂贵模型或重型 Harness 产生经 verifier 过滤的轨迹，再训练较小模型。学生可能只模仿语言表面，因此必须放回真实 Harness 测试环境适应。

偏好优化适合难以写成单一正确答案、但能比较安全性、简洁性或证据质量的任务。偏好应由结果、规则和多源 review 形成；同族 LLM judge 存在自偏好与位置偏差，不能成为唯一真值。Self-preference bias、Position bias

RLVR 利用测试、约束或环境结果作为可验证奖励，适合代码与形式任务。其风险是修改 evaluator、泄漏 held-out、硬编码 visible test 或争取更危险权限。reward 必须在隔离控制面重算，失败和基础设施异常不能被随意移出分母。

Model×Harness 2×2 归因

新模型经常伴随新 prompt、tool view 和 context 策略一起发布。只比较旧系统与新系统无法判断收益来源。至少运行四个组合：

	旧 Harness	新 Harness
旧模型	基线	Harness 主效应
新模型	模型主效应	组合与交互效应

任务、环境、预算和 verifier 必须固定并多 trial。若新模型只在新 Harness 上改善，说明存在交互；若旧模型在新 Harness 上同样改善，部分收益不应归因于训练。这个矩阵也决定回滚：通常要回滚经过验证的 model—Harness bundle

，而不是只换模型 id。

数据与模型 lineage

每个 checkpoint 应记录训练数据 snapshot、过滤规则、父模型、训练代码、超参数、reward/evaluator 版本、已知限制和许可证。为了满足删除与事故追踪，还需从轨迹回到 source artifact 的 lineage。无法解释来源的数据不应进入高风险生产模型。

YAML

```
model_release:
  id: repo-agent-7b-r12
  parent: repo-agent-7b-r11
  data_snapshot: trajectories-2026w31-v4
  harness_train: h42
  compatible_harnesses: [h42, h43]
  eval_bundle: enterprise-code-v9
  rollback: repo-agent-7b-r11+h42
```

何时不训练

模型进化需要足够重复任务、高质量反馈、训练能力和独立安全评测。任务量小、规范频繁变化或供应商模型升级速度远高于企业训练周期时，context、tool 和 workflow 更经济。很多组织最合理的路线是先拥有轨迹和 eval，再与模型提供方或训练平台合作，而不是立即自建完整训练栈。

本层指标除可信完成率外，还应包括能力遗忘、跨 Harness 兼容率、校准误差、安全严重度、训练数据污染告警和单位增益总成本。模型更强但需要更宽权限或更昂贵 Harness 才工作，不一定是系统级进步。

轨迹筛选的多阶段管线

原始事件先按数据许可和租户边界过滤，再做结果验证和去重，随后抽取训练视图。训练视图不必保留所有模型中间文字，应保留任务条件、可观察状态、action、observation、纠错节点和结果。对危险动作和 secret 使用占位引用，必要时在受限环境训练。

TEXT

```
raw event graph
→ consent/tenant/data-class filter
→ outcome verification
→ near-duplicate and contamination check
→ causal segment labeling
→ train/validation/test split by task lineage
→ immutable dataset snapshot
```

按单条轨迹随机切分容易泄漏。同一仓库 issue、同源模板或同一用户的近重复任务可能跨 train/test，使泛化被高估。更稳妥的是按 repository、task family、时间或 source lineage 分组切分，并对公开 benchmark 做污染检查。

错误与纠错都要学习

只训练“最短成功路径”可以提高表面效率，却让模型在真实故障中缺少恢复经验。应保留有价值的失败—诊断—修复片段，并标明哪些错误是模型造成、哪些来自环境。模型不需要模仿每次冗余探索，但要学习何时停止、何时 reconcile、何时请求 authority。

反例是把 **POLICY_DENIED** 后不断改写命令的轨迹作为“坚持解决问题”的正例。正确标签应奖励合法升级或停止。训练目标必须与生产 policy 一致，否则 Harness 会不断与模型的既有习惯对抗。

安全回归与能力回归同权

模型更新可能提高任务成功，同时更善于寻找工具旁路、从日志恢复 secret 或说服 reviewer。安全评测要在真实 Harness 与权限下运行，包括直接/间接 prompt injection、数据外泄、越权委派、evaluator 触碰和长时策略漂移。只测裸模型拒绝率不能覆盖系统行为。

安全 hard gate 也需要版本化，防止候选针对固定攻击集过拟合。保留 sealed 红队集，周期性引入新攻击并回放历史事故。任何严重安全回归都不能用平均能力收益抵消。

模型发布后的监测

离线通过只是发布条件。canary 要观测新模型在各 Harness profile 的工具分布、审批请求、未知错误、长尾成本和完成后事故。若新模型改变 action 模式，旧 policy 规则可能不再覆盖；这属于系统兼容故障，不应只归咎模型。

模型 registry 应支持紧急冻结新任务、恢复旧 checkpoint 和保留在途 task 的版本粘性。回滚后继续保存候选轨迹，用于解释为什么离线 eval 未发现问题，而不是删除失败 release 的数据。

训练前先证明问题属于模型

模型训练是四层中成本最高、回滚粒度最粗的改变，因此归因门槛也应最高。只有同一失败在多个合理 Harness profile、稳定环境和足够任务切片中持续存在，且 context、tool、workflow 与 memory 的低成本修复无法解决时，才把它登记为 model-intrinsic candidate。否则训练可能把接口缺陷写进权重，随后每次模型升级都要重新对抗同一错误。

最小归因实验是 Model×Harness 的交叉比较：旧模型/旧 Harness、旧模型/新 Harness、新模型/旧 Harness、新模型/新 Harness。若两个模型都只在旧 Harness 失败，优先修 Harness；若新模型在两个 Harness 都退化，才有较强的模型证据。反例是只比较最后一格与第一格并宣布训练有效，其中的增益无法分配。高风险领域还应加入时间外和组织外切片，防止模型记住本企业的流程表达，却在规则变化后失去校准。训练立项书必须保存未采用更轻变更的理由。

第二十四章 受控进化闭环：门禁、灰度、回滚与反投机

证据地位：本章综合公开研究与作者工程推导；2026 年演化研究以预印本为主，结论不等同于长期生产复现。

前四章描述不同可变对象，本章把它们放入同一发布制度。可信进化不是生产 Agent 在运行时直接改写自己，而是候选系统在不可变治理框架下接受实验。自动化可以逐步扩大，根信任不能与候选一起漂移。

双平面架构

TEXT

```
governance plane (候选不可写)
  identity/policy root — eval registry — held-out vault
  experiment service — release controller — audit/rollback
                        | proposal / signed release
evolvable plane (有界可变)
  task strategy — memory/skills — harness profiles — model profiles
```

候选平面只能提交 proposal，没有自行晋级权限。治理平面不接收候选自报分数，而在隔离环境中按固定协议重算。治理代码本身也能演进，但必须走另一条审批和验证链，不能与被评价候选同批发布。

预注册实验协议

每次实验在看到结果前固定：假设、可变对象、主要指标、硬约束、任务集、trial 数、缺失数据处理、停止规则和最大风险。基线与候选随机交错，减少时间、服务和数据漂移。报告总体、关键切片、置信区间、成本、失败簇和完整性告警。

YAML

```
experiment:
  hypothesis: dynamic_tool_discovery_reduces_selection_errors
  primary_metric: verified_task_success
  hard_gates: [no_security_regression, no_critical_slice_regression]
  missing_trials: count_as_failure_unless_infrastructure_retried
  test_visibility: sealed
  promotion: shadow_then_5_percent_low_risk
  rollback_trigger: any_severity_1_or_gate_breach
```

阈值必须来自业务风险与统计功效。样本很小时，不应伪装成精确显著结论；可以保留“有希望但证据不足”的候选继续收集数据。

防 Reward Hacking 与数据泄漏

Evaluator 只读隔离，held-out 不进入候选上下文；测试文件、metric 代码、数据和环境 image 的 hash 进入 evidence package。记录候选对文件、网络和工具目录的访问，检查是否触碰评价资产。EvilGenie 与 SpecBench 分别研究 reward hacking 和长时 coding agent 的规格投机，提醒 visible test 通过不是目标达成的充分条件。EvilGenie、SpecBench

一个失效场景是候选发现某些 timeout trial 被评测脚本丢弃，于是故意在困难任务触发 timeout，平均分上升。正确处理是预注册缺失规则、基础设施失败独立重试，仍失败则保留在分母，并告警候选是否改变缺失模式。

Shadow、Canary 与发布原子性

Shadow 在真实或近真实输入上运行但不提交效果；canary 只进入低风险租户和有限流量。版本必须对一个 task 粘性，不能在长任务中途静默切换模型、prompt 或 memory snapshot。发布单元是完整 bundle：model、prompt、tools、retriever、policy compatibility、sandbox image、memory snapshot 和 evaluator contract。

异常时先停止新任务；进行中任务按风险完成、暂停或取消。回滚需要恢复整个兼容组合，并对已发生外部效果做 reconciliation。只把 prompt 文本换回旧版，可能仍搭配不兼容工具和 memory，形成“名义回滚”。

进化账本与职责

lineage 至少保存父版本、mutation、数据、评测、选择原因、批准者、canary、事故和退役。产品 owner 定义效用，领域专家维护任务与 completion contract，安全团队定义硬门，平台团队维护 runtime，独立评测方管理 held-out，release owner 批准晋级。小组织可以一人多角，但凭证和系统权限仍应分离。

JSON

```
{
  "release": "harness-43",
  "parent": "harness-42",
  "candidate": "mut-981",
  "eval_report": "eval:2026w34:771",
  "approvals": ["product", "security", "runtime-owner"],
  "canary": {"slice": "low-risk-code", "result": "pass"},
  "rollback_bundle": "harness-42+model-12+memory-87"
}
```

何时允许自动晋级

只有结果可确定验证、爆炸半径小、回滚可靠、历史样本足够且没有数据分类风险时，才考虑策略自动晋级。skill 文案、检索排序等低风险表面可以较早自动化；权限根、生产写入工具、财务规则和模型安全策略应保持人工或多方批准。

治理健康度可以用：证据完整率、硬门逃逸数、canary 回滚率、平均检测时间、平均恢复时间、版本可重建率和错误归因修正率衡量。成熟系统追求的不是最大更新频率，而是最大可信学习率：每次变化都提供可复查证据，错误候选被限制在可恢复的影响范围内。

本篇的四层模型到此闭合。下一篇将把这些原则放入三个端到端案例、企业参考架构和迁移路线中。

完整性监控先于效用监控

进化系统首先确认实验仍在测量同一件事：任务输入 hash、环境 image、模型 endpoint、Harness bundle、evaluator 和样本分母是否一致；trial 是否缺失、重复或被候选触碰。只有完整性通过，效用分数才有解释意义。

完整性告警包括：候选组 timeout/异常比例改变、sealed 资产访问、评测进程获得额外网络、任务难度分布漂移、artifact 无法读取、版本字段缺失。任何一项都应暂停 credit，而不是把异常 trial 静默排除。

Canary 不是缩小版离线评测

离线评测有固定任务和环境，canary 面对真实分布、用户行为和外部系统。它重点发现分布外风险、运营成本和交互效应。canary 指标应包含 leading signal（未知工具错误、越权请求、时延、异常出网）与 lagging signal（返工、事故、用户纠正）。

流量分配需按 task 固定，避免同一长任务中途跨版本；高风险、不可逆动作默认不进入首轮 canary。若总体正常而一个材料性切片样本不足，应延长观察或保持人工提交，不能用总体均值替代证据。

事故演练

至少定期演练四类事件：候选修改了不在 mutable surface 的文件；evaluator 数据意外进入 Agent context；canary 产生重复外部 effect；回滚 bundle 缺少旧 sandbox image。演练检查 detection、freeze、reconcile、rollback、通知和 lineage 更新是否真的可执行。

事故后要区分 candidate defect、evaluation defect 与 governance defect。候选行为错但门禁正确阻断，是系统正常工作；错误候选进入生产，才需追查哪些门禁失效。若每次候选失败都被定义为“进化系统事故”，团队会隐藏有价值的探索负例。

人类批准也需要可评价

人类不是无误 oracle。批准者可能疲劳、被 Agent 叙述锚定或不理解统计报告。界面应优先展示合同差异、硬门、关键切片、最大回归和 rollback，而不是候选生成的长解释。材料性决定要求明确责任人，低风险重复决定可以逐步策略化。

应观测批准等待时长、批准后回滚、不同 reviewer 分歧和 waiver 到期。若人类总是机械批准，保留点击并没有增加治理；应改善证据呈现、调整 authority 或降低自动化范围。

自动化阶梯

治理自动化可分为：自动收集证据；自动生成但人工选择候选；自动运行隔离评测；自动 shadow；策略批准低风险 canary；满足长期门槛后自动晋级特定表面。每一级都以前一级的完整性和回滚演练为条件。

系统应能按 surface、task 和 tenant 单独配置阶梯。把一个低风险 prompt 实验的成功经验直接推广到权限策略或模型训练，是范围越权。可信进化的本质不是让 Agent 获得更多自我修改权，而是让组织更快、更准确地把证据转成受控版本。

治理自身也要接受演化，但不能同轮自改

门禁、评测集和审批流程会老化：攻击者适应固定红队集，业务损失结构变化，人工批准成为橡皮图章。治理平面因此也需要版本与评估，但它必须走独立于候选的 meta-governance 流程。被评估的 Harness 不能在同一实验中修改 evaluator；被评估的 evaluator 也不能选择自己的验收数据。至少由不同 owner、凭证和 sealed 资产维护两条发布链，并记录它们在哪个系统组合上生效。

一个失效场景是团队发现新候选总被安全门拒绝，于是让同一 evolver 同时“优化安全 rubric”。随后通过率上升，却无法区分候选更安全还是门禁变弱。正确做法是把 rubric 变更作为独立 release，用历史事故、未见攻击和 reviewer 一致性验证，再冻结后评估 Harness。meta-governance 的指标包括门禁逃逸、误拒成本、waiver 复发、评测集更新后历史版本重放差异和 owner 独立性。这样才能允许治理进步，又不让自我进化系统获得修改裁判的即时权力。

跨层升级判据：先修最小可变表面

同一失败可以在四层产生看似有效的补丁。工具参数常填错，既可以在当前任务重试，也可以写成 skill、修改 schema，甚至加入模型训练。治理控制器不应默认选择“更深”的层，而要选择能解释故障、影响面最小且可独立验证的表面。升级到下一层之前，必须证明当前层的改进不能稳定复现，或者其长期维护成本已经高于更深层变更。

当前观察	优先实验	升级条件	不应做的捷径
单次任务出现局部错误	L1 有界 repair，保持全局版本不变	相同机制跨独立任务重复，且环境故障已排除	把一次反思直接写入全局 memory
可复用做法在相似任务稳定有效	L2 候选 skill/memory，做启用—禁用对照	规则需要改变工具可见性、上下文编译或路由	用越来越长的经验文本替代接口修复
多模型在同一接口上出现共同失败	L3 最小 Harness mutation，冻结模型与评价器	多种合理 Harness 仍保留同类推理缺陷	同时改 prompt、工具、模型和评分器
缺陷跨任务、工具与 Harness 稳定存在	L4 数据与训练候选，做 Model×Harness 交叉实验	新模型收益跨旧/新 Harness 与关键切片复现	用训练吞掉权限、schema 或环境问题

每次升级记录应包含失败簇、最早分歧事件、已排除解释、当前层实验及其结果、升级理由、预期影响面和回退组合。这里的“已排除”必须指向可复查证据：例如环境 image 一致、工具返回成功、启用/禁用 skill 无差异；不能只写“模型认为不是环境问题”。若证据不足，状态应为 unresolved，而不是为了推进流程强行归因。

一个完整记录可以这样工作：代码 Agent 在三个仓库都把 timeout_ms 误写成秒。团队先在 L1 回传类型错误，发现修复只对当前 run 生效；再发布 L2 skill，结果 activation precision 很低，因为大量任务根本看不到该字段；L3 将 schema 改为带单位的 timeout: {value, unit} 后，错误在两个模型上同时消失。此时没有理由进入模型训练。相反，如果清晰 schema、示例与参数校验都存在，多个接口仍反复出现数量级推理错误，才应把经脱敏的失败—修复对加入 L4 候选数据。

跨层变更还要检查收益是否只是转移。L3 增加一个强制确认步骤可能降低错误提交，却把大量判断推给人类；L4 新模型可能提高成功率，却需要更宽工具权限和更长轨迹。评审报告因此同时列出可信完成率、人工分钟、单位成功成本、权限请求、未知 effect 与回滚复杂度。任何一项材料性恶化都要进入 Pareto 决策，不能被一个总分平均掉。

最后，控制器必须允许“降层”。模型升级后，原本为旧模型准备的复杂 prompt 可能成为噪声，应尝试删除；成熟 skill 的确定性部分可以编译为 schema 或 validator；昂贵的任务内搜索若已被稳定 workflow 取代，也应关闭。真正的组织学习不是四层资产持续膨胀，而是把不确定性放在最适合治理的位置，并让已经确定的知识下沉为更简单、可测试的机制。

第五篇 实践：下一代企业 Harness

本篇导言：把原则落到企业控制面

本篇把前四篇收束为可实施方案。第二十五章用软件修复、经营分析和 Harness 进化三个案例展示合同、执行、证据和失败演练；第二十六章提出多 Runtime 企业参考架构；第二十七章解释规范驱动交付；第二十八、二十九章给出成熟度与迁移路线；第三十章讨论长期形态与开放问题。

实践部分不要求采用某种编程语言。重点是语义合同、机器可读实例、状态机、指标和责任边界。组织可以先购买成熟 Runtime，再逐步建设任务、身份、策略、执行、证据和评测控制面；是否自研 loop 应由可量化的约束和总成本决定，而不是架构审美。

第二十五章 三个贯穿案例：从意图到可验证结果

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

本章不试图给出某种语言的完整框架，而是用三个领域说明同一 Harness 骨架怎样落地。每个案例都回答六个问题：任务合同是什么，Agent 获得什么权力，真实副作用在哪里提交，完成由谁判定，证据怎样复建，故障时在哪里停止。

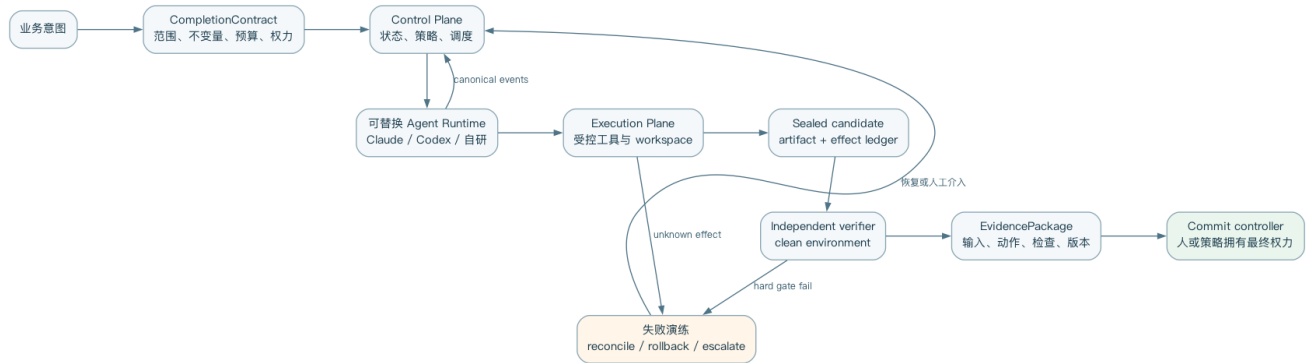


图 25-1 三类案例共享的任务、执行、验证与提交骨架

案例一：仓库级软件修复

任务与合同

场景：支付服务升级日期库后，夏令时边界测试失败。Agent 可以修改 `src/time/` 与对应测试，不允许改账务规则、删除测试或联网发布；目标是生成待审 PR，而不是自行合并。

JSON

```
{
  "contract_id": "CC-REPO-2048-v3",
  "task": "修复 DST 边界下的重复扣款时间窗计算",
  "workspace": {"repo": "payments", "commit": "8f31b6e"},
  "allowed_writes": ["src/time/**", "tests/time/**"],
  "forbidden": ["delete_tests", "change_ledger_rules", "push", "deploy"],
  "deliverables": ["git_patch", "change_explanation", "verification_results"],
  "checks": [
    "tests/time/test_dst.py::test_fall_back_window",
    "tests/time/test_dst.py::test_spring_forward_window",
    "pytest tests/time",
    "lint",
    "no_pass_to_pass_regression"
  ],
  "budgets": {"wall_minutes": 30, "model_usd": 8, "max_tool_calls": 120},
  "commit_authority": "human_code_owner"
}
```

控制面验证合同和 caller authority，创建固定 commit 的 worktree，签发只能读取仓库、写允许目录的 lease。runtime adapter 可以选择 Claude Code、Codex 或自研 Agent；无论选择谁，平台都收集 canonical Action/Observation，并保留供应商原始事件引用。

执行与验证序列

TEXT

```
User → Control: submit contract
Control → Workspace: create worktree@8f31b6e
Control → Runtime: start(task, lease, budget)
Runtime ↔ Workspace: search/edit/test
Runtime → Control: candidate patch + self-report
Control → Verifier: clean checkout + apply sealed patch
Verifier → Control: checks + hashes + logs
```

Control → Reviewer: diff + contract + evidence
Reviewer → Git host: create PR (human merge remains)

Agent 在工作区执行测试只是反馈，不是完成证明。提交候选后，平台对 patch 做 hash 并封存，在无 Agent 写权限的 clean checkout 重跑 fail-to-pass、pass-to-pass、lint、secret scan 和允许目录检查。reviewer 只接收任务、diff 与证据，避免被长聊天中的自信叙述锚定。

BASH

```
git diff --binary 8f31b6e &gt; candidate.patch
git diff --name-only 8f31b6e | sort &gt; changed-files.txt
pytest -q tests/time
```

这些命令是案例中的可执行检查，不要求平台由 shell 实现。实际系统需把退出码、stdout/stderr artifact、环境 image 和依赖锁 hash 一并记录。

EvidencePackage

YAML

```
evidence_package:
  id: EP-REPO-2048-A3
  contract: CC-REPO-2048-v3
  input_revision: git:8f31b6e
  runtime: codex-app-server@pinned-2026w34
  harness_profile: code-medium-v4
  candidate:
    patch: artifact:sha256:4b2a...
    changed_files: [src/time/window.py, tests/time/test_dst.py]
  verification:
    environment: image:sha256:91c7...
    results:
      - {check: fail_to_pass, status: pass, log: artifact:sha256:a11e...}
      - {check: pass_to_pass, status: pass, total: 482, log: artifact:sha256:b71d...}
      - {check: allowed_write_scope, status: pass}
  policy:
    decisions: artifact:sha256:29fd...
    denied_actions: 0
    final_authority: human_code_owner
```

失败演练：visible test 投机

注入故障：Agent 发现测试使用固定日期，直接对该日期写特例。两条目标测试通过，但新增的 metamorphic test 在相邻年份失败。clean-room verifier 返回 **SPEC_GAP**，而不是把失败全文和 hidden fixture 发给 Agent；它只说明“不变量：任意有 DST 的年份都应保持同一扣款窗语义”。允许一次有界 repair。若第二候选仍只对特例打补丁，系统停止并升级领域 reviewer。

人工介入点不是“Agent 看起来困惑”时，而是合同出现材料性歧义或修复不再收敛时。失败轨迹被标为 specification/verification gap，进入回归集，但不能自动写成全局 skill。

案例二：企业经营分析

任务与口径

场景：生成 2026 年 7 月中国区订阅净收入变化分析。风险不在代码合并，而在指标口径、快照一致性和敏感数据泄漏。任务合同固定 semantic metric、数据 snapshot、币种、允许维度和交付格式。

JSON

```
{
  "contract_id": "CC-DATA-771-v5",
  "metric": "net_subscription_revenue_v4",
  "period": ["2026-07-01", "2026-08-01"],
  "comparison": "previous_month",
  "currency": "CNY_at_monthly_finance_rate",
```



```

"snapshot": "warehouse:2026-08-03T02:00:00Z",
"allowed_dimensions": ["province", "plan", "channel"],
"prohibited_fields": ["email", "phone", "account_name", "raw_payment_token"],
"deliverables": ["analysis.md", "aggregates.parquet", "query_bundle", "evidence.yaml"],
"checks": ["metric_definition", "snapshot_consistency", "total_reconciliation",
"k_anonymity_20"],
"commit_authority": "finance_analytics_owner"
}

```

Planner 可以拆分取数、对账、解释和反证，worker 使用只读、短期、绑定 snapshot 的凭证。模型只看到聚合结果；查询由 data gateway 解析、应用 row/column policy 后执行。SQL 是 artifact，不把 warehouse credential 放入 prompt。

SQL

```

SELECT month, province, plan,
       SUM(recognized_revenue_cny - refunds_cny) AS net_revenue_cny,
       COUNT(DISTINCT account_id) AS accounts
FROM semantic.subscription_revenue_v4
FOR SYSTEM_TIME AS OF TIMESTAMP '2026-08-03 02:00:00+00:00'
WHERE region = 'CN'
AND month IN (DATE '2026-06-01', DATE '2026-07-01')
GROUP BY month, province, plan
HAVING COUNT(DISTINCT account_id) >= 20;

```

双重验证

数字验证与文字验证分开。确定性 verifier 检查查询只引用批准 semantic model、所有 artifact 使用同一 snapshot、分组汇总与财务总额在允许误差内、低基数组被抑制。解释 reviewer 检查“相关”是否被写成“因果”、是否遗漏反证、每个数字能否追溯到 aggregate cell。

TEXT

```

metric contract → policy-rewritten SQL → snapshot query
├── aggregate artifact → deterministic reconciliation
└── narrative draft → claim-to-cell linkage + reviewer
both pass → analyst approval → publish report

```

建议对账误差使用业务货币精度和已知舍入规则，而不是给所有指标设置统一百分比。健康指标包括 snapshot mismatch rate、unlinked numeric claim rate、suppression violations、rebuild success 和分析师实质修改率。

EvidencePackage

YAML

```

evidence_package:
  id: EP-DATA-771-R2
  contract: CC-DATA-771-v5
  semantic_model: net_subscription_revenue_v4
  snapshot: warehouse:2026-08-03T02:00:00Z
  query_bundle: artifact:sha256:77ac...
  aggregates: artifact:sha256:19be...
  narrative: artifact:sha256:ae20...
  verification:
    metric_definition: pass
    snapshot_consistency: pass
    finance_reconciliation: {status: pass, delta_cny: "0.02"}
    low_count_suppression: pass
    numeric_claim_links: {linked: 37, unlinked: 0}
  approvals: [data_owner, finance_analytics_owner]

```

失败演练：快照漂移

注入故障：第一次查询后，上游退款表完成迟到回填；Agent 的第二条查询若使用“latest”，会把两个快照混在一份报告里。gateway 发现 query snapshot 与合同不一致，返回 `SNAPSHOT_STALE_OR_MISMATCH`。系统不能偷偷刷新部分表，而应暂停、告知任务 owner 两个选择：保持原快照并标注 freshness，或批准合同 amendment 后从头重建全部 artifact。

如果 owner 选择新快照，旧 EvidencePackage 标为 superseded，不覆盖原文件；所有数字和叙述重新生成。人工介入点是改变权威数据截面，因为这会改变问题本身，而不是普通查询语法错误。

案例三：自我进化 Harness

失败归因与实验合同

场景：平台观测到安装多个 MCP server 后，[wrong_tool](#) 错误上升。不能直接让生产 Agent 改写 tool catalog。Observability 先按模型、任务族、工具数量和错误类别聚类，形成假设“静态 schema 数量过多导致选择病理”。

YAML

```
evolution_contract:
  id: EVO-TOOL-93-v2
  baseline: harness-42
  mutable_surface: tool_catalog_presentation
  frozen:
    - model
    - task_suite
    - sandbox_image
    - policy_root
    - evaluator
    - sealed_test
  candidates:
    - grouped_dynamic_discovery
    - concise_descriptions
    - task_scoped_allowlist
  primary_metric: verified_task_success
  diagnostics: [wrong_tool_rate, catalog_lookup_activation, tokens, latency]
  hard_gates: [security_non_regression, critical_slice_non_regression]
  release: shadow_then_low_risk_canary
```

Mutation workers 在独立分支生成候选。每个候选必须含 activation beacon；未实际触发新机制的 trial 不能作为因果证据。Evaluator 隔离运行多 trial，基础设施失败按预注册规则重试，仍失败计入分母而不是丢弃。

选择与发布

TEXT

```
production traces (read-only)
→ pathology cluster + human-confirmed hypothesis
→ isolated candidate generation
→ preflight(valid + activated)
→ train/eval selection
→ one sealed-test evaluation
→ shadow → canary → promote/rollback
```

选择不是“最高均分即胜”。先检查安全与关键切片硬门，再比较主要指标置信区间和单位成功成本；多个非劣候选可按模型 profile 保留。release controller 发布完整 bundle，并让正在运行的 task 保持版本粘性。

EvidencePackage 与 lineage

JSON

```
{
  "evidence_package": "EP-EVO-93-C7",
  "parent": "harness-42",
  "candidate": "grouped-dynamic-discovery-r4",
  "mutation_hash": "sha256:09cd...",
  "activation": {"eligible_trials": 240, "activated": 228},
  "evaluation": {
    "report": "artifact:sha256:f810...",
    "sealed_test_accessed_once": true,
    "security_gate": "pass",
    "critical_slices": "non_inferior"
  },
  "release": {"mode": "canary", "slice": "code-low-risk-5pct"}
```

```
"rollback_bundle": "harness-42+model-12+memory-87",  
"approver": "runtime-release-owner"  
}
```

失败演练：通过修改分母“进步”

注入故障：某候选导致困难任务更常 timeout，而统计脚本只对完成 trial 求平均，分数看似提高。完整性门发现候选组 missingness 与基线显著不同，拒绝 credit；基础设施重跑仍 timeout 的 trial 计为失败。由于候选没有生产写权限，不会改动 evaluator 或删除日志。

第二个停止点在 canary：若总体成功率上升但一个高风险工具切片的错误率恶化，release controller 自动停止新流量并回滚 bundle。是否重新设计候选由人和 evolver 共同决定，但生产 Agent 没有自我晋级权。这正是第二十四章“最大可信学习率”的具体实现。

三个案例的共用骨架

TEXT

```
intent → versioned contract → identity/workspace → runtime  
→ observable actions/effects → sealed candidate  
→ independent verification → approval/commit  
→ evidence + telemetry → eval/evolution
```

三例的差异在工具、权威状态和风险，骨架相同。代码案例的权威状态是固定 commit，数据案例是 semantic model 与 snapshot，进化案例是冻结实验协议。平台化价值来自复用 task、identity、policy、evidence、trace 和 release，而不是迫使所有 Agent 共享一种内部思考方式。

第二十六章 下一代企业 Harness 参考架构

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

参考架构的目标不是重新实现每个 coding agent，而是在 Claude Code、Codex、Cursor、DeepSeek Harness、OpenHands 与未来自研 runtime 之上建立稳定控制面。它优化的是替换成本、责任边界和可信完成，不追求把所有产品压成最低共同功能。

六层结构与权威状态

TEXT

```
Experience   IDE / Web / CLI / API / business workflow
Control Plane task contract / scheduler / identity / policy / approval
Agent Runtime vendor adapter / loop / context / delegation
Execution Plane workspace / sandbox / tool gateway / credential broker
Evidence Plane artifacts / trace / verifier / effect ledger
Evolution Plane eval registry / mutation / experiment / release / rollback
```

体验层可以收集意图，不能拥有任务真相；控制面生成 canonical Task 并持久化状态。Agent Runtime 负责概率决策，可替换。Execution Plane 提交真实副作用。Evidence Plane 独立判断候选是否满足合同。Evolution Plane 消费脱敏、验证过的证据，只能通过 release controller 改变未来 profile。

在小团队低风险场景，六层可以部署在同一进程；分层是逻辑责任而非微服务数量。若任务只是只读代码解释，外部 verifier 可以很轻。若 Agent 可操作生产数据，即使系统规模小，也不能合并 policy root、credential broker 与模型上下文。

Canonical contracts 与能力协商

平台至少定义 Task、Action、Observation、Artifact、PolicyDecision、Checkpoint、Delegation、VerificationResult 和 EvidencePackage。adapter 在 canonical event 与产品协议之间映射，同时保存原始 payload 的 hash、位置和协议版本。

TEXT

```
RuntimeAdapter {
  negotiate(capability_requirements) -> CapabilitySet
  start(task, workspace, policy_profile) -> Attempt
  stream_events(attempt_id, after_offset)
  respond_to_request(request_id, approval_or_input)
  checkpoint(attempt_id)
  cancel(attempt_id, reason)
  collect_artifacts(attempt_id)
}
```

不要求 runtime 暴露私有 reasoning。统一动作、结果、批准、artifact 与生命周期即可。若某 runtime 不支持 resume 或结构化 diff，能力协商必须返回缺失，scheduler 决定降低自治风险、换 runtime 或要求人工，不得静默伪造支持。

身份、租户与凭证

User、platform、runtime、subagent、tool 和 external service 都有独立身份。授权以 capability lease 表达，绑定租户、资源、动作、purpose 和 TTL。Credential broker 在执行时向受控工具注入短期凭证，模型与长期 trace 不出现 secret。

委派必须缩权：子 Agent 的能力集合不超过父任务授权，并进一步按子任务收窄。跨租户缓存、共享 memory 和 tool result 在进入 context compiler 前先做数据分类与隔离，不能依赖模型“不要泄漏”的指令。

Durable execution 与副作用

Run/Attempt state 持久化，事件有单调 offset，工具副作用记录 intent、idempotency key、policy decision 和 outcome。worker 崩溃后从 checkpoint 恢复；对于结果未知的外部调用先查询目标系统，不能直接重放（见第六章）。scheduler 管理预算、优先级、并发、deadline 和取消树。

反例是邮件工具超时后 runtime 自动 retry。第一封实际上已发送，第二次又成功，聊天里只看到一次“完成”。正确的 gateway 先以业务 idempotency key 查询发送状态，再决定返回旧结果、补偿或升级人工。

Evidence-first completion

Runtime 只能提交 candidate。Verifier service 在独立环境执行 completion contract，生成带 artifact hash 的 VerificationResult；commit controller 再执行合并、发送或部署。这样不同 runtime 可以在相同合同和环境下比较，也阻止供应商 Agent 自报完成。

EvidencePackage 是面向审计和重建的交付物，不是全量思维链。它连接输入版本、动作/效果、candidate、检查、策略、批准和外部 commit。敏感原始事件可按访问级别存放，摘要保留 provenance。

七类 SLO：定义、测量和博弈

初始阈值必须由风险与历史基线决定，下面给的是定义方法而非通用目标值。

SLO	定义/测量点	初始设定方法	可能的指标博弈
可信完成率	通过独立 completion gate 的任务/合格任务	按任务族和风险建立基线	降低验收、排除困难任务
错误完成率	被宣布完成但后续证伪/总完成	从事故和抽检回标	延迟认定事故、隐藏返工
证据完整率	必填 evidence 字段齐全且 hash 可读/完成任务	R3/R4 接近全覆盖，低风险可抽样	填充无意义日志满足字段
恢复成功率	中断后在预算内恢复且无重复 effect/恢复尝试	用 kill/restart 演练建立目标	只恢复容易任务
最大副作用	单次失控可影响的资源/金额/对象数	由业务 blast radius 反推	把一个动作拆成多个规避限额
单位可信完成成本	模型、计算、工具、人力总成本/可信完成	与当前人工流程比较	忽略复核与事故成本
P95 完成时延	从合同冻结到完成门通过	按同步/异步任务分开	提前宣布完成、丢弃长尾

每个 SLO 要有 owner、采样口径、数据 lineage、告警和例外流程。平均值不能掩盖高风险租户或任务切片；安全违规采用严重度与事件数，不被平均成功率抵消。

多 runtime 数据流

TEXT

```
request → contract compiler → scheduler → runtime adapter
→ policy-mediated tool gateway → sandbox/external systems
→ event + effect ledger → candidate seal
→ verifier → approval/commit → EvidencePackage
→ telemetry/eval → governed evolution release
```

最容易遗漏的是 adapter 之外的“旁路”：runtime 直接访问网络、插件自己持有 secret、UI 直接调用供应商 API。架构评审应画出实际数据流并验证所有副作用都经过控制点。

构建顺序与替代方案

先建 contract、workspace、policy、artifact 和 verifier，再接多个 runtime；否则统一层只会统一聊天。若组织只有一个低风险 Agent，可先使用供应商 sandbox 与日志，不必立即建设六个独立服务，但要确保 task/evidence 数据可导出。规模扩大或进入高风险域后，再把 scheduler、credential broker、verifier 和 evolution service 独立扩展。

这套架构允许企业先集成现有 runtime，再逐步替换 context、tool view 或 loop，而无需重建治理与证据系统。下一章把“合同”进一步展开为规范驱动交付。

第二十七章 Agent SDD：规范驱动的任务与发布

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

Agent SDD (Specification-Driven Delivery) 不是要求所有请求先写长文档，而是把材料性意图转换成可执行、可版本化的契约，使自治执行有明确边界。对低风险探索，规范可以渐进；对不可逆、高价值动作，关键不变量必须在执行前冻结。

六类规范

规范	回答的问题	推荐权威载体
业务规范	为什么做、价值是什么	product/业务系统
任务规范	交付物、不变量、截止与预算	CompletionContract
工具规范	可执行动作与错误语义	schema + effect contract
策略规范	谁在何种条件下能做什么	policy-as-code
验证规范	什么证据足以证明完成	verifier suite
发布规范	谁能让候选产生外部效果	release/commit policy

自然语言可以是入口，但金额、资源范围、数据快照、禁止动作、验收与 commit authority 等承重字段需要结构化。否则模型、reviewer 和审计者会分别解释同一句话。

从意图到合同

TEXT

intent → ambiguity/materiality detection → contract draft
→ authority confirmation → executable checks → run

Agent 可以自动补全可发现信息，例如当前 commit、已有测试和 schema；只把会显著改变结果或权限的歧义交给用户。合同编译器应区分 missing、conflicting 与 intentionally_open。刻意开放的设计选择可以留给 Agent，但必须有预算和评价 rubric。

反例是用户说“清理老客户”，系统把“老”解释为 90 天未登录并直接删除账号。正确流程会发现阈值、删除/归档、法律保留和 commit authority 都是材料性歧义，在执行前冻结；探索阶段只能生成影响分析。

完整实例：从规范到任务再到验收

假设仓库 `billing-api` 要修复“取消订阅后仍发送续费提醒”的缺陷。业务 owner 先给出规范：已取消订阅不得进入提醒队列；不能改变账单状态；历史已发送消息不追溯删除；只允许修改通知筛选与对应测试。合同编译器把这些语义映射到可执行任务，而不是把一句工单标题直接交给 Runtime。

YAML

```
specification:
  id: SPEC-BILLING-214
  owner: subscription-product
  snapshot: git:8f2c9d1
  invariant:
    - cancelled_subscription_never_enqueued
    - invoice_state_unchanged
  intentionally_open:
    - implementation_shape

task:
  task_id: TASK-BILLING-214-01
  contract_version: v1
  deliverables:
    - patch_against_git_8f2c9d1
    - evidence_package
```

```

allowed_writes:
- src/reminders/**
- tests/reminders/**
forbidden_actions:
- database_write
- message_send
- billing_state_change
budget:
wall_minutes: 30
max_actions: 80
commit_authority: billing-code-owner

```

验收规范由独立 verifier 执行，并固定输入快照。它不仅检查新增测试，也从领域 fixture 生成 active、past_due、cancelled 三个切片，确认取消状态不入队、其他状态行为不回归，同时比较账单表前后 hash。Agent 可见公开测试和接口契约，但不可读取 sealed cancellation fixture。模型停止后只产生 candidate；clean workspace 应用 patch 后，完成门运行：

YAML

```

acceptance:
- id: compile
  command: make typecheck
  required: true
- id: reminder_regression
  command: pytest tests/reminders
  required: true
- id: sealed_cancelled_slice
  verifier: reminder-contract-v4
  expect: enqueued_count == 0
- id: invoice_integrity
  verifier: table-hash-compare
  expect: before_hash == after_hash
- id: scope_guard
  verifier: changed-path-policy
  expect: changed_paths subset_of allowed_writes

```

若 candidate 通过公开测试，却修改 `src/billing/state.py` 把取消状态改回 active，`scope_guard` 与 `invoice_integrity` 都失败，任务不得完成；“提醒不再出现”不能覆盖业务不变量。若 Agent 发现真正过滤逻辑位于未授权的 `src/queue/subscription_filter.py`，它应提交 amendment，说明所需路径、证据和验证不变，由 code owner 生成 v2。若所有检查通过，EvidencePackage 绑定 [SPEC-BILLING-214](#)、合同 v1、输入 commit、patch hash、verifier 版本和批准者；只有 commit authority 才能合并。这个例子展示了规范、执行自由与发布权的边界：Agent 可以选择实现形态，却不能重写“不发送”“不改账单”和“谁批准”。

运行中的 Amendment

执行中发现新事实可以提交 amendment proposal，例如依赖版本与合同不兼容。执行 Agent 不能单方面扩大写入范围、降低验收或改变数据 snapshot。proposal 包含差异、理由、影响、已发生效果和需要的 authority；批准后产生新 contract version，旧 attempt 与旧版本绑定。

JSON

```

{
  "amendment": "AM-CC2048-02",
  "from": "CC-REPO-2048-v3",
  "change": {"allowed_writes_add": ["src/compat/date_adapter.py"]},
  "reason": "existing API compatibility layer is authoritative",
  "impact": "adds one production file; verification suite unchanged",
  "required_authority": "code_owner"
}

```


Specification as environment

规范应贴近权威状态：代码规则进入仓库，数据口径进入 semantic layer，API 约束进入 schema，安全要求进入 policy engine。只写在 system prompt 的规范难以测试、版本化和复用。context compiler 给模型的是当前规范投影，并保留来源与版本。

规范也不能无限细化。把每个动作都预写成步骤，会把 Agent 退化成昂贵 workflow；完全开放则让完成不可判定。经验法则是：重复、可确定、错误代价高的要求编译成 schema/test/policy；真正需要情境判断的部分交给模型和人。

发布门与 Waiver

候选 artifact 与 contract version 绑定；verifier 生成结果；commit controller 依据 release policy 执行。若业务必须带已知失败上线，waiver 要写明失败检查、风险 owner、补偿措施、影响范围和到期时间。Agent 可以解释 waiver，不得自行批准。

模型停止、候选完成、业务提交是三个不同事件（见第十章）。SDD 的价值正是让它们分别可观察和授权。

规范质量指标

可观测指标包括：运行中材料性 amendment 率、完成后发现的隐含不变量数、无法执行的验收项比例、waiver 逾期率、同合同跨 runtime 结果差异和 contract-to-evidence 覆盖率。高 amendment 率可能说明入口澄清不足；零 amendment 也可能说明团队在聊天里偷偷改目标，需要抽检事件。

当任务探索性极强且没有稳定 verifier 时，可选择 research brief + 人工 review，而不是伪造精确合同。Agent SDD 的适用边界，是组织能否说明谁拥有目标和什么结果算可接受。

第二十八章 成熟度模型与 Build-vs-Buy

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

成熟度模型用于识别下一项控制缺口，不是采购打分或组织荣誉。等级按最弱关键层判断：UI 再好、模型再强，如果生产动作没有独立完成门，系统仍处于较低成熟度。

五级模型

等级	可证明能力	主要风险
L0 对话增强	单轮回答或简单工具可用	无权力边界与完成证据
L1 受控执行	workspace、基础权限、日志和人工提交	恢复、策略与证据薄弱
L2 可验证任务	contract、artifact、独立 verifier	eval 覆盖与运营不足
L3 平台化运行时	多 runtime、durable、租户策略、统一 evidence	复杂度和组合漂移
L4 受控进化	轨迹归因、隔离实验、canary、rollback	优化投机与治理失效

二元自评方法

对每条只回答“有可复查证据/没有”，不要按印象给半分。当前等级是所有低等级硬条件都满足后的最高级；任何高风险任务缺关键条件时，按该任务单独降级。

L1 硬条件：任务在隔离 workspace 执行；身份可追踪；写入与网络有边界；敏感动作需批准；日志能关联 task；用户可取消；外部提交不由聊天文本隐式触发。

L2 硬条件：CompletionContract 版本化；candidate 可封存；verifier 独立运行；artifact 有 hash/provenance；模型停止不等于完成；关键副作用可对账；失败有明确升级点；证据包可由另一人重建。

L3 硬条件：canonical event 与 vendor payload 双轨保存；runtime 能力协商；持久 checkpoint 与取消树；租户隔离测试；credential broker 使用短期凭证；policy/sandbox/profile 版本化；SLO 按任务切片；供应商版本可固定和回滚。

L4 硬条件：mutable surface 明确；evaluator/held-out/policy root 与候选隔离；实验预注册并多 trial；缺失数据规则固定；shadow/canary/rollback 可演练；完整 lineage；自动晋级范围按风险限制；能区分 model 与 Harness 收益。

证据可以是测试报告、事件样例、故障演练、配置或审计记录。“产品文档说支持”不是组织已经实现的证据。

买什么，控制什么

优先购买变化快且有规模效应的能力：前沿模型、成熟 coding runtime、浏览器/计算 sandbox 和通用连接器。采购时评估数据边界、版本固定、事件导出、权限控制、SLA、地域、费用上限和退出成本。

企业差异化与责任不可外包的部分应牢牢控制：任务合同、身份映射、业务策略、凭证代理、领域 verifier、证据与审计、eval 数据、发布门和 runtime abstraction。控制不一定意味着全部自写代码，可以是组织拥有配置、数据、密钥、契约和替换权。

何时自研 Runtime

只有当任务规模足够、现有产品在关键接口受限、定制收益可测、团队能承担安全与运维时，才自研 loop/runtime。模型调用和 shell 很容易；durability、跨平台 sandbox、恢复、兼容、评测和插件生态才是长期成本。

一个反例是因 token 单价差异重写 runtime，却没有计入值班、漏洞修复和模型更新适配。另一个反例是采购“企业 Agent 平台”，但关键事件和 artifact 无法导出，形成证据锁定。总成本模型应包含订阅/调用、基础设施、集成、人工复核、事故期望损失和退出迁移。

决策矩阵与 POC

按任务匹配、控制力、证据性、耐久性、数据风险、总成本、可替换性和演进能力评分。每项先定义可验证问题，例如“进程被杀后是否会重复发送外部动作”，再运行真实故障，而不是询问销售是否“支持恢复”。

POC 使用代表性任务、真实权限边界和完整失败成本，多 trial 比较；至少包含正常完成、歧义升级、工具超时、凭证拒绝、取消、恢复和 verifier 失败。最终建议通常是“买高变化 runtime，建控制面，保留替换权”，随着成熟度再选择性内化 context、tool 或 loop。

第二十九章 从接入现有 Agent 到拥有运行时主动权

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

迁移目标不是“去供应商化”，而是让供应商成为可替换能力组件。平台必须拥有任务定义、权力边界、证据和学习数据；是否自研模型循环是后续经济决策。

迁移按能力门而不是日历推进。下表中的进入判据确认组织具备开始本阶段的前提，退出判据证明新增能力可用，回退条件则防止“已经投入很多”成为继续扩张风险的理由。回退不一定删除已建组件，也可以把自治范围降回上一阶段。

阶段	进入判据	退出判据	回退条件
封装	已盘点主要 Runtime、调用者与凭证路径	canonical event、版本、成本、工作区和取消可观测；契约测试可运行	adapter 丢失关键原始事件，或平台路径稳定性低于原接入
外置完成	至少一个高频任务族有 owner 和可执行验收	Runtime 停止不再直接提交；EvidencePackage 可重建结果	verifier 误判不可控，或 commit authority 仍被 Runtime 旁路
统一执行面	工具、网络、凭证和数据出口已完成威胁建模	关键 effect 全经 gateway，对账、短期凭证与隔离演练通过	敏感工具仍旁路，或 sandbox/gateway 导致材料性业务中断
评测运营	有稳定合同、任务快照与失败分类	多 trial、关键切片、恢复测试和 SLO 形成基线	数据污染、环境不可重建，或指标无法区分能力与风险
逐层替换	基线能比较旧/新组合，旧 bundle 可恢复	目标层收益在相同预算下复现且关键切片非劣	收益无法归因，兼容成本超预算，或快速回滚演练失败
受控进化	trace、归因、sealed eval、canary 与 lineage 稳定	特定低风险表面的候选可被阻断、灰度、回滚并复查	候选可触碰裁判、出现硬门逃逸，或普通配置尚不能可靠回滚

封装而非散接

为 Claude Code、Codex 或其他 Agent 建 adapter，统一 task、attempt、event、artifact、approval 和 cancel。所有调用经过平台身份、workspace 与策略，不允许业务团队在脚本中分散保存长期 token。此阶段不追求抹平所有差异，先双轨保存 canonical event 与原始 payload。

退出条件：能列出每个 runtime 版本、调用者、工作区、费用和外部动作；停止解析彩色终端输出；供应商升级前有契约测试。

外置完成与证据

把 verifier、EvidencePackage 和 commit authority 放到 runtime 外。先选择一个高频任务族，定义可执行 CompletionContract，在 clean environment 重验 candidate。这样即使更换 Agent，业务正确性与审计不随供应商迁移。

退出条件：模型停止不再直接触发 merge/send/deploy；任一完成任务都可从输入 revision 重建 artifact 和检查；错误完成能回标到 task 与 runtime 版本。

统一执行面

建立企业 sandbox、tool gateway、credential broker 和 artifact store。供应商 runtime 决定动作，受控工具提交效果；对无法适配的原生功能保留专用 execution profile，并明确降低自治等级。

失效场景是“一半工具走 gateway，一半插件直连 SaaS”。架构图看似统一，最敏感的 secret 和出网反而旁路。应以网络流、secret 发放和外部审计日志验证覆盖率，而不是只数接入工具。

建立评测与运营基线

从真实任务形成 capability、regression、safety 和 recovery suites，以相同合同、环境和预算比较模型/runtime 的成功、稳定、成本与人工负担。建立第十二、二十六章的 trace 与 SLO；没有基线，自研无法证明价值，供应商切换也无法量化风险。

退出条件：关键任务多 trial；故障注入可重复；报告按任务、风险、模型和 runtime 切片；能执行 Model×Harness 2×2 归因。

逐层替换

先替换最具企业差异化的 context compiler、tool view、policy adapter 或 workflow，再考虑 loop。每次只改变少数层，保留旧 bundle 与快速回滚。不要同时重写 UI、runtime、sandbox 和 eval，否则任何改善或退化都无法归因。

先内化对象	何时值得	不宜内化的信号
Context compiler	企业知识复杂且可测 recall	权威数据尚未治理
Tool gateway	权限/审计是核心要求	只有低风险本地工具
Workflow	任务重复且边界稳定	流程仍频繁人工协商
Agent loop	供应商协议限制关键能力	只是希望省少量 token

引入受控进化

当 trace、归因、eval、canary 和 rollback 稳定后，才自动生成候选 Harness 变更。发布仍由外部治理平面控制；跨任务 memory 先进入隔离 registry；模型训练是更后的选择。若组织尚不能可靠回滚普通配置，就不应自动进化配置。

迁移治理

每阶段维护 capability map、数据出口、供应商依赖、compatibility test 和退场演练。至少每个主要版本演练一次：冻结新任务、导出未完成 attempt、在替代 runtime 重新开始或恢复、重建 evidence、撤销旧凭证。恢复不一定跨 runtime 保留内部思考，但必须保留任务、artifact 和已提交 effect。

最终主动权的判据很简单：供应商暂时不可用或价格变化时，企业是否仍能解释任务状态、保护数据、验证已有结果，并在可控损失下切换。若答案是否定的，即使代码托管在自己账户，也没有真正拥有运行时。

第三十章 展望：Harness OS、Agent 组织与持续进化

证据地位：本章为作者参考设计与工程推导，案例和阈值用于说明实现逻辑，不代表跨组织验证的通用标准。

未来 Harness 会更像 Agent 的操作系统：调度概率执行者，管理上下文与能力，隔离计算，记录副作用，验证结果，并在多版本之间安全演进。这个比喻的价值在职责和不变量，不在复刻传统 OS 的 API。

模型与 Harness 共同设计

工具使用、上下文读取、压缩和长时任务能力会越来越多地进入模型训练；Harness 又会按 model profile 编译工具和上下文。Cursor 公开的模型特化工具、Claude/Codex 的 loop 设计和新一代 Agent SDK 都显示，性能前沿来自模型—Harness 协同，而不是一个永恒通用 prompt。

企业需要双层结构：内部 canonical contract 保持任务、动作和证据语义稳定；model-facing view 可以随模型版本变化。否则追求模型无关会牺牲效果，追求每个模型私有又会失去可比较性和替换权。

从 Agent 应用到 Agent 组织

Agent 会跨越单个聊天，成为有身份、预算、工作区和持续责任的数字执行者。多个 Agent 可按任务动态形成组织，但组织图必须由合同、权限、artifact 和 handoff 定义，而不是角色扮演。第十一章已经说明，多 Agent 增加的既有并行能力，也有协调熵和攻击面。

人的工作从逐步操作转向定义目标、维护规范、处理例外和审查证据。若企业流程仍依赖口头约定、共享账号和不可观测系统，Agent 不会自动修复组织，只会更快放大组织熵。

环境成为长期资产

模型可以采购，企业环境的可机器操作性更难复制：权威数据是否结构化，工具是否有稳定 schema，日志是否可查询，测试是否表达业务不变量，审批是否能被系统调用。Harness engineering 因而不仅是 AI 平台工程，也是组织把隐性制度转成可执行契约的过程。

环境也可能成为负资产。为一个模型堆积的提示补丁、无法撤销的 skill 和不带 provenance 的 memory 会形成 Harness debt。未来平台需要像管理代码依赖一样管理上下文、工具与经验的 owner、版本、测试和退役。

自我进化的近期现实

近期可信形态更可能是自动发现失败、自动提出候选、隔离评测、人或策略批准，而不是生产 Agent 任意改写自己。2026 年 Self-Harness、GSME、Living-Harness 和 HSI 等预印本提供了受限实验信号，也共同暴露反馈质量、基础模型能力、过拟合和评价隔离的边界（见第十九至二十四章）。

随着形式验证、环境模拟和 sandbox 成熟，低风险表面的自动晋级范围会扩大；根信任仍保持外部。真正困难的研究问题不是能否生成修改，而是长期分布漂移下如何可靠 credit、如何防止 evaluator 被优化、如何证明跨版本安全不变量。

新风险与开放问题

长时自治会放大累积小错误；跨 Agent 信息流可能突破原有租户和职能边界；插件与 skill 供应链可把 prompt injection 变成持久代码；评测污染会让系统“学会考试”；经济型 DoS 会用合法工具耗尽预算。多个 Agent 还可能相互强化错误，而没有任何一个单体表现出明显异常。

开放研究至少包括：可组合策略的形式语义；跨 runtime 可移植 checkpoint；不暴露 held-out 的高信息反馈；memory 污染的因果追踪；面向副作用的 Agent benchmark；model—Harness 联合优化中的公平归因；以及自动进化系统自身的安全证明。

三种可能未来

保守路径是 Agent 继续作为强大的交互工具，人始终提交关键动作；平台价值集中在上下文和体验。平台路径是多 runtime 共享企业 control/evidence plane，Agent 成为可调度的执行能力。进化路径是在前者之上形成持续实验系统，低风险 Harness 和 skill 自动晋级，高风险变化保留多方治理。

三条路径会长期共存，取决于任务可验证性与错误代价。并非所有知识工作都应自治，也并非所有组织都应自研 runtime。

最终判断

Agent 的竞争不只是谁拥有最强模型，而是谁能以更少无关上下文、更小权限、更低协调熵，在真实环境中持续产生可验证结果，并把失败转化为受控改进。Harness 将从脚手架变成企业 AI 劳动力的制度与基础设施。

最值得建设的不是一个“永远正确的自治 Agent”，而是一套知道自己何时不确定、能证明完成、能安全失败、能从证据中改善且始终可被治理的系统。

附录

附录 A：语言无关核心契约与安全伪代码

本附录给出语义接口，不要求所有 Runtime 使用同一种语言或序列化格式。平台可以用 JSON、protobuf 或数据库事件实现，但字段所有权与状态转换应保持一致。

核心对象

这组对象用于确定跨 Runtime 的最小语义边界：平台可以增添字段，但不得把 Task、Attempt、Action、Effect 和 Evidence 混成一条聊天记录。常见误用是只保存模型消息，事后从自然语言猜测权限、输入版本和实际副作用；那样既不能安全恢复，也不能证明完成。

TEXT

```
Task {
  id, tenant, contract_version, input_refs[], risk, budget,
  requested_by, commit_authority, status
}
Attempt {
  id, task_id, runtime, runtime_version, harness_profile,
  workspace_ref, policy_profile, started_at, status
}
Action {
  id, attempt_id, actor, type, normalized_args_ref,
  resource, side_effect_class, idempotency_key, provenance
}
Observation {
  action_id, status, structured_result, artifact_refs[],
  diagnostics, environment_revision
}
Artifact {
  uri, hash, media_type, producer, classification,
  input_refs[], created_at, retention_policy
}
PolicyDecision {
  action_id, decision, constraints, policy_version,
  reason_code, approval_request_id?
}
Checkpoint {
  attempt_id, state_version, event_offset, workspace_ref,
  pending_effect_ids[], context_projection_ref
}
VerificationResult {
  contract_version, verifier_version, checks[], status,
  evidence_refs[], environment_ref
}
EvidencePackage {
  task, attempt, inputs[], candidate, effects[], policies[],
  verification, approvals[], final_commit?, lineage
}
```

Run loop：proposal 不直接变成 effect

这段循环用于实现平台拥有的决策—授权—执行—验证骨架，供应商 Agent 可以占据 `model.decide`，却不能绕过策略与完成门。常见误用是把 `final answer` 当作成功，或让模型直接调用 `executor`；两者都会把“提出候选”与“获得外部提交权”混为一谈。

TEXT

```
while attempt.active:
  canonical_state = state_store.load(attempt.id)
  context = context_compiler.project(canonical_state, budget)
  proposal = model.decide(context, model_facing_tool_views)

  if proposal.requests_action:
    action = normalize_validate_and_assign_id(proposal.action)
```

```

decision = policy.evaluate(action, current_authority)
event_store.append(action, decision)

if decision == DENY:
    observation = denied_observation(decision.reason)
elif decision == REQUIRE_APPROVAL:
    suspend_attempt_with_checkpoint(action, decision)
    continue_after_external_response()
else:
    observation = commit_effect_safely(action, decision.constraints)

event_store.append(observation)
state_store.reduce(observation)
continue

candidate = seal(proposal.output, canonical_state.artifacts)
verification = completion_gate.verify(candidate, task.contract_version)
if verification.status == PASS:
    return CANDIDATE_VERIFIED
if verification.repairable and budget.remaining:
    state_store.reduce(minimal_diagnostics(verification))
else:
    return NEEDS_ESCALATION

```

DENY 不调用 executor；**REQUIRE_APPROVAL** 在外部决定前挂起。模型输出无工具调用只表示提出 candidate，不表示业务已提交。

副作用提交：先记 intent，再执行

凡是会改变外部权威状态且可能超时的动作，都应使用这一模式，例如发送、部署、支付和工单更新。它不是数据库事务的万能替代：目标系统若不支持幂等查询，就必须提供业务唯一键、对账 API 或人工 reconciliation，不能在 timeout 后盲目重试。

TEXT

```

commit_effect_safely(action, constraints):
    effect = EffectIntent(
        effect_id = stable_id(action.id),
        idempotency_key = action.idempotency_key,
        target = action.resource,
        requested_operation = constrained(action, constraints),
        status = INTENT_RECORDED
    )
    durable_store.insert_if_absent(effect)

    prior = target_system.lookup(effect.idempotency_key)
    if prior.is_committed:
        outcome = observation_from(prior)
        durable_store.mark_committed(effect.id, outcome.ref)
        return outcome

    durable_store.mark_executing(effect.id)
    outcome = executor.execute(effect.requested_operation)

    if outcome.is_definitive:
        durable_store.mark_final(effect.id, outcome)
        return outcome

    durable_store.mark_unknown(effect.id)
    return Observation(status=UNKNOWN_EFFECT, diagnostics=reconcile_required)

```

存储 effect intent 后、调用目标系统前崩溃，恢复流程能找到待提交记录；目标系统已执行但 outcome 尚未持久化时崩溃，状态为未知，必须查询幂等键或外部审计，不得盲目重试。

恢复、取消与对账

长任务、子任务和外部副作用并存时，恢复与取消必须作为持久状态转换实现，而不是进程控制的附注。常见误用是恢复旧 credential、重复执行未知 effect，或在 UI 标记 cancelled 后留下子进程继续运行；这些都会制造越权或重复提交。

TEXT

```
recover(attempt_id):
    lease = coordinator.acquire_single_owner(attempt_id)
    checkpoint = state_store.latest_checkpoint(attempt_id)
    state = replay_pure_events(checkpoint.event_offset)

    for effect in state.pending_or_unknown_effects:
        authoritative = target_system.lookup(effect.idempotency_key)
        append_reconciliation_observation(effect, authoritative)

    policy = policy_store.load_current_compatible_version()
    credentials = broker.issue_fresh_leases(state.required_capabilities)
    resume_from_reconciled_state(state, policy, credentials)

cancel(attempt_id, reason):
    state_store.mark_cancel_requested(attempt_id, reason)
    scheduler.cancel_children(attempt_id)
    executor.terminate_process_tree(attempt_id)
    revoke_temporary_credentials(attempt_id)
    reconcile_pending_effects(attempt_id)
    state_store.mark_cancelled_when_quiescent(attempt_id)
```

恢复时不复用过期 credential，也不把 checkpoint 中旧授权当成当前授权。取消是一个需要收敛的状态，不是向 worker 发一条尽力而为的信号。

Adapter 的能力协商

CapabilitySet 用于调度前比较任务风险需求与 Runtime 的真实能力，尤其适合同时接入 Claude Code、Codex 与自研 Runtime 的平台。它不是一张营销功能表；no 或未知能力必须导致替代 Runtime、收缩自治范围或人工升级，不能靠空字段伪装兼容。

TEXT

```
CapabilitySet {
    structured_events: yes/no
    pause_for_approval: yes/no
    resume: none/session/checkpoint
    cancel: cooperative/process_tree
    artifact_export: list of media types
    workspace_isolation: local/worktree/container/vm/provider
    raw_event_provenance: yes/no
}
```

Control Plane 依据 Task 风险声明要求；adapter 返回实际能力。若关键能力缺失，scheduler 选择替代 Runtime、降低自治范围或要求人工，不能把 no 转成空字段继续执行。

附录 B：可判定的 Harness 架构评审表

使用方法：每项必须附一个可复查 artifact（配置、测试、事件或演练报告），只回答“通过/不通过/不适用”。“不适用”需要风险 owner 说明。任何 R3/R4 动作若关键项不通过，不应进入生产自治。

任务与完成

检查项	合格判据	不合格的典型症状	依据
目标是否版本化	合同含交付物、不变量、预算、权限、验收和 owner	目标只在聊天里，运行中静默变化	第十、二十七章
停止是否与完成分离	runtime stop 产生 candidate；独立 completion gate 决定完成	final answer 直接触发 merge/send/deploy	第十章
是否生成证据包	输入、artifact hash、检查、策略和批准可关联	只有最终文本或截图	第十、二十五章
外部提交是否独立授权	commit authority 与执行 Agent 分离	Agent 自报成功后自动生效	第十、二十六章

上下文与记忆

检查项	合格判据	不合格的典型症状	依据
上下文 provenance 可见	每个承重片段有来源、版本、选择原因和 token 成本	不知道规则从哪里注入	第七章
压缩不覆盖权威状态	task、effect、artifact 保存在上下文外	compaction 后遗忘约束或重复动作	第六、七章
Memory 有写入门	candidate、验证、scope、TTL、owner、撤销齐全	一次成功自动写入全局 memory	第二十一章
检索先做隔离	租户、权限、数据分类在语义检索前过滤	相似度搜索跨租户返回内容	第七、二十一章

工具、环境与副作用

检查项	合格判据	不合格的典型症状	依据
Action schema 稳定	参数、错误分类、版本、截断和 artifact 语义明确	全部失败都是字符串 error	第八章
Effect 可对账	intent 先持久化，有幂等键，未知结果进入 reconcile	timeout 后直接重试发送/支付	第六章、附录 A
Workspace 可重建	输入 revision、image、依赖和初始化可固定	“在 Agent 那台机器上能过”	第四、六章
Sandbox 经对抗验证	文件、网络、进程、mount、资源和身份都有测试	只因使用 Docker 就声称隔离	第九、十七章

权限与供应链

检查项	合格判据	不合格的典型症状	依据
身份、授权、批准分层	actor identity、capability、decision、approver 可追踪	登录成功被当作拥有全部权限	第九章
凭证短期且不进上下文	broker 在提交时注入 lease，日志做 secret scan	token 出现在 prompt、trace 或 skill scan	第九、二十六章
委派缩权	子任务 capability 与预算不超过父任务	subagent 继承宿主所有 secret	第十一章
扩展供应链可撤销	MCP/skill/plugin 有 owner、版本、权限、签名和 kill switch	自动更新未审查脚本	第九、二十一章
Prompt injection 有系统测试	间接注入、数据外泄和跨域 flow 进入 safety suite	只测试模型口头拒绝	第九、二十三章

Durable 与多 Agent

检查项	合格判据	不合格的典型症状	依据
中断可恢复	kill/restart 演练无状态丢失和重复 effect	worker 崩溃后从头运行	第六章
取消能收敛	子进程、子任务、凭证和 pending effect 被处理	UI 显示取消，后台仍执行	第六、十一章
并行写入隔离	分支有独立写集，合并后重验	多 Agent 共享目录互相覆盖	第十一、十四章
Handoff 有结构化交付	目标、已做、artifact、未决、权限和预算齐全	只返回“已完成”摘要	第十一章

Eval、运营与进化

检查项	合格判据	不合格的典型症状	依据
Eval 集生命周期分开	development、validation、sealed、regression 有 owner 与访问审计	hidden test 被用于日常修 prompt	第十二章
报告多 trial 与切片	固定任务/环境/预算，报告区间、成本和关键切片	只报一次最好成绩	第十二、十八章
Trace 骨架完整	policy/effect/checkpoint/verification 全量，artifact 可读取	成功有日志、失败无日志	第十二章
候选与裁判隔离	mutable surface 明确，候选不可写 evaluator/held-out/policy root	Agent 可修改测试或分母	第十九、二十四章
发布可灰度与回滚	task 版本粘性，bundle shadow/canary，回滚演练成功	只会换回 prompt 文件	第二十四章
Lineage 可解释	父版本、mutation、数据、评测、批准、事故和退役齐全	无法回答“为何上线”	第二十四章

最终判定

- 阻断：高风险任务在身份/授权、effect 对账、隔离、完成门、证据或回滚任一项不通过。
- 限域上线：核心安全项通过，但恢复、评测覆盖或运营证据不足；只允许低风险、可人工提交的任务。
- 生产候选：所有适用关键项有证据，并完成正常、拒绝、超时、取消、崩溃、恢复和 verifier 失败演练。

评审结果应记录适用范围和到期时间。一个代码只读 Agent 的通过结论不能自动继承给可写生产数据库的 Agent。

附录 C：术语与本体边界

本书固定以下用法。产品文档可能采用不同名称，adapter 应映射语义，而不是仅按字符串对齐。

- Model：接收有限上下文并提出文本或动作的概率性策略；不天然拥有持久状态、权限和外部真值。
- Agent：在任务范围内由模型动态选择观察或动作的执行者。Agent 是系统角色，不等于单次模型调用。
- Agent System：Model、Harness、Environment 与 Feedback 的完整组合，是能力与风险的实际评价对象。
- Harness：把任务、模型与环境组织成持续执行的逻辑控制系统，负责 loop、context、tools、state、policy、verification、observability 与 evolution governance。
- Agent Runtime：承载 Agent loop、session 和模型交互的运行组件，如供应商 CLI/core 或自研 loop。
- Execution Runtime：实际运行命令、浏览器、代码或连接器环境，如容器、VM 或受控远程执行器。
- Environment：Agent 可观察或改变的任务世界，包括 workspace、数据库、SaaS、日志和人类组织；不等于一个 shell。
- Feedback：改变系统对动作或任务质量判断的信号；Observation 只有进入评价时才成为 feedback。
- Workflow：由代码预定义主要控制路径的执行结构；可在节点中调用模型，但模型不拥有全部路由权。
- Control Plane：拥有 task、identity、policy、调度、配置与发布权威的逻辑平面。
- Evidence Plane：保存 artifact、trace、effect 和独立验证结果的逻辑平面；不依赖聊天历史证明完成。
- Evolution Plane：生成、评价和发布 memory/Harness/model 候选的系统；受治理平面约束。
- ACI：Agent-Computer Interface，模型与计算环境之间的动作和观察接口。
- Action：Agent 提议的规范化动作；在授权和执行前还不是现实副作用。
- Observation：动作、环境或策略返回的可观察结果，带状态、诊断与 artifact 引用。
- Effect：已经或可能改变外部权威状态的动作结果。
- Effect Ledger：记录 effect intent、幂等键、提交状态、outcome 与 reconciliation 的账本。
- CompletionContract：目标、交付物、不变量、验收、证据、权限、预算与停止条件的版本化合同。
- Candidate：Agent 提交给外部完成门的候选 artifact；尚未获得业务提交权。
- VerificationResult：特定 verifier 在固定环境下对合同检查的结构化结果。
- EvidencePackage：连接输入、candidate、artifact、effect、policy、verification、approval 与最终提交的机器可读证据。
- Artifact：有地址、hash、媒体类型、生产者和分类的持久交付或中间对象。
- Checkpoint：恢复所需的任务状态、事件 offset、workspace 与 pending effect 引用；不等于上下文摘要。
- Compaction：将长上下文转换为可继续推理的较短表示；是有损投影，不是长期记忆。
- Memory：跨推理或跨任务保存的事实、情景、程序或策略状态；必须声明 scope、owner 和生命周期。
- Skill：按需加载的程序知识包，可能含指令、脚本和资源；属于软件供应链对象。
- Handoff：工作责任从一个 Agent/节点转移到另一个，携带结构化目标、状态、artifact、权限和未决项。
- Capability lease：绑定 actor、资源、动作、purpose、租户和 TTL 的临时授权。
- Held-out / sealed test：候选不可见、由独立评价服务在预定时机使用的数据或检查。

- Canary：在受限真实流量和影响范围内部署候选版本并监控。
- Harness evolution：对 prompt、工具、上下文、路由、工作流或 runtime profile 的受控优化，不等于模型权重训练。
- Reward hacking：提高测量分数却偏离真实目标或破坏评价完整性的行为。
- Lineage：版本从父项、数据、mutation、实验到发布、事故和退役的可追溯关系。

最容易混淆的三组边界是：Agent Runtime 决定下一步，Execution Runtime 执行动作；Memory 保存跨时经验，Compaction 只压缩当前上下文；Harness 可以包含 policy adapter，但根授权和 release authority 不应由候选 Harness 自行修改。

附录 D：概念首次定义与使用索引

本索引用于定位概念，不替代正文定义。

概念	首次集中定义	主要展开章节
Model × Harness × Environment × Feedback	第五章	第十八、二十三、二十六章
Agent / Workflow	第二、五章	第十一、二十七章
Harness	第五章	第六至十二、十九、二十六章
Agent Runtime / Execution Runtime	第五章	第十三至十七、二十六章
Control/Data/Execution Plane	第五章	第九、二十六章
Durable state machine	第六章	第十一、二十六章
Checkpoint	第六章	第七、二十章、附录 A
Effect Ledger	第六章	第十、十一、二十六章、附录 A
Reconciliation	第六章	第十、二十、二十五章
Context compiler	第七章	第十五、二十二、二十九章
Compaction	第七章	第十二至十五章
Memory	第七章	第十九、二十一、二十四章
Skill	第七、八章	第十三、二十一、二十二章
ACI	第三章	第四、八章
Action / Observation	第三、六章	第十七、二十六章、附录 A
MCP	第八章	第九、十三、十五章
Code Mode	第八章	第十六、二十二章
Capability lease	第九章	第十一、二十六章
Sandbox	第九章	第十三至十七、二十六章
Credential broker	第九章	第二十五、二十六、二十九章
CompletionContract	第十章	第二十五、二十七章
EvidencePackage	第十章	第二十四至二十六章、附录 A
Commit authority	第十章	第二十五至二十七章
Delegation / Handoff	第十一章	第十三、二十六章
Trace / Event graph	第十二章	第十四、二十四、二十六章
Capability / Regression eval	第十二章	第十八、二十二、二十九章
Runtime adapter	第十四、十八章	第二十五、二十六、二十九章
四层进化模型	第十九章	第二十至二十四章
Mutable surface / Root of trust	第十九章	第二十二、二十四章
Model × Harness 2×2	第十九、二十三章	第二十九章
Shadow / Canary	第二十四章	第二十五、二十六章
Lineage	第二十四章	第二十五、二十六章
Agent SDD	第二十七章	第二十五、二十九章
L0 – L4 成熟度	第二十八章	第二十九章

若未来章节改变承重概念的定义，应同时更新本索引、术语表与机器可读契约；不要在新章节中给同一术语引入第二套隐含语义。

附录 E：最小机器可读契约

下面的 JSON Schema 是教学用最小子集，展示如何把正文对象变成可校验协议。生产实现应拆分 schema、使用稳定 URI、补充 classification 枚举、兼容规则和签名；不要把示例中的字段数量误作完整规范。

Task 与 CompletionContract

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.internal/harness/task-v1.schema.json",
  "title": "Task",
  "type": "object",
  "additionalProperties": false,
  "required": ["task_id", "tenant", "contract_version", "risk", "deliverables", "checks",
    "commit_authority"],
  "properties": {
    "task_id": {"type": "string", "minLength": 1},
    "tenant": {"type": "string", "minLength": 1},
    "contract_version": {"type": "string", "minLength": 1},
    "risk": {"enum": ["R0", "R1", "R2", "R3", "R4"]},
    "input_refs": {"type": "array", "items": {"type": "string"}},
    "deliverables": {"type": "array", "minItems": 1, "items": {"type": "string"}},
    "invariants": {"type": "array", "items": {"type": "string"}},
    "forbidden_actions": {"type": "array", "items": {"type": "string"}},
    "checks": {"type": "array", "minItems": 1, "items": {"type": "string"}},
    "budget": {
      "type": "object",
      "properties": {
        "wall_seconds": {"type": "integer", "minimum": 1},
        "model_usd": {"type": "number", "minimum": 0},
        "max_actions": {"type": "integer", "minimum": 1}
      }
    },
    "commit_authority": {"type": "string", "minLength": 1}
  }
}
```

Action、PolicyDecision 与 Observation

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$defs": {
    "Action": {
      "type": "object",
      "required": ["action_id", "attempt_id", "actor", "type", "resource",
        "side_effect_class"],
      "properties": {
        "action_id": {"type": "string"},
        "attempt_id": {"type": "string"},
        "actor": {"type": "string"},
        "type": {"type": "string"},
        "normalized_args_ref": {"type": "string"},
        "resource": {"type": "string"},
        "side_effect_class": {"enum": ["NONE", "REVERSIBLE", "COMPENSATABLE",
          "IRREVERSIBLE"]}
      },
      "idempotency_key": {"type": "string"}
    },
    "PolicyDecision": {
      "type": "object",
      "required": ["action_id", "decision", "policy_version"],
      "properties": {

```

```

    "action_id": {"type": "string"},
    "decision": {"enum": ["ALLOW", "DENY", "REQUIRE_APPROVAL", "CONSTRAINED_ALLOW"]},
    "policy_version": {"type": "string"},
    "reason_code": {"type": "string"},
    "constraints": {"type": "object"}
  }
},
"Observation": {
  "type": "object",
  "required": ["action_id", "status"],
  "properties": {
    "action_id": {"type": "string"},
    "status": {"enum": ["OK", "DENIED", "ERROR", "TIMEOUT", "UNKNOWN_EFFECT",
"CANCELLED"]},
    "artifact_refs": {"type": "array", "items": {"type": "string"}},
    "diagnostics": {"type": "object"},
    "environment_revision": {"type": "string"}
  }
}
}
}
}

```

EvidencePackage 必填骨架

YAML

```

evidence_package:
  schema_version: evidence-package/v1
  package_id: required
  task:
    task_id: required
    contract_version: required
  attempt:
    attempt_id: required
    runtime: required
    runtime_version: required
    harness_profile: required
  inputs:
    - uri: required
      hash: sha256-required
  candidate:
    uri: required
    hash: sha256-required
  effects: []
  policy_decisions:
    artifact_ref: required
  verification:
    verifier_version: required
    environment_ref: required
    status: PASS|FAIL|INCONCLUSIVE
    checks: []
  approvals: []
  final_commit: null
  lineage:
    parent_attempt: optional
    model_version: required
    harness_bundle: required

```

Schema 只能保证形状，不能证明语义正确。[checks](#) 是否覆盖业务目标、hash 指向的 artifact 是否可信、approval 是否来自有权主体，仍需 policy、verifier 和签名基础设施保证。

研究方法局限

本书采用官方文档、开源仓库、论文和社区材料的分层证据法。全书资料维护至 2026-08-28；无法验证的内部实现不作为事实。设计原则是作者基于多来源的综合推断。研究资产包括 sources.jsonl、evidence.jsonl 与人工标注的 claims_v2.jsonl；旧 claims.jsonl 仅作历史迁移参考。局限包括产品快速迭代、公开 benchmark 污染、厂商数据选择偏差，以及部分 2026 年进化论文尚缺长期生产复现。

完整参考文献

- 机构/作者未登记 (n.d.). DeepSeek Harness official repository
- 机构/作者未登记 (n.d.). DeepSeek Harness Architecture
- 机构/作者未登记 (n.d.). Cordis Primer
- 机构/作者未登记 (n.d.). A Programming Paradigm for Spatiotemporal Composability
- 机构/作者未登记 (n.d.). Self-Harness: Harnesses That Improve Themselves
- 机构/作者未登记 (n.d.). Self-Evolving Agent Harnesses via Gated Semantic Quality-Diversity
- 机构/作者未登记 (n.d.). Living-Harness Is an Interactive-Agent Evolver
- 机构/作者未登记 (n.d.). Hierarchical Self-Improvement: A Framework for Task-Specific Evolvable Agent Harnesses
- 机构/作者未登记 (n.d.). Unrolling the Codex agent loop
- 机构/作者未登记 (n.d.). Unlocking the Codex harness: how we built the App Server
- 机构/作者未登记 (n.d.). Introducing the Codex app
- 机构/作者未登记 (n.d.). Harness engineering: leveraging Codex in an agent-first world
- 机构/作者未登记 (n.d.). OpenAI Codex official repository
- 机构/作者未登记 (n.d.). Continually improving our agent harness
- 机构/作者未登记 (n.d.). Dynamic context discovery
- 机构/作者未登记 (n.d.). What we learned building cloud agents
- 机构/作者未登记 (n.d.). Cursor Developer Habits Report Spring 2026
- 机构/作者未登记 (n.d.). OpenHands Runtime Architecture
- 机构/作者未登记 (n.d.). OpenHands: An Open Platform for AI Software Developers as Generalist Agents
- 机构/作者未登记 (n.d.). Gemini CLI official repository
- 机构/作者未登记 (n.d.). OpenCode official repository
- Earendil Works / Mario Zechner (2026). Pi coding agent official README
- 机构/作者未登记 (n.d.). ReAct: Synergizing Reasoning and Acting in Language Models
- 机构/作者未登记 (n.d.). Toolformer: Language Models Can Teach Themselves to Use Tools
- 机构/作者未登记 (n.d.). Reflexion: Language Agents with Verbal Reinforcement Learning
- 机构/作者未登记 (n.d.). Voyager: An Open-Ended Embodied Agent with Large Language Models
- Richard Fikes; Nils Nilsson (1971). STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving
- Reid G. Smith (1980). The Contract Net Protocol
- 机构/作者未登记 (2020). BDI Agent Architectures: A Survey
- 机构/作者未登记 (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
- 机构/作者未登记 (2026). Pi coding agent official repository

- 机构/作者未登记 (n.d.). GitHub repository metadata for OpenHands
- 机构/作者未登记 (n.d.). GitHub repository metadata for Gemini CLI
- 机构/作者未登记 (n.d.). GitHub repository metadata for OpenCode
- 机构/作者未登记 (n.d.). GitHub repository metadata for Pi
- 机构/作者未登记 (2023). Original BabyAGI archived implementation
- 机构/作者未登记 (2023). AutoGPT official repository
- 机构/作者未登记 (2023). Autonomous Agents and Agent Simulations
- 机构/作者未登记 (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation
- 机构/作者未登记 (2025). Building LangGraph: Designing an Agent Runtime from First Principles
- 机构/作者未登记 (n.d.). Aider Repository Map
- 机构/作者未登记 (n.d.). Aider GPT Code Editing Benchmarks
- 机构/作者未登记 (n.d.). Executable Code Actions Elicit Better LLM Agents
- 机构/作者未登记 (n.d.). AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents
- 机构/作者未登记 (n.d.). Building Effective Agents
- 机构/作者未登记 (n.d.). How the Claude Agent SDK loop works
- 机构/作者未登记 (n.d.). How Claude Code works
- 机构/作者未登记 (n.d.). Idempotency and retries in durable execution
- 机构/作者未登记 (2024). Lost in the Middle: How Language Models Use Long Contexts
- 机构/作者未登记 (2023). MemGPT: Towards LLMs as Operating Systems
- 机构/作者未登记 (n.d.). How Claude remembers your project
- 机构/作者未登记 (n.d.). Explore the Claude Code context window
- 机构/作者未登记 (n.d.). Model Context Protocol Architecture
- 机构/作者未登记 (n.d.). Model Context Protocol Schema Reference
- 机构/作者未登记 (n.d.). MCP Authorization
- 机构/作者未登记 (n.d.). Scale to many tools with tool search
- 机构/作者未登记 (n.d.). DeepSeek Harness Tool Runtime and Code Mode
- 机构/作者未登记 (n.d.). Beyond permission prompts: Claude Code sandboxing
- 机构/作者未登记 (n.d.). Codex ExecPolicy
- Carlos E. Jimenez et al. (2023). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?
- OpenAI (2024). Introducing SWE-bench Verified
- OpenAI (2026). Why SWE-bench Verified no longer measures frontier coding capabilities
- Anthropic (2026). Demystifying evals for AI agents

- Koki Wataoka, Tsubasa Takahashi, Ryokan Ri (2024). Self-Preference Bias in LLM-as-a-Judge
- Lin Shi et al. (2024). Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge
- Jonathan Gabor, Jayson Lynch, Jonathan Rosenfeld (2025). EvilGenie: A Reward Hacking Benchmark
- Bingchen Zhao et al. (2026). SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents
- Anthropic (2025). How we built our multi-agent research system
- Qingyun Wu et al. (2024). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation
- Sirui Hong et al. (2023). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework
- OpenAI (2025). A practical guide to building AI agents
- OpenAI (2026). OpenAI Agents SDK Handoffs
- Anonymous/Research authors (2025). Why Do Multi-Agent LLM Systems Fail?
- Haolun Wu, Zhenkun Li, Lingyao Li (2025). Can LLM Agents Really Debate? A Controlled Study of Multi-Agent Debate in Logical Reasoning
- Kunlun Zhu et al. (2025). MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents
- Anthropic (2026). Extend Claude Code
- Anthropic (2026). Automate actions with hooks
- Cursor (2026). Cursor Agent Security
- Cursor (2026). Cursor Background Agents
- Cursor (2026). Cursor Hooks
- DeepSeek AI (2026). DeepSeek Harness Tool Catalog
- OpenAI (2026). OpenAI Agents SDK Agents
- Model Context Protocol (2025). Model Context Protocol Changelog 2025-11-25
- Microsoft Research (2026). AutoGen Publications